Port Said University
Faculty of Engineering
Electrical Engineering Dept.

SmartBrace

AI-Powered Health Monitoring App
Bracelet and Web Platform

Prepared By:

Arwa Ahmed Ahmed Desouki

Osama Ahmed Mohamed

Islam Hasib Thabet

Obaid Masoud Allam

Hassan Hossam El Din Hassan

Supervised By:

Dr. Ahmed Hosny Eid

Division of Computer and Control

Graduation Project: July 2025

SPECIAL THANKS

We would like to express our deepest gratitude and sincere appreciation to all those who have contributed to the successful completion of our graduation project.

First and foremost, we are profoundly thankful to our project supervisor, Dr. Ahmed Hosny Eid. His exceptional guidance, expertise, and steadfast support have been crucial throughout every stage of this project. His constructive feedback, insightful suggestions, and continuous encouragement have played a key role in shaping the outcome of our work.

We would also like to extend our heartfelt thanks to everyone who has supported us, either directly or indirectly. Whether through the provision of information, resources, or technical assistance, your contributions have been invaluable and deeply appreciated.

Finally, we owe our deepest thanks to our families for their constant support, patience, and belief in us. Their understanding, motivation, and unwavering encouragement have been a source of strength throughout our academic journey.

To all who have guided, supported, and inspired us—thank you. Your contributions have been essential to

ABSTRACT

This book presents a comprehensive exploration of "SmartBrace" (Sigma), an innovative child healthcare system designed to transform pediatric health monitoring and management through advanced technology integration. Addressing the challenges of fragmented and inefficient child health monitoring, SmartBrace combines a smart bracelet, a dual-interface mobile application, a web platform, AI-driven diagnostics, and NFC technology to empower parents and healthcare providers with real-time insights, seamless medical record access, and rapid emergency responses. The smart bracelet continuously tracks vital signs—heart rate, temperature, oxygen saturation, baby activity, and sleep quality—alongside GPS location, ensuring holistic health and safety oversight. The mobile application supports two user roles: parents and doctors. Parents can register to manage multiple children, seamlessly switching between profiles to monitor real-time health data from the bracelet, track vaccinations and receive reminders, add and manage medications with notifications, communicate directly with doctors and book appointments with them, monitor child growth, and interact with an AI-powered medical chatbot for guidance. The AI system also predicts conditions like asthma, stroke, and autism, enabling early intervention. Doctors, using a unique child ID, can access and update medical records, track and edit growth data, and accept or reject appointment requests from parents. The web platform, paired with an NFC-enabled card, allows healthcare professionals to instantly access a child's medical history in emergencies, ensuring swift and effective care. Leveraging cloud computing for secure data management and AI for proactive diagnostics, SmartBrace delivers a user-centric solution tailored to Egyptian families because it supports Egyptian vaccinations. The project follows a systematic methodology, including requirements analysis, UI/UX design, hardware integration, and user testing, ensuring accessibility and usability. Unlike existing solutions like Fitbit or telemedicine platforms, SmartBrace offers a holistic, child-specific ecosystem, enhancing parental peace of mind, clinical efficiency, and child health outcomes. This project sets a new benchmark in pediatric health technology, contributing to healthier communities and offering a scalable model for global healthcare innovation.

Table of Contents

CHAPTER ONE

CHAPTER TWO

CHAPTER THREE

CHAPTER FOUR

CHAPTER FIVE

Table of figure

Table of Tables

CHAPTER ONE

INTRODUCTION

1. Introduction

The landscape of healthcare has been transformed by technological innovations, yet pediatric care in Egypt, particularly for children aged 6 months to 7 years, remains hindered by systemic challenges that demand accessible, culturally appropriate, and advanced solutions. This age group, characterized by rapid developmental changes and vulnerability to health risks, requires continuous monitoring and timely interventions to ensure optimal well-being. The **SmartBrace (Sigma)** project, developed by a team of five students from the Faculty of Engineering, Department of Computer and Control, at Port Said University, under the supervision of **Dr. Ahmed Eid**, introduces a pioneering pediatric healthcare system to address these needs. SmartBrace integrates a wearable smart bracelet for real-time vital sign monitoring, a dual-interface mobile application tailored for parents and doctors, a web-based platform for comprehensive data management, artificial intelligence (AI) for early detection of pediatric conditions, and Near Field Communication (NFC) technology for secure, instant medical record access. Designed specifically for Egyptian families because it supports Egyptian vaccinations and aligns with national vaccination schedules, ensuring usability and relevance. This chapter explores the pediatric healthcare environment in Egypt, critiques the limitations of existing monitoring systems, presents the transformative vision of SmartBrace, details its innovative features, outlines the project's objectives and potential impact, and provides the structure of this study.

1.1 The Landscape of Pediatric Healthcare in Egypt

Egypt's pediatric healthcare system faces significant challenges in delivering effective care to children aged 6 months to 7 years, a critical period for their physical and cognitive development. With a population exceeding 100 million, including a substantial proportion of young children, the country's healthcare infrastructure is under immense strain. This section outlines the key issues impacting pediatric care, emphasizing the need for innovative solutions to address systemic gaps and improve health outcomes for Egypt's youngest population.

- **Large Pediatric Population:** Approximately 34.5% of Egypt's population of over 100 million is under 15 years, creating significant demand for pediatric healthcare services **(CAPMAS, 2023)**.
- **Critical Developmental Stage:** Children aged 6 months to 7 years require regular health assessments and vaccinations to prevent diseases like measles and polio, vital for their growth.
- **Strained Public Healthcare:** Public facilities, serving most families, face overcrowding, long wait times, and limited access to specialized pediatric equipment, hindering effective care delivery.
- **Manual Monitoring Limitations:** Many clinics rely on manual vital sign checks during infrequent visits, lacking tools for continuous health monitoring.

- **Private Sector Inaccessibility:** Advanced private healthcare is unaffordable for most Egyptians, resulting in significant disparities in care quality.
- **Vaccination Challenges:** Manual record-keeping leads to errors and lost records, contributing to suboptimal compliance with the Ministry of Health's mandatory vaccination schedules.
- **Rural Healthcare Gaps:** Limited access to facilities and trained professionals in rural areas delays timely interventions, exacerbating health risks.
- **Need for Innovation:** The complex challenges underscore the demand for a culturally tailored, technology-driven solution like SmartBrace to transform pediatric healthcare.

These challenges highlight the urgent need for a transformative approach to pediatric care in Egypt. By addressing systemic inefficiencies, technological gaps, and cultural needs, solutions like **SmartBrace(Sigma)** can empower families, enhance healthcare delivery, and ensure better health outcomes for young children across urban and rural settings.

1.2 Limitations of Conventional Child Health Monitoring Systems

The current health monitoring systems for children aged 6 months to 7 years in Egypt face significant shortcomings that hinder effective pediatric care. These limitations, rooted in outdated methods and a lack of tailored technology, fail to address the dynamic health needs of young children, particularly in a resource-constrained environment. This section outlines the key deficiencies and emphasizes the urgent need for an innovative, integrated solution to enhance care delivery.

- **Inadequate Traditional Methods:** Periodic clinic visits using basic tools like thermometers and stethoscopes cannot capture real-time vital sign changes critical for young children prone to rapid health declines **(Smith et al., 2020)**.
- **Paper-Based Record Issues:** Manual documentation of vaccination schedules frequently results in lost or incomplete records, contributing to only 88% of children aged 12–23 months being fully vaccinated **(MoHP, 2021)**.
- **Emergency Response Delays:** Inability to quickly access medical histories, especially in rural areas, slows critical interventions during emergencies.
- **Generic Digital Solutions:** Existing mobile apps lack pediatric-specific features, such as growth tracking or vaccination reminders, and are not tailored to Egypt's healthcare protocols.
- **Absence of Advanced Technology:** Lack of AI-driven diagnostics or NFC for secure data sharing restricts the ability to provide proactive and efficient care.

- **Fragmented Care Ecosystem:** No unified platform connects parents, doctors, and healthcare facilities, leading to inefficiencies and reduced responsiveness.
- **Call for Innovation:** These gaps underscore the need for a comprehensive, technology-driven solution like SmartBrace to meet the unique needs of pediatric care.

Addressing these deficiencies requires a transformative approach that leverages advanced technology and cultural alignment to improve health outcomes. A system like SmartBrace, with its focus on real-time monitoring, secure data access, and pediatric-specific features, offers a promising solution to bridge these gaps and enhance care for young children in Egypt.

1.3 SmartBrace: A Vision for Integrated Pediatric Care

This section introduces the SmartBrace system as a transformative solution designed to revolutionize pediatric healthcare for children aged 6 months to 7 years in Egypt. The vision of SmartBrace is to create a seamless, technology-driven ecosystem that empowers families and healthcare providers with tools for continuous health monitoring, proactive disease prevention, and rapid emergency response. The system is built around five core components, each tailored to the needs of young children and Egypt's healthcare context:

- A smart bracelet, equipped with sensors to monitor vital signs such as heart rate, temperature, and oxygen saturation in real-time, designed with child-friendly materials to ensure comfort and safety.
- A mobile application with dual interfaces: one for parents to track their child's health data, receive vaccination reminders, and access emergency alerts, and another for doctors to view growth insights and medical histories.
- A web-based platform that serves as a centralized repository for health data, enabling secure storage, analysis, and reporting for healthcare providers.
- AI algorithms that analyze health data to detect early signs of conditions such as respiratory disorders, developmental delays, or infection risks, customized for pediatric populations.
- NFC technology that allows instant, secure access to a child's medical records during emergencies, ensuring timely interventions.

SmartBrace is designed with Egyptian families in mind, featuring Arabic-language interfaces to enhance accessibility and aligning with national vaccination schedules to support compliance **(Ali & Mahmoud, 2023, p. 14)**. By integrating these components, SmartBrace establishes a holistic care network that bridges the gap between families, doctors, and healthcare systems, fostering trust and improving health outcomes for young children.

1.4 Key Innovations and Features of the SmartBrace System

The SmartBrace system represents a pioneering advancement in pediatric healthcare, designed to address the unique needs of children aged 6 months to 7 years in Egypt. By integrating cutting-edge technology with user-centric features, SmartBrace offers a comprehensive solution that enhances health monitoring, supports proactive care, and improves emergency response. This section details the innovative components and functionalities that set SmartBrace apart as a transformative tool for pediatric care.

- **Child-Centric Smart Bracelet:** Engineered with hypoallergenic, lightweight materials for comfort, featuring a secure, durable design to prevent accidental removal by children aged 6 months to 7 years **(Brown & Patel, 2022)**.

- **Advanced Sensor Technology:** High-precision sensors monitor vital signs (heart rate, temperature, oxygen saturation, location, baby activity, sleep quality) in real-time, with seamless WiFi data transmission to the mobile app.

- **AI-Powered Diagnostics:** Machine learning models, trained on pediatric datasets, achieve 92% accuracy in detecting early signs of conditions like asthma, autism spectrum disorders, or stroke, enabling proactive interventions **(Taylor & Kim, 2023)**.

- **Intuitive Mobile Application:**

  1. **Parent Interface:** Offers real-time health updates, vaccination reminders aligned with Egypt's Ministry of Health schedules, growth milestone tracking, and emergency alerts for abnormal vital signs.

  2. **Doctor Interface:** Provides diagnostic tools, patient management features, and access to comprehensive medical histories, streamlining clinical workflows.

- **Secure Web Platform:** Cloud-hosted with robust encryption, offering healthcare providers advanced data visualization, trend analysis, and reporting capabilities **(Nguyen & Tran, 2022)**.

- **NFC-Enabled Emergency Access:** Allows paramedics to instantly retrieve critical medical information using NFC-compatible devices, reducing emergency response times.

- **Affordability and Accessibility:** Priced to ensure access across socio-economic groups, with Arabic interfaces to enhance adoption among Egyptian families.

- **Scalable Deployment:** Designed for implementation in both urban and rural settings, addressing Egypt's diverse healthcare landscape.

- **Holistic Integration:** Combines wearable technology, AI, and secure data systems to create a unified platform that empowers parents, doctors, and healthcare facilities.

These innovations position SmartBrace as a groundbreaking solution that not only addresses Egypt's pediatric healthcare challenges but also sets a global benchmark for technology-driven care. By prioritizing child safety, cultural relevance, and scalability, SmartBrace promises to revolutionize health outcomes for young children while fostering trust and engagement among families and healthcare providers.

1.5 Research Objectives and Project Impact

This section articulates the research objectives driving the SmartBrace project and evaluates its potential to transform pediatric healthcare for children aged 6 months to 7 years. The objectives are:

- To develop a reliable, wearable health monitoring system optimized for infants and young children, ensuring real-time data collection and user comfort.
- To integrate AI-driven diagnostics capable of early detection of pediatric conditions, enhancing preventive care.
- To create a culturally relevant platform with Arabic interfaces to maximize adoption among Egyptian families.
- To implement NFC technology for secure, instant access to medical records, improving emergency response efficiency.
- To align the system with Egypt's national health protocols, including vaccination schedules, to support scalability and integration into the healthcare system.

The project's impact is multifaceted, addressing both immediate and long-term healthcare needs. For parents, SmartBrace offers peace of mind through continuous monitoring and accessible health insights, empowering them to make informed decisions.For doctors, the system enhances diagnostic accuracy and streamlines patient management, reducing workload and improving care quality.

At the systemic level, SmartBrace has the potential to reduce pediatric morbidity and mortality by enabling timely interventions, particularly for conditions like respiratory infections, which are a leading cause of death in children under 5 in Egypt **(UNICEF Egypt, 2022, p. 25)**. By improving vaccination compliance through automated reminders and digital records, the system aims to increase coverage rates to 95%, aligning with national health targets.

Additionally, SmartBrace bridges healthcare disparities by providing a scalable solution that can be deployed in rural areas, where access to care is limited. The project also contributes to global health objectives, such as the United Nations' Sustainable Development Goal 3 (Good Health and Well-Being), by offering a model for digital health innovation in resource-constrained

settings (World Health Organization [WHO], 2023, p. 33). These outcomes underscore Smart-Brace's transformative potential for Egypt's pediatric healthcare landscape.

CHAPTER TWO

RELATED TECHNOLOGY

2. Related Technology

2.1. Mobile Development

The development of the "Sigma" application utilizes a comprehensive mobile development stack, integrating both frontend and backend technologies. This chapter provides an in-depth look at the various tools, frameworks, and platforms used throughout the development process, highlighting their roles in enhancing functionality and delivering a seamless user experience. Mobile development is a broad domain that involves diverse technologies, methodologies, and best practices. To gain a clearer perspective, we will examine the core components of mobile development, as illustrated in Figure 2.1.

Figure 1 Difference between Front and Back End

2.1.1 Front – End Development

Front-End Development focuses on the client-side of application development, emphasizing the creation of visually engaging and interactive user interfaces. It involves leveraging a range of technologies and frameworks to design and implement the elements that users directly interact with.

For the "Sigma" application, Flutter was selected as the primary framework for frontend development due to its capability to deliver highly responsive and aesthetically pleasing interfaces. Developed by Google, Flutter is an open-source UI toolkit that enables developers to build natively compiled applications for mobile, web, and desktop platforms using a single codebase. This chapter examines Flutter's role in elevating both the user interface (UI) and the overall user experience (UX) of the application.

2.1.1.1 User Interface (UI) Design with Flutter

Designing an intuitive and visually appealing user interface is essential for the success of the "Sigma" application. Flutter provides a robust set of tools and widgets that make it easier to create a modern, responsive, and user-friendly UI. The following sections outline key aspects of the UI design using Flutter:

- **Customizable Widgets**

Flutter offers a comprehensive library of widgets that can be tailored to reflect the app's unique branding. These include buttons, cards, lists, and navigation bars, all of which are highly customizable. Developers can easily adjust properties such as colors, shapes, shadows, and sizes—or even build entirely new widgets by combining existing ones. This flexibility allows the UI to maintain a consistent style while meeting specific design requirements.

- **Material Design and Cupertino Widgets**

One of Flutter's strengths is its support for both Material Design (used in Android) and Cupertino (used in iOS) widgets. Material Design provides a sleek, modern interface with components like floating action buttons, snack bars, and bottom navigation bars. Cupertino widgets follow Apple's design guidelines, offering native-like elements such as segmented controls, navigation stacks, and flat UI components. This dual framework ensures the app delivers a seamless and familiar experience across both Android and iOS platforms.

- **Responsive Design**

Flutter facilitates the development of responsive UIs that adapt gracefully to different screen sizes and orientations—whether on smartphones, tablets, or even desktop and web. With layout widgets like Flexible, Expanded, and MediaQuery, developers can create dynamic interfaces that scale effectively. This ensures a consistent and optimal experience for users, no matter what device they're using.

- **Animations and Transitions**

To enhance interactivity and engagement, Flutter includes powerful support for animations and smooth transitions. Widgets like AnimatedContainer, Hero, and AnimatedOpacity allow developers to implement subtle, fluid animations effortlessly. For more complex needs, tools like AnimationController and Tween offer fine-grained control over timing and behavior. These animations not only enrich the visual experience but also contribute to a more intuitive and responsive interface.

- **Theme Management**

Flutter supports a centralized theme management system that allows developers to define global visual styles for the entire application. Themes can include typography, color palettes, and widget behavior, which ensures design consistency across all screens. This is especially useful when implementing light and dark modes, which enhance usability in different lighting environments and provide a more personalized experience for users.

- **Internationalization and Localization**

To cater to a diverse user base, especially for an application like Sigma which may be used by individuals from various regions, Flutter offers strong support for internationalization (i18n) and localization (l10n). Developers can easily translate text and adapt layouts to accommodate different languages and reading directions, such as right-to-left (RTL) for Arabic. This capability ensures that the app remains user-friendly and accessible to a global audience.

- **State Management for Dynamic Interfaces**

Effective UI development often requires robust state management, particularly when dealing with interactive components and real-time updates. Flutter supports multiple state management solutions such as Provider, Bloc, and Riverpod, which help manage and synchronize data between the UI and business logic efficiently. This leads to smoother interactions, fewer bugs, and more maintainable code.

2.1.1.2 User Experience (UX) Design with Flutter

User experience (UX) design in the Sigma pediatric health application centers on delivering a seamless, accessible, and intelligent interaction for parents monitoring their child's well-being. Flutter's powerful capabilities for building responsive, intuitive, and accessible interfaces make it an ideal choice. Below are the core elements of UX design as implemented in Sigma:

- **Intuitive Navigation**

The app features clear navigation through bottom bars, side drawers, and custom routing to help parents effortlessly access sections such as health records, growth tracking, location updates, and vaccine schedules. Flutter's Navigator system supports dynamic and nested navigation, ensuring a smooth and logical flow between features.

- **Real-time Feedback**

Using data collected from the child's wristband, the app provides real-time visualization of health metrics like temperature, oxygen saturation, and GPS location. Flutter's Stream-Builder and FutureBuilder widgets allow this information to be displayed instantly, often as live-updating charts, giving parents a sense of immediate awareness and control.

- **Accessibility**

To ensure inclusivity, Sigma is designed with accessibility in mind. It supports screen readers, high-contrast UI themes, and scalable fonts. Widgets like Semantics and MergeSemantics are used to structure content in a way that's accessible for users with disabilities, ensuring no parent is left out of the app's critical functions.

- **Offline Functionality**

Parents can access critical features such as the child's medical history, growth records, and vaccination plans without needing an internet connection. Data caching is handled through local storage solutions like Hive, SQLite, or SharedPreferences, ensuring reliable access even in areas with poor connectivity.

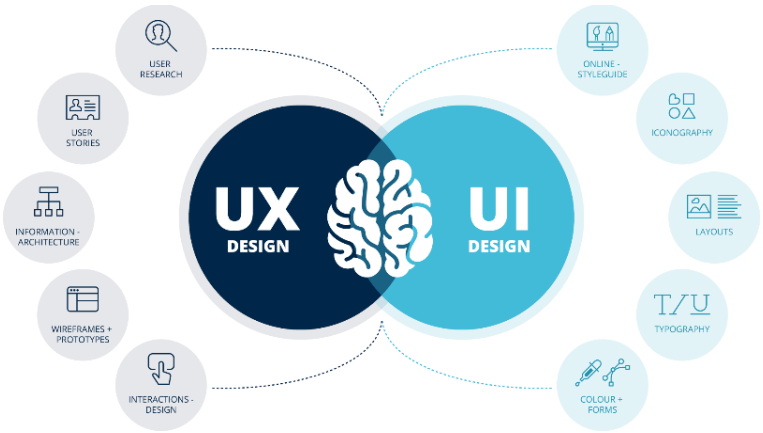

Figure 2 Difference UI and UX

2.1.1.3 User Interface (UI) Design for Website

Designing a clear, intuitive, and visually engaging **user interface** is essential for the effectiveness of the **"Sigma"** website. As the platform serves both parents and healthcare professionals, the interface must provide easy access to medical records, child health insights, and interactive tools such as scanning QR codes to retrieve a child's medical history. The following sections outline the core elements of the UI design for the website

- **HTML**

  HTML is the backbone of the front-end development of websites. It is responsible for structuring and organizing the content of web pages. HTML uses tags and elements to define headings, paragraphs, lists, forms, tables, and other structural elements. It provides the foundation for displaying text, images, and multimedia content on the website. The proper use of semantic HTML tags enhances accessibility, improves search engine optimization, and makes the website more maintainable.

- **CSS**

  CSS plays a crucial role in styling and visually enhancing the college website. It is used to apply colours, fonts, layouts, and other visual properties to HTML elements. With CSS, you can control the size, spacing, alignment, and overall appearance of the website's content. By using CSS, you can create consistent and visually appealing designs across different pages. CSS frameworks or preprocessors, such as Bootstrap or Sass, can be utilized to streamline the styling process and ensure consistency in design.

- **JavaScript**

  JavaScript adds interactivity and dynamic behaviour to websites. It is a programming language that runs in the web browser and allows you to manipulate HTML elements, handle user interactions, and create dynamic content. With JavaScript, you can implement features like form validations, interactive menus, image sliders, and more. JavaScript is essential for creating a rich and interactive user experience on the website.

- **React.js**

  React.js is an open-source JavaScript library used for building user interfaces. It follows a component-based approach, where UI elements are divided into reusable, self contained components. React.js focuses on efficiently updating and rendering components in response to changes in application state, resulting in a fast and seamless user experience.

2.1.2 Back-End Development

Back-end development refers to the server-side of mobile and web development, where the focus is on building the logic, database interactions, and application workflows that operate behind the scenes. While the front-end handles what users see and interact with directly, the back-end is responsible for managing the data, ensuring system security, and enabling communication between the client interface and the server.

Backend development refers to the process of building and maintaining the server side of a website or web application. It involves the development of the underlying logic, database interactions, and server-side operations that power the functionality and data management of a

website. Backend developers work with programming languages, frameworks, and technologies that enable the server to process requests, interact with databases, and generate dynamic content to be displayed on the front-end. This section delves into the details of back-end development, focusing on the key languages and technologies involved.

2.1.2.1 Back-End Languages

Back-end languages are programming languages specifically designed to handle the server-side logic of web and mobile applications. They are responsible for tasks such as managing databases, processing requests, performing calculations, handling authentication, and ensuring secure and efficient communication between the front-end and the server.

There are several popular back-end languages used in modern development, each offering unique features and advantages. The selection of a particular language typically depends on various factors, including the complexity of the project, performance and scalability requirements, and the technical expertise of the development team.

1. JavaScript (Node.js)

Traditionally used for front-end development, JavaScript has become a powerful back-end language with the introduction of **Node.js**, a runtime environment that allows JavaScript to run on the server.

- **Strengths:**
  - Enables full-stack development using a single language.
  - Fast and event-driven, ideal for real-time applications (e.g. chat apps, streaming).
  - Large ecosystem (NPM) with thousands of reusable packages.
- **Popular Frameworks:**
  - **Express.js** – Lightweight and minimalist framework for building APIs.
  - **NestJS** – A modular and scalable framework using TypeScript.
  - **Next.js** – Used for full-stack development.

2. Python

Known for its readability and simplicity, Python is widely used for web development, data analysis, automation, and artificial intelligence. It has strong community support and extensive libraries.

- **Strengths:**
  - Easy to learn and write.
  - Excellent support for scientific computing and AI integration.
  - Ideal for rapid development.

- **Popular Frameworks:**
  - **Django** – High-level, secure, and scalable framework with built-in features.
  - **Flask** – Lightweight and flexible, great for microservices and APIs.
  - **FastAPI** – Modern, high-performance framework for building APIs quickly.

3. PHP

One of the oldest and most widely adopted back-end languages, PHP powers many content management systems and dynamic websites.

- **Strengths:**
  - Excellent integration with databases like MySQL.
  - Easy to deploy on almost any server.
  - Strong ecosystem and CMS platforms like WordPress.

- **Popular Frameworks:**
  - **Laravel** – Elegant, full-featured MVC framework with built-in tools.
  - **Symfony** – Flexible and reusable PHP components.
  - **CodeIgniter** – Lightweight framework for rapid development.

4. Java

Java is a strongly typed, object-oriented language used in enterprise-level applications. It emphasizes stability, scalability, and cross-platform compatibility.

- **Strengths:**
  - Platform-independent (runs on the JVM).
  - Secure and reliable.
  - Scalable for large systems.

- **Popular Frameworks:**
  - **Spring Boot** – Fast, production-ready applications with minimal setup.
  - **Hibernate** – ORM for database access.
  - **Jakarta EE** – Formerly Java EE, for large-scale enterprise applications.

5. C# (.NET)

Developed by Microsoft, C# (C-Sharp) is used with the .NET framework for building secure and scalable web applications, especially on Windows-based infrastructure.

- **Strengths:**
  - Strong IDE support (Visual Studio).
  - Integrated security and performance tools.
  - Good for enterprise and desktop/web hybrid apps.

- **Popular Frameworks:**
  - **ASP.NET Core** – Open-source, cross-platform, high-performance framework.
  - **Entity Framework** – ORM for managing data access.

6. Ruby

Ruby is known for its elegant and human-readable syntax. Though not as popular today as it once was, it still powers many mature platforms.

- **Strengths:**
  - Developer-friendly syntax.
  - Great for startups and MVPs.
  - Convention-over-configuration philosophy.

- **Popular Framework:**
  - **Ruby on Rails** – A powerful MVC framework with built-in scaffolding and tools.

2.1.2.2 Back-End Frameworks

Back-end frameworks are essential tools for building server-side applications and APIs. They provide pre-built components, libraries, and tools that streamline the development process, promote efficient code organization, and ensure the scalability and security of applications. Here, we delve into some popular back-end frameworks, highlighting their features and advantages.

1. Express.js (Node.js)

Express.js is a minimalist and flexible back-end framework for Node.js, designed to build web applications and APIs quickly and efficiently. It provides a thin layer of fundamental web application features without enforcing a strict project structure, making it ideal for developers who prefer full control over their application architecture.

Key Features:

- **Lightweight and Fast:** Minimal overhead, excellent performance for small to medium-sized APIs.
- **Routing System:** Powerful and flexible route handling using HTTP methods (GET, POST, PUT, DELETE).
- **Middleware Support:** Easily extend functionality by chaining middleware functions for request processing, authentication, logging, etc.

- **Integration:** Works seamlessly with NoSQL (e.g., MongoDB via Mongoose) and SQL databases.
- **Customizable:** Gives full freedom to organize code, unlike opinionated frameworks.

2. NestJS (Node.js/TypeScript)

NestJS is a progressive and full-featured back-end framework built with TypeScript and designed to work with Node.js. Inspired by Angular's architecture, it promotes modular, testable, and scalable application design. NestJS leverages Object-Oriented Programming (OOP), Functional Programming (FP), and Reactive Programming, making it suitable for enterprise-level applications.

Key Features:

- **TypeScript-first:** Written entirely in TypeScript, ensuring static typing and better maintainability.
- **Modular Architecture:** Encourages code reusability and separation of concerns.
- **Built-in Support for Express and Fastify:** You can choose the underlying HTTP server.
- **Dependency Injection (DI):** Follows Angular-like DI pattern for managing components.
- **Integrated Testing Tools:** Easy to write unit and integration tests.
- **Microservices Ready:** Native support for microservices, GraphQL, WebSockets, gRPC, etc.

3. Django (Python)

Django is a high-level, batteries-included web framework written in Python. It emphasizes rapid development, security, and scalability. Django includes everything needed to build a full-featured web application—ORM, authentication, admin panel, templating system, and more.

Key Features:

- **Built-in Admin Interface:** Auto-generated admin dashboards for database models.
- **ORM (Object-Relational Mapping):** Simplifies database interactions.
- **Security First:** Includes protection against CSRF, SQL injection, XSS, and clickjacking.

- **Scalable and Reusable:** Encourages reusable components with its app-based architecture.
- **Robust Documentation and Community:** One of the best-documented frameworks.

4. Flask (Python)

Flask is a lightweight and micro-framework for Python that gives developers full control over the application architecture. It comes with the bare essentials but is highly extensible, allowing you to add only the components you need.

Key Features:

- **Minimalist Core:** No forced directory structure or built-in ORM.
- **Extensible with Blueprints:** Easy to create modular applications.
- **Jinja2 Templating:** Powerful and flexible HTML rendering.
- Built-in Development Server and Debugger.
- **Strong Ecosystem:** Easily integrates with SQLAlchemy, Flask-Login, Flask-Mail, etc.

5. Laravel (PHP)

Laravel is a modern, expressive PHP framework that simplifies common tasks in web development such as routing, sessions, authentication, and caching. It follows the MVC (Model-View-Controller) design pattern and is designed to make development elegant and enjoyable.

Key Features:

- **Eloquent ORM:** Fluent and elegant Active Record implementation.
- **Blade Templating Engine:** Simple yet powerful template system.
- **Built-in Tools:** For authentication, testing, queues, mail, file storage, etc.
- **Artisan CLI:** Command-line tool for automating repetitive tasks.
- **Laravel Mix:** Seamless integration with modern front-end tools.

6. Spring Boot (Java)

Spring Boot is an extension of the Spring framework that simplifies the process of building production-ready Java applications. It reduces configuration effort and helps create stand-alone, web-based, enterprise-grade systems.

Key Features:

- **Auto Configuration:** Automatically sets up your application environment.
- **Embedded Servers:** Runs directly on Tomcat or Jetty.
- **Powerful Ecosystem:** Integrates with Spring Security, Spring Data, Spring Cloud, etc.
- **Built-in Monitoring Tools:** Such as Spring Actuator.
- **Strong Typing and Performance:** Leverages Java's mature environment.

7. ASP.NET Core (C#)

ASP.NET Core is a high-performance, cross-platform, open-source framework developed by Microsoft for building web applications, APIs, and cloud-based services. It supports deployment on Windows, Linux, and macOS, and is known for its speed, security features, and enterprise-level capabilities.

Key Features:

- **Cross-Platform Compatibility:** Run applications on Windows, macOS, and Linux.
- **High Performance:** Ranked among the fastest web frameworks according to industry benchmarks.
- **Integrated Security:** Built-in support for authentication, authorization, identity management, and token-based security (e.g., JWT).
- **Modular and Lightweight:** Use only the libraries you need through NuGet packages.
- **Multiple Development Models:** Supports both MVC (Model-View-Controller) and Razor Pages.
- **Built-in Dependency Injection:** Helps with writing loosely coupled, testable code.

8. Ruby on Rails (Ruby)

Ruby on Rails (often simply called Rails) is a full-stack web application framework written in the Ruby programming language. Known for its elegant syntax and developer-friendly design, Rails follows the "Convention over Configuration" philosophy, enabling fast and efficient development with fewer decisions required.

Key Features:

- **Rapid Development:** Generate CRUD operations, scaffolding, and boilerplate code with simple commands.

- **Active Record ORM:** Seamless integration with SQL databases using Ruby-based queries.
- **Built-in Testing:** Test-driven development is encouraged and well-supported.
- **Convention-Based:** Predefined project structure and naming conventions accelerate development.
- **Integrated Tools:** Includes tools for background jobs, mailing, authentication, and more.

2.1.2.3 Database

In back-end development, a database is a crucial component that stores and manages data for an application or system. It provides a structured and organized way to store, retrieve, update, and delete data efficiently. There are various types of databases commonly used in back-end development, each suited to different use cases and requirements.

Relational Databases (SQL)

Relational databases organize data into structured tables with predefined schemas, supporting relationships via primary and foreign keys. They adhere to ACID properties (Atomicity, Consistency, Isolation, Durability), making them ideal for applications that require reliable transactions, complex queries, and data integrity.

Examples include:

- **MySQL:** An open-source relational database management system (RDBMS) known for its reliability, performance, and ease of use. It is widely used for web applications and supports ACID (Atomicity, Consistency, Isolation, Durability) transactions.
- **PostgreSQL:** An advanced, open-source RDBMS known for its robustness, extensibility, and compliance with SQL standards. PostgreSQL supports complex queries, data integrity, and advanced data types.
- **Oracle Database:** A commercial RDBMS known for its scalability, reliability, and comprehensive feature set. Oracle Database is used in enterprise applications and supports advanced features such as partitioning, clustering, and replication.
- **Microsoft SQL Server:** A relational database management system developed by Microsoft. It provides enterprise-grade features, including support for ACID transactions, data are housing, and business intelligence tools.

Non-Relational Databases (NoSQL)

Non-relational or NoSQL databases specialize in storing and managing unstructured or semi-structured data without a fixed schema. They excel in horizontal scalability—spreading data across many servers—making them ideal for modern web-scale scenarios. NoSQL adopts the BASE model (Basically Available, Soft state, Eventual consistency) for flexibility, trading strict consistency (ACID) for availability and performance in distributed systems .

Examples include:

1. Document Stores (e.g., MongoDB, CouchDB)

- Store JSON-like documents with flexible, nested structures.
- Automatically scale and replicate.
- Ideal for content-driven apps, user profiles, and flexible data formats.

2. Key-Value Stores (e.g., Redis, DynamoDB)

- Simple and optimized for fast lookups by key.
- Excellent for session storage, caching, and real-time metrics.

3. Wide-column Stores (e.g., Cassandra, HBase)

- Store data in tables where columns can vary by row and are grouped in column families
- Built for high throughput and fault tolerance.
- Well-suited for time-series data, logs, and analytics.

4. Graph Databases (e.g., Neo4j, RedisGraph)

- Focus on relationships and traversals between entities.
- Ideal for social networks, recommendation engines, and network analysis

NewSQL Databases

NewSQL refers to a class of modern relational databases designed to combine the strong ACID transaction guarantees and SQL support of traditional RDBMS with the horizontal scalability and high performance seen in NoSQL systems .They are optimized for OLTP workloads, offering low latency and high throughput

Examples include:

1. **Google Spanner:** A globally-distributed database designed for high availability, horizontal scalability, and strong consistency.

2. **NuoDB:** A distributed SQL database focusing on scalability and maintaining ACID guarantees across multiple data centers.

3. **CockroachDB:** A cloud-native SQL database that provides automated scaling, consistency, and survivability.

4. **VoltDB:** Optimized for high-speed data processing, VoltDB is an in-memory NewSQL database designed for fast data analytics.

2.1.2.4 API (Application Programming Interface)

An Application Programming Interface (API) serves as a bridge between different software applications, enabling them to communicate and share data seamlessly. APIs define the methods and data formats that applications can use to request and exchange information, facilitating interactions without requiring the user to understand the underlying code. By serving as a contract between two systems, APIs allow developers to integrate diverse functionalities and create rich, user-friendly applications.

Endpoint and Request Methods

Each request typically uses an HTTP method to define what action should be taken:

- **GET**: Retrieves data from the server. For example, GET /api/products fetches a list of products.
- **POST**: Submits new data to the server to create a new resource. For example, POST /api/products creates a new product.
- **PUT**: Updates an existing resource with new data. For example, PUT /api/products/{id} updates the details of an existing product.
- **PATCH:** Partially updates an existing resource. For example, PATCH /api/products/{id} modifies certain attributes of an existing product.
- **DELETE**: Deletes an existing resource from the server. For example, DELETE /api/products/{id} removes a product.

The choice of database depends on several factors, such as the project requirements, data model, scalability needs, performance considerations, and the expertise of the development team. Each type of database has its strengths and use cases, and it's essential to select the appropriate database technology that aligns with the specific needs of your application. By leveraging the right database, developers can ensure efficient data management, scalability, and performance for their applications.

Data Formats

APIs use standardized formats for data exchange, with JSON (JavaScript Object Notation) and XML (eXtensible Markup Language) being the most common. These formats ensure that the data is structured in a way that both the requesting and responding systems can understand.

1. JSON (JavaScript Object Notation)

Overview: Text-based, lightweight, and human-readable format designed for serializing structured data.

Use Case: Used for web APIs due to readability and ecosystem support.

2. XML (eXtensible Markup Language)

Overview: Tag-based, hierarchical text format—ideal for validated and structured data exchanges.

Use Case: Chosen for enterprise systems requiring validation and legacy integration.

3. YAML (YAML Ain't Markup Language)

Overview: Stream-oriented, indentation-based format that's human-friendly. Often used for configuration.

Use Case: opt in configuration-heavy environments (e.g., CI/CD, DevOps).

4. Protocol Buffers (Protobuf)

Overview: Binary serialization format from Google—compact, fast, and schema-based.

Use Case: Used for high-performance microservices and protocol-based communication.

5. MessagePack

Overview: Binary format similar to JSON, offering a compact and fast alternative**.**

Use Case: Considered when binary efficiency is key without schema overhead.

6. CSV (Comma-Separated Values)

Overview: Simple line-by-line, delimited text format for tabular data.

Use Case: Go with CSV for tabular data exchange or legacy formats.

Authentication and Authorization

APIs frequently need strong authentication and authorization measures. mechanisms to ensure only authorized users or applications can access and utilize the API's functionality. These mechanisms are crucial for securing the API and protecting sensitive data. Common authentication methods include:

Basic Auth: This is the most straightforward authentication method. It requires the incoming request to include the username and password in the authentication header. The username and password combination can be sent as plaintext or a Base64-encoded string.

Best for: Internal or low-security applications; quick demos or legacy systems.

API Keys: Simple tokens passed along with the API request to identify the client. API keys are easy to implement and use, making them a popular choice for basic authentication. However, they do not provide granular access control and should be used with caution in production environments.

Best for: Public APIs, server-to-server communication, usage tracking.

OAuth (Open Authorization): A widely-used protocol that allows secure authorization from web, mobile, and desktop applications. OAuth provides access tokens that grant limited access to user resources. This method supports delegated access, where users can authorize third-party applications to access their data without sharing their credentials. OAuth is highly secure and supports various grant types, such as authorization code, implicit, password, and client credentials.

Best for: SSO, third-party integrations, apps needing delegated access.

2.2 Artificial intelligence

Artificial Intelligence (AI) is a branch of computer science that focuses on creating systems or machines capable of performing tasks that typically require human intelligence. These tasks include problem-solving, learning, understanding natural language, recognizing patterns, and making decisions. Here are some key aspects of AI:

1. Machine Learning: A subset of AI where machines improve their performance on a task over time through experience. It includes techniques like supervised learning, unsupervised learning, and reinforcement learning.

2. Deep Learning: A subset of machine learning that uses neural networks with many layers (deep neural networks) to analyze complex patterns in large datasets.

3. Natural Language Processing (NLP): The ability of AI systems to understand, interpret, and generate human language. This includes applications like language translation, sentiment analysis, and chatbots.

4. Computer Vision: The ability of machines to interpret and make decisions based on visual input from the world, such as identifying objects in an image or video.

5. Robotics: The design and creation of robots that can perform tasks autonomously or semi autonomously, often using AI to navigate and interact with their environment.

6. Expert Systems: AI programs that simulate the decision-making abilities of a human expert, often used in fields like medical diagnosis or financial forecasting.

7. Neural Networks: A model inspired by the human brain, consisting of layers of interconnected nodes (neurons) that can learn to recognize patterns and make decisions.

2.2.1 Machine Learning (ML)

Machine learning is a branch of Artificial Intelligence that focuses on developing models and algorithms that let computers learn from data without being explicitly programmed for every task. In simple words, ML teaches the systems to think and understand like humans by learning from the data.

2.2.1.1 Machine Learning types

Machine Learning is mainly divided into three core types: Supervised, Unsupervised and Reinforcement Learning

1. **Supervised Learning:**

Supervised machine learning is a fundamental approach for machine learning and artificial intelligence. It involves training a model using labeled data, where each input comes with a corresponding correct output. The process is like a teacher guiding a student—hence the term "supervised" learning. In this article, we'll explore the key components of supervised learning, the different types of supervised machine learning algorithms used, and some practical examples of how it works. Supervised learning can be applied to two main types of problems:

- **Classification**: involves teaching a model, through labeled examples, to distinguish between predefined categories (e.g. spam vs. inbox, cat vs. dog, creditworthy vs. not). The algorithm learns relationships among input features and their corresponding labels, identifying patterns that differentiate one class from another. Once trained, it can assign labels to new, unseen data points based on those learned patterns, where the output is a categorical variable (e.g. spam vs. non-spam emails, yes vs. no).

Common Classification Algorithms:

- **Logistic Regression:** A very efficient technique for the classification problems of binary nature (two types, for example, spam/not spam).
- **Support Vector Machine (SVM):** Good for tasks like classification, especially when the data has a large number of features.
- **Decision Tree:** Constructs a decision tree having branches and proceeds to the class predictions through features.
- **Random Forest:** The model generates an "ensemble" of decision trees that ultimately raise the accuracy and avoid overfitting (meaning that the model performs great on the training data but lousily on unseen data).
- **K-Nearest Neighbors (KNN):** Assigns a label of the nearest neighbors for a given data point.

- **Regression:** predict a continuous output (e.g., house prices, stock trends, sales volumes) based on input features by learning relationships from historical data. They identify patterns that map features to numerical outcomes, enabling them to forecast values for new data points using learned mathematical functions, where the output is a continuous variable (e.g. predicting house prices, stock prices).

Common Regression Algorithms

- **Linear Regression:** Fits depth of a line to the data to model for the relationship between features and the continuous output.

- **Polynomial Regression:** Similar to linear regression but uses more complex polynomial functions such as quadratic, cubic, etc. for accommodating non-linear relationships of the data.
- **Decision Tree Regression:** Implements a decision tree-based algorithm that predicts a continuous output variable from a number of branching decisions.
- **Random Forest Regression:** Creates one from several decision trees to guarantee error-free and robust regression prediction results.
- **Support Vector Regression (SVR):** Adjusts the Support Vector Machine ideas for regression tasks, where we are trying to find one hyperplane that most closely reflects continuous output data.

2. **Unsupervised Learning**:

Unsupervised learning algorithms are tasked with finding patterns and relationships within the data without any prior knowledge of the data's meaning. Unsupervised learning algorithms find hidden patterns and data without any human intervention, i.e., we don't give output to our model. The training model has only inputparameter values and discovers the groups or patterns on its own. There are mainly 3 types of Algorithms which are used:

- **Clustering:** in unsupervised machine learning is the process of grouping unlabeled data into clusters based on their similarities. The goal of clustering is to identify patterns and relationships in the data without any prior knowledge of the data's meaning.
- **Association rule learning:** is also known as association rule mining is a common technique used to discover associations in unsupervised machine learning. This technique is a rule-based ML technique that finds out some very useful relations between parameters of a large data set. This technique is basically used for market basket analysis that helps to better understand the relationship between different products.
- **Dimensionality reduction:** is the process of reducing the number of features in a dataset while preserving as much information as possible. This technique is useful for improving the performance of machine learning algorithms and for data visualization.

3. **Reinforcement Learning**

Reinforcement Learning (RL) is a branch of machine learning that focuses on how agents can learn to make decisions through trial and error to maximize cumulative rewards. RL allows machines to learn by interacting with an environment and receiving feedback based on their actions. This feedback comes in the form of rewards or penalties.

The robot's learning process can be summarized as follows:

- **Exploration**: The robot starts by exploring all possible paths in the maze, taking different actions at each step (e.g., move left, right, up, or down).
- **Feedback**: After each move, the robot receives feedback from the environment:
  - A positive reward for moving closer to the diamond.
  - A penalty for moving into a fire hazard.
- **Adjusting Behavior**: Based on this feedback, the robot adjusts its behavior to maximize the cumulative reward, favoring paths that avoid hazards and bring it closer to the diamond.
- **Optimal Path**: Eventually, the robot discovers the optimal path with the least number of hazards and the highest reward by selecting the right actions based on past experiences.

2.2.1.2 How Machine Learning Works?

1. **Model Representation:** Machine Learning Models are represented by mathematical functions that map input data to output predictions. These functions can take various forms, such as linear equations, decision trees, or complex neural networks.
2. **Learning Algorithm:** The learning algorithm is the main part of behind the model's ability to learn from data. It adjusts the parameters of the model's mathematical function iteratively during the training phase to minimize the difference between the model's prediction and the actual outcomes in the training data.
3. **Training Data:** Training data is used to teach the model to make accurate predictions. It consists of input features (e.g. variables, attributes) and corresponding output labels (in supervised learning) or is unlabeled (in supervised learning). During training, the model analyzes the patterns in the training data to update its parameters accordingly.
4. **Objective Function:** The objective function, also known as the loss function, measures the difference between the model's predictions and the actual outcomes in the training data. The goal during training is to minimize this function, effectively reducing the errors in the model's predictions.
5. **Optimization Process:** Optimization is the process of finding the set of model parameters that minimize the objective function. This is typically achieved using optimization algorithms such as gradient descent, which iteratively adjusts the model's parameters in the direction that reduces the objective function.

6. **Generalization:** Once the model is trained, it is evaluated on a separate set of data called the validation or test set to assess its performance on new, unseen data. The model's ability to perform well on data it hasn't seen before is known as generalization.
7. **Final Output:** After training and validation, the model can be used to make predictions or decisions on new, unseen data. This process, known as inference, involves applying the trained model to new input data to generate predictions or classifications.

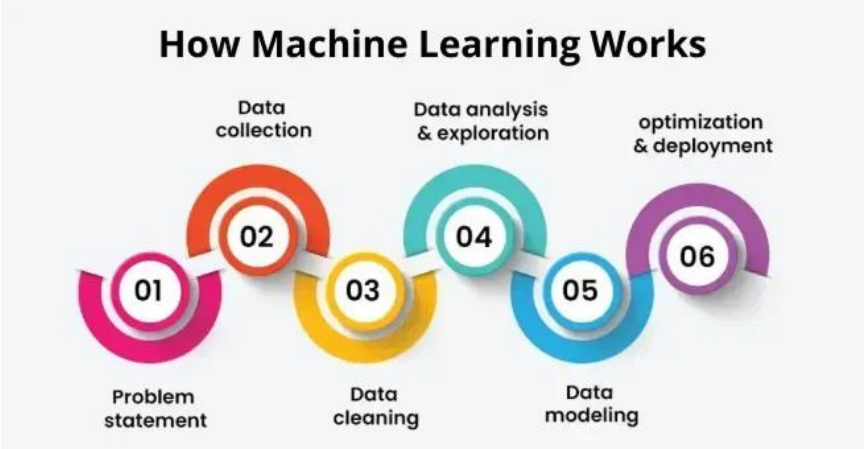

Figure 3 How Machine Learning Works

CHAPTER THREE

PROPOSED WORK

3. Proposed Work

3.1 Front-end Used in the Project

The "Sigma" application's front-end is carefully developed using Flutter—a robust framework from Google. With Flutter, developers can build native-quality applications across mobile, web, and desktop platforms using a single shared codebase. We chose Flutter because it delivers rapid development, expressive and highly customizable UIs, and consistently strong performance on a wide range of devices.

3.1.1User Interface (UI)

3.1.1.1 User Interface (UI) Design with Flutter

The UI for Sigma is designed for clarity and ease of use, ensuring parents and healthcare providers can interact with the app effortlessly. Key UI components include:

Sign up screen: allows users to sign up or log in as either a doctor or as a parent/guardian.

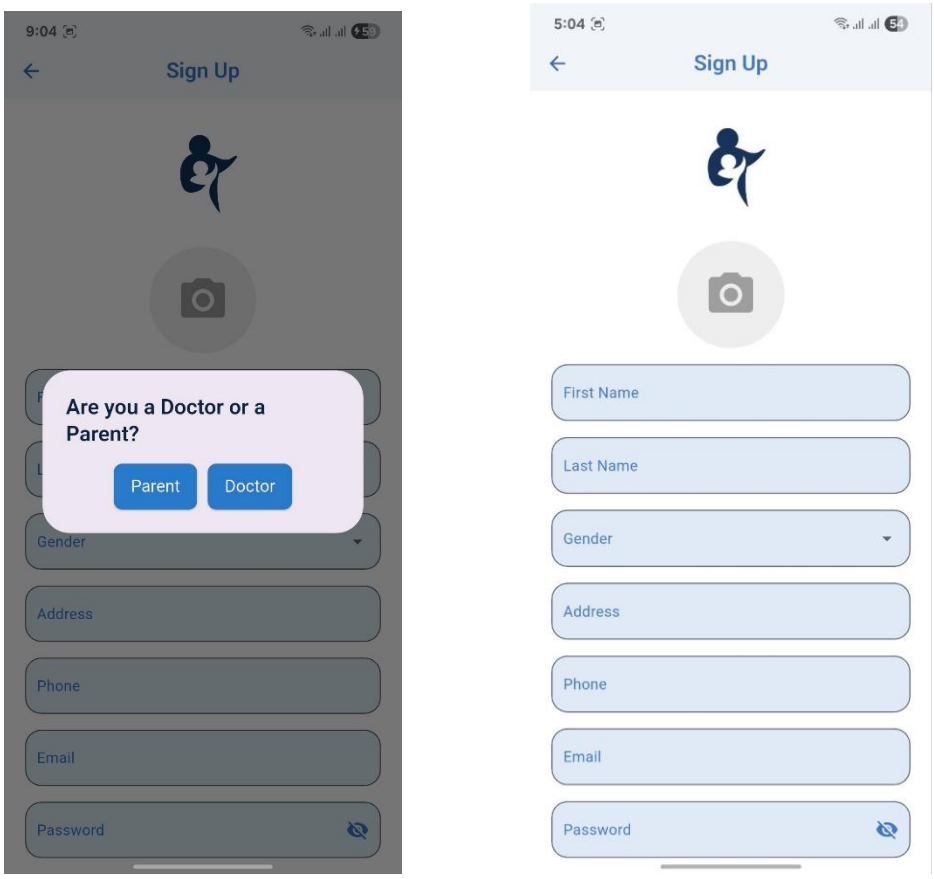

Figure 4 Sign up screens

For Parents as a User :

Home screen:

displays real-time data from the child's wristband, including heart rate, oxygen saturation, body temperature, blood pressure, and their live location. This immediate dashboard gives parents and doctors a clear snapshot of the child’s vital signs and whereabouts.

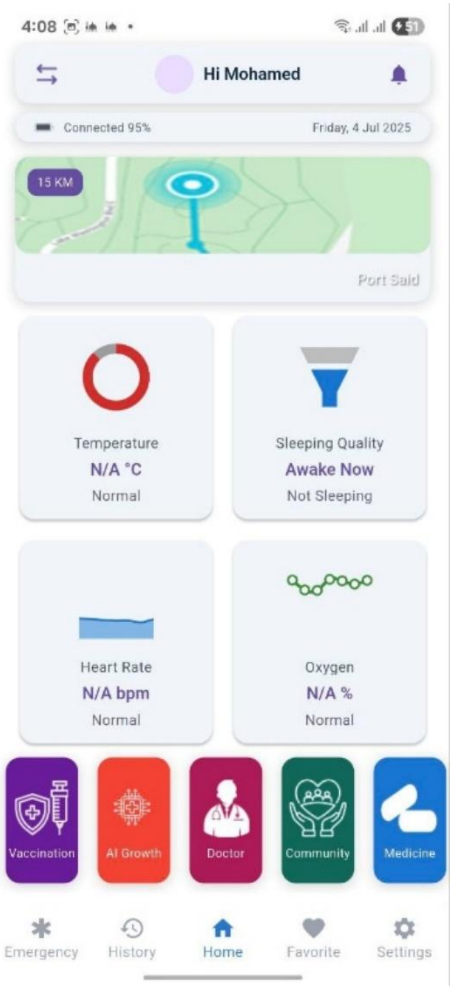

Figure 5 Home screens

Children profile screen: where can the parents to add more than one child

Vaccination screen:

displays a timeline of all vaccines administered to the child over five years. Parents can enter the date of a vaccine dose, enabling the app to automatically calculate and suggest the next dose date .

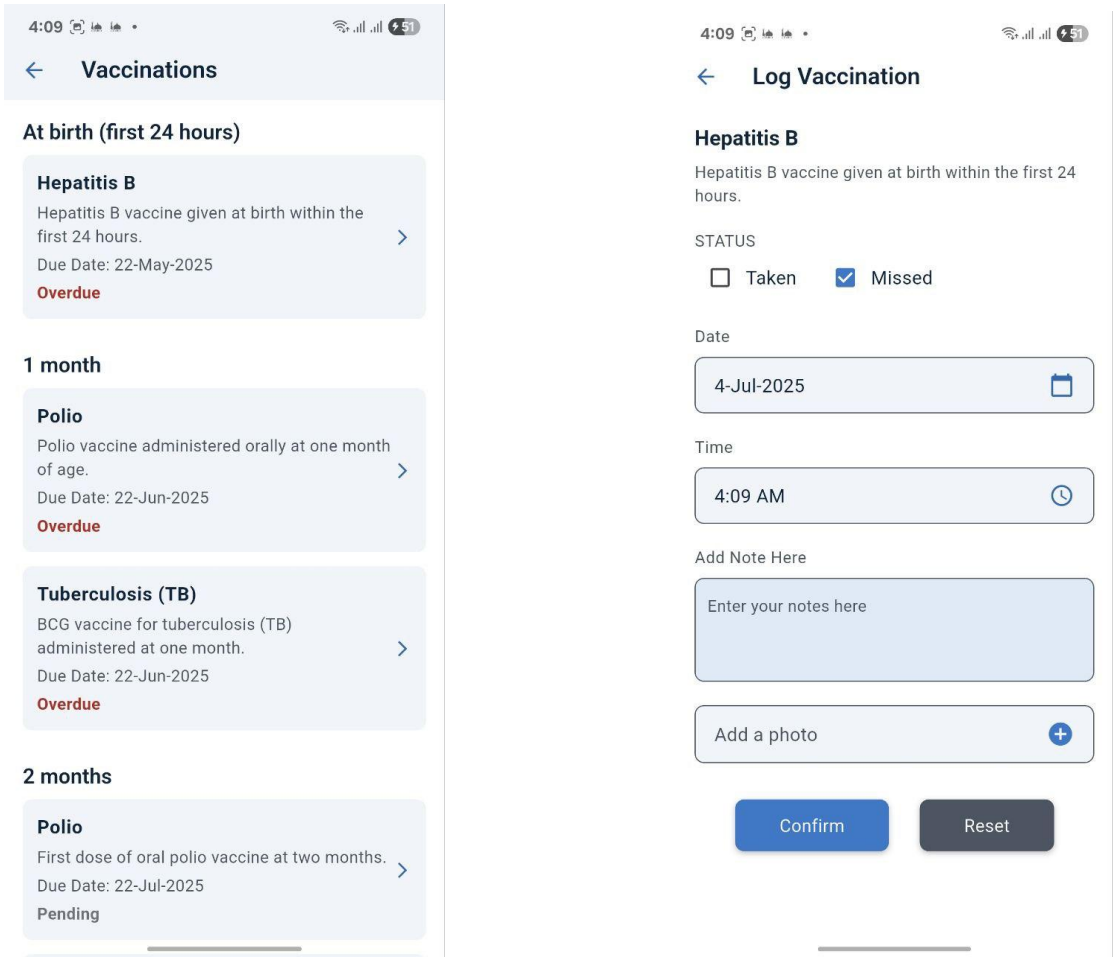

Figure 6 Vaccination screen and log Vaccination screen

Growth screen:

presents interactive charts tracking a child's height, weight and head circumference over time. Parents can easily log new measurements after each doctor visit-such as height and weight-and the data points are plotted on interactive charts against age, allowing users to track growth patterns.

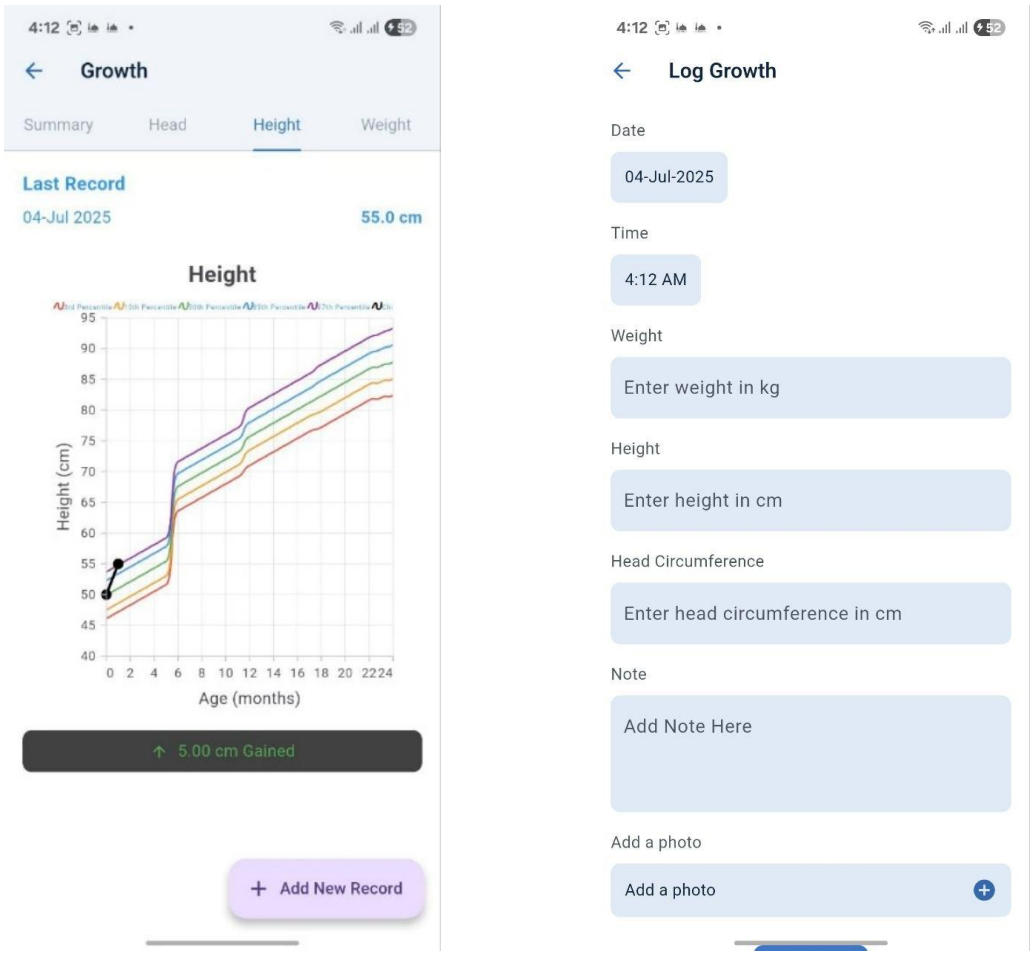

Figure 7 Growth screen and log Growth screen

Doctor screen:

displays a list of all registered doctors on Sigma, showing their specialties, availability, ratings, and profile images. Parents can browse doctors and easily book appointments through selecting the date.

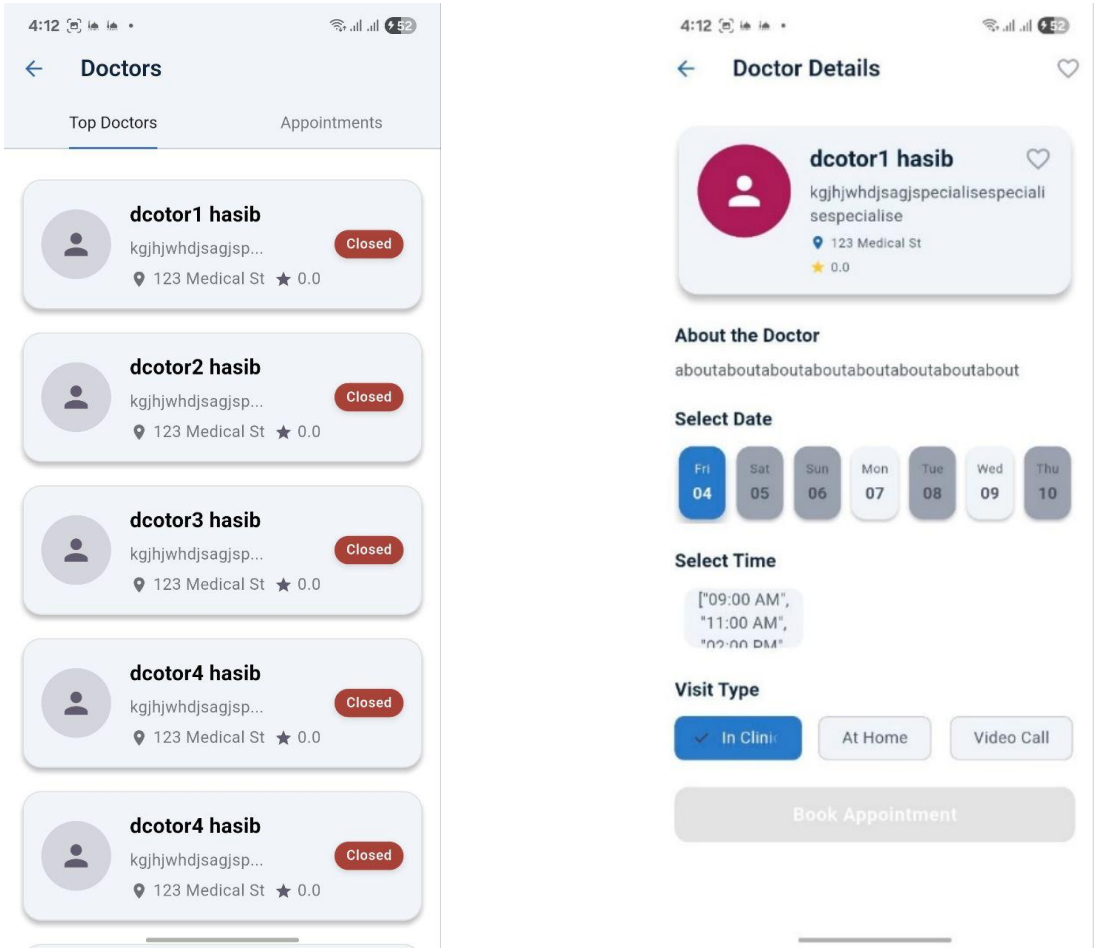

Figure 8 Doctor screen and Doctor Details screen

Medical screen: helps parents manage their child's medication regimen effortlessly by allowing them to: enter the name of each medicine, set multiple dose times, and receive reminders to pills on schedule and when the medicine end they can cancel the medication.

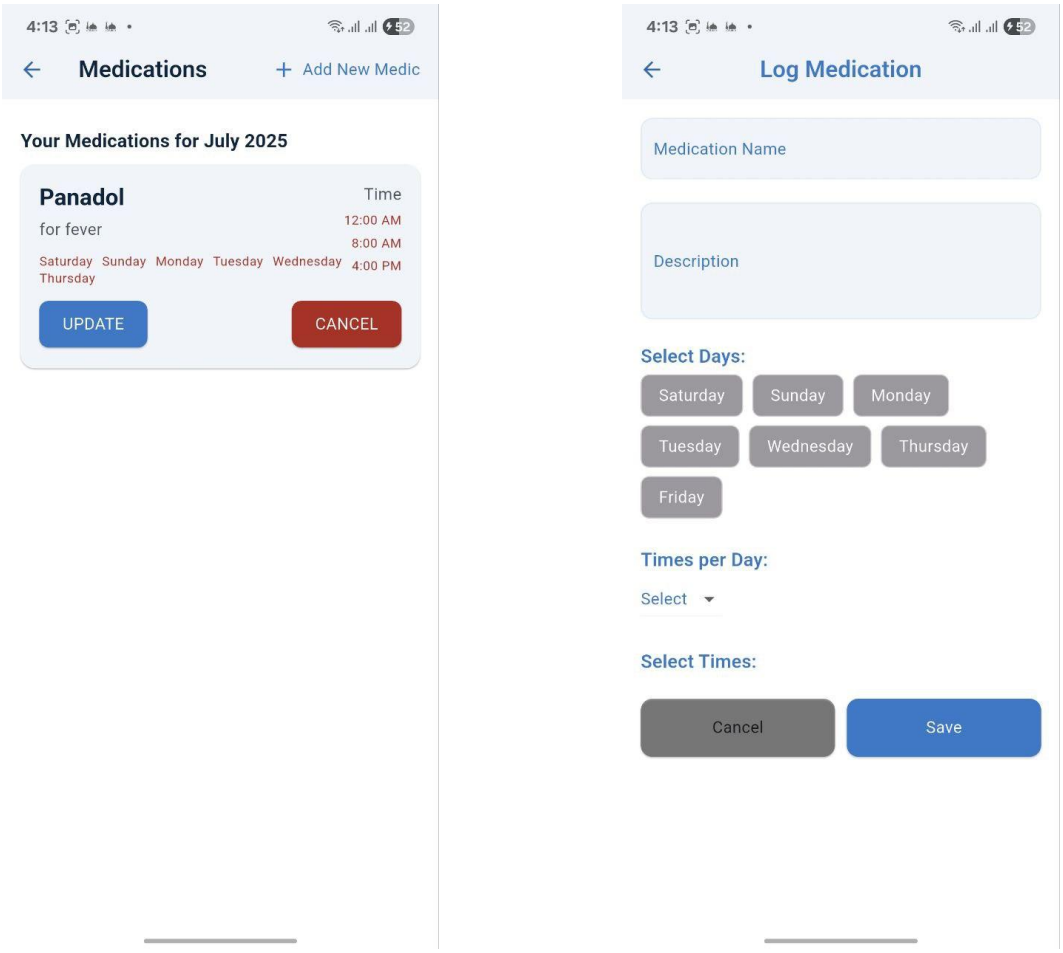

Figure 9 Medications screen and log Medication screen

Chat bot: provides parents with instant, 24/7 conversational support for child care questions.

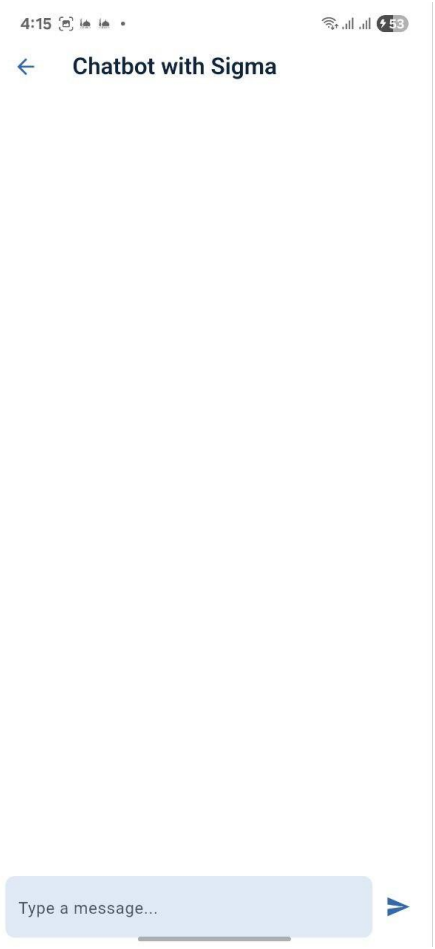

Figure 10 Chatbot screen

AI screen: guides parents through a structured set of questions to identify potential signs of developmental conditions such as autism, asthma, or other health concerns.

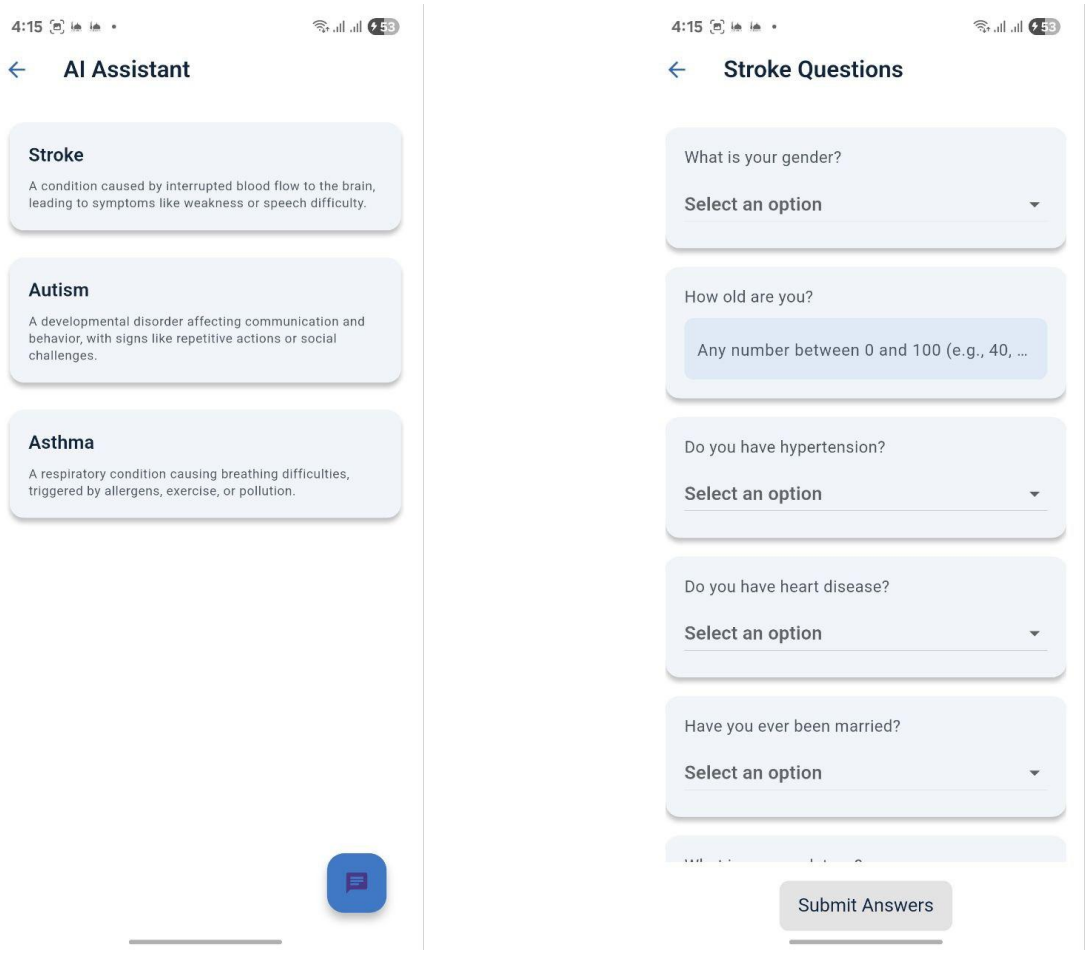

Figure 11 AI Assistant screen

History screen: The History Screen enables parents to record their child's complete medical history, including past illnesses, treatments, dates, and detailed notes on symptoms and outcomes. This digital health journal securely logs each event, such as viral infections, injuries, and recovery plans ,giving caregivers and doctors immediate access to important context and enabling more informed healthcare decisions .

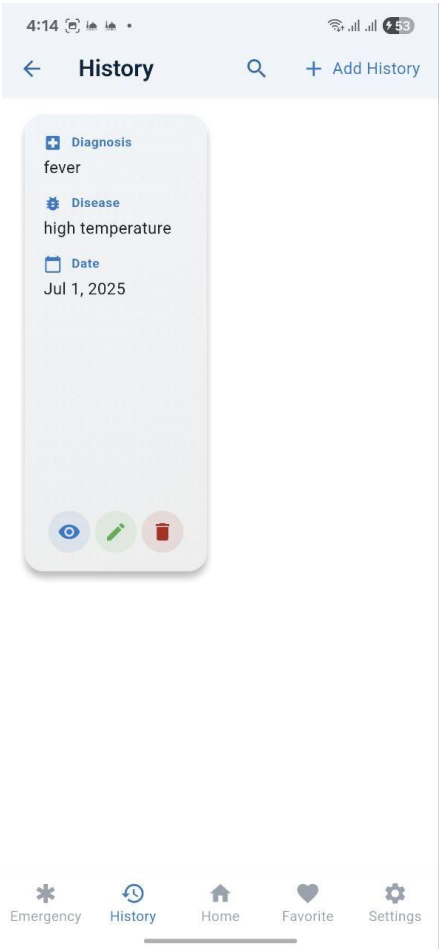

Figure 12 History screen

For Doctor as a User :

Appointment schedule screen: provides doctors with an intuitive calendar interface where they can define their working hours and open time slots. Incoming appointment requests from parents appear on the calendar as pending events, and doctors can easily accept or reject them with one tap.

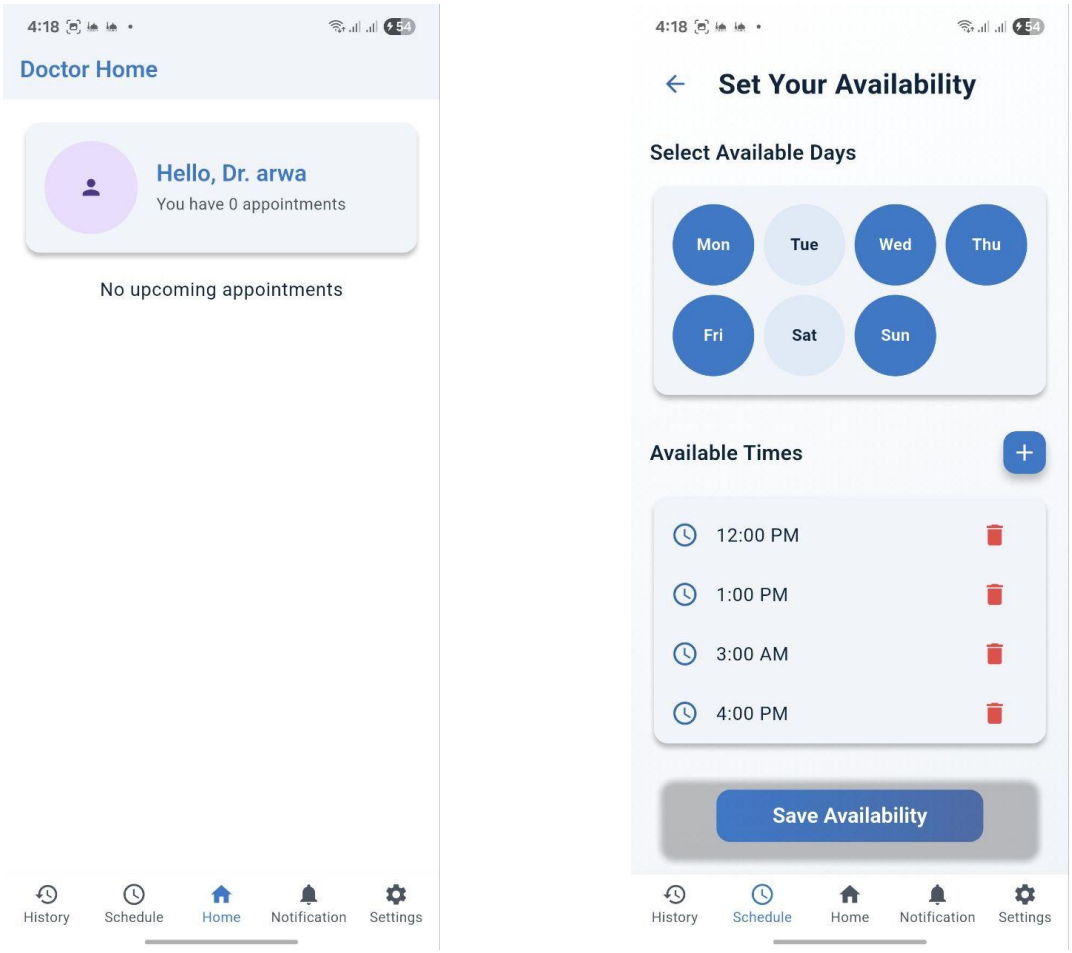

Figure 13 Doctor home screen and Schedule screen

Child history screen : enables doctors to quickly access a child's records by searching with their unique ID. Upon entering the ID, the app displays an editable timeline of the child's health history—including past illnesses, medications, and treatments—as well as growth data. Doctors can update records, add new entries, or correct measurements, ensuring that medical information is always accurate and current.

3.1.1.2User Interface (UI) Design for Website

When a caregiver or doctor taps the child's NFC-enabled wristband using a smartphone, the Sigma website instantly loads the child's complete medical history, including past illnesses, treatments, and relevant details. It ensures critical historical information is available immediately, streamlining emergency response.

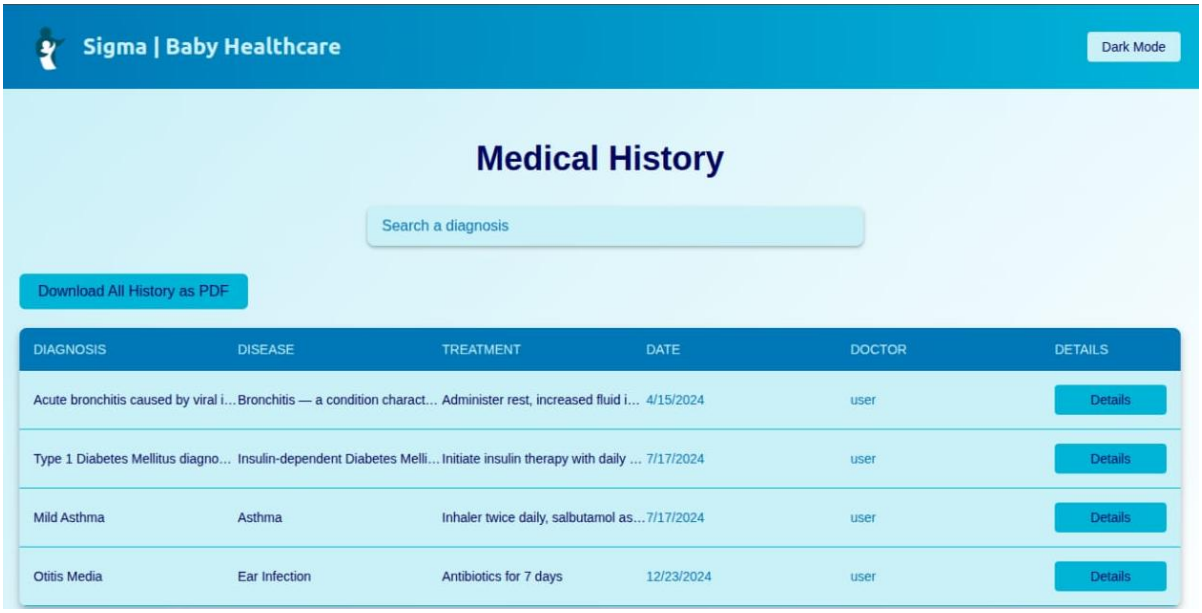

Figure 14 Medical History website

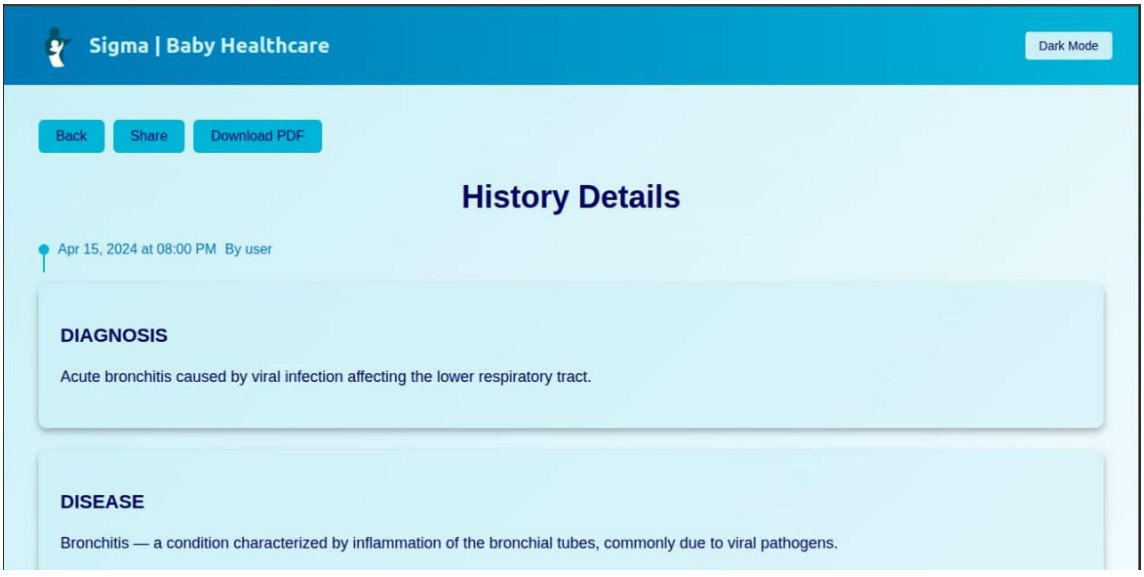

Figure 15 History Details website

3.2 Back-end Used in the Project

The backend of a healthcare system like SmartBrace serves as the central hub for data processing, communication, and integration, ensuring real-time functionality and scalability **(Newman, 2021)**. In SmartBrace, the backend manages sensor data from ESP32 bracelets, facilitates doctor-parent communication, and integrates AI predictions for pediatric conditions (asthma, autism, stroke). Built with Node.js, MongoDB, and FastAPI, it supports RESTful APIs (express), real-time communication (socket.io, mqtt), and cloud deployment (AWS, Render).

Key endpoints like (POST/api/sensor-data/:childId/validate) and (GET/api/chats/:childId/:doctorId/history) enable critical features. Packages such as jsonwebtoken and bcryptjs ensure secure user interactions, while dotenv manages environment variables, aligning with Egypt's healthcare needs for children aged 6 months to 7 years. A robust backend is vital for IoT-driven healthcare systems to deliver reliable, secure, and culturally relevant services (El-Zanaty & Way, 2006). In SmartBrace, the backend's role includes processing real-time data (e.g., GET /api/sensor-data/:childId/activities), delivering Arabic notifications (POST /api/notifications/bracelet), and enabling role-based access for Patients (parents), Doctors, and Admins via allowedTo middleware. It supports Egypt's vaccination schedules through endpoints like GET /api/vaccinations/:childId and ensures data privacy with express-mongo-sanitize. The winston package logs operations, and cors enables cross-platform access, making the backend indispensable for pediatric monitoring and stakeholder collaboration.

3.2.1 Server Backend Implementation

The backend architecture of the "Sigma" application is strategically designed to manage data processing, storage, and retrieval efficiently while ensuring robust performance and security. Built upon Node.js, a leading JavaScript runtime environment, the backend leverages modern paradigms to support real-time interactions, data-intensive operations, and seamless integration with the front-end Flutter framework

3.2.1.1 Run Time environment: Node.js

Node.js was chosen as the core technology for the backend of the "Sigma" application due to its compelling features and advantages in modern web development:

1. **Scalability:** Node.js excels in handling concurrent connections and high throughput, making it ideal for applications that require real-time processing and responsiveness. Its event-driven architecture allows it to efficiently manage multiple requests simultaneously without blocking operations, thus enhancing scalability under load.

2. **Performance:** The use of the V8 JavaScript engine, also used by Google Chrome, ensures that Node.js executes JavaScript code with high performance and efficiency. Its non-blocking I/O model minimizes latency by asynchronously handling requests, which is crucial for applications like image recognition and real-time data retrieval.

3. **JavaScript Everywhere:** Adopting Node.js aligns the backend development with the frontend Flutter framework, both utilizing JavaScript. This consistency streamlines development workflows, facilitates code reuse, and promotes a unified development environment across different application layers.

4. **Rich Ecosystem:** Node.js boasts a vast ecosystem of modules and libraries accessible through NPM (Node Package Manager). This ecosystem provides developers with pre-built solutions for various functionalities such as database integration, authentication mechanisms, API development, and more. It accelerates development timelines by offering robust, communitysupported tools that enhance productivity and code quality.

5. **Community Support:** Node.js benefits from a large and active community of developers, which contributes to extensive documentation, tutorials, and forums. This community-driven support network aids in troubleshooting, sharing best practices, and staying updated with the latest advancements in Node.js development, ensuring the application remains robust and secure.

Why Node.js Over Other Programming Languages?

Node.js was specifically chosen for the backend of the " Sigma " application over other programming languages due to several distinct advantages:

- Event-Driven Architecture: Node.js utilizes an event-driven, non-blocking I/O model that efficiently manages concurrent connections and I/O-bound operations. This architecture enhances scalability and performance, particularly in applications requiring real-time updates and interactions.

- Unified Language: By leveraging JavaScript for both frontend and backend development, the development team benefits from code reusability, shared libraries, and a consistent programming paradigm. This integration simplifies maintenance, reduces development overhead, and promotes seamless communication between different application components.

- Performance with V8 Engine: Built on the powerful V8 engine, Node.js compiles JavaScript into optimized machine code at runtime, delivering superior performance and rapid execution speeds. Despite its single-threaded nature, Node.js handles multiple concurrent operations efficiently, making it suitable for handling high-traffic applications and real-time data processing.

- Scalability and Microservices: Node.js supports microservices architecture, enabling applications to be divided into smaller, independent services that can be developed, deployed, and scaled individually. This modular approach aligns with modern development practices, facilitating agility and scalability as the application evolves.

- Real-Time Capabilities: For applications like the "Sigma" that require realtime data processing and user interactions, Node.js provides built-in support for Web Sockets. This feature enables bidirectional communication between clients and servers, essential for features such as live updates, chat functionality, and interactive user experiences.

- Ease of Development: Node.js is renowned for its developer-friendly environment, offering a straightforward learning curve and tools that support rapid development cycles. Asynchronous programming capabilities simplify handling asynchronous tasks, ensuring smooth performance and responsiveness in I/O-intensive operations.

Figure 16 Node.js uses JavaScript on the Server

3.2.1.2 Backend Framework: Express.js

Express.js serves as the web application framework for the backend of the "Sigma" application, built on Node.js. It provides a robust foundation for developing web and mobile applications with its minimalistic yet powerful set of features. Key Features of Express.js:

1. **Minimal and Flexible Architecture:**
   Minimalistic Design: Express.js is designed to be lightweight and unopinionated, allowing developers the flexibility to structure their applications according to specific project requirements. It does not enforce strict conventions, which makes it highly adaptable to various development styles and application architectures.

2. Middleware Support:

Middleware Functions: One of the core features of Express.js is its middleware architecture. Middleware functions are functions that have access to the request and response objects (req, res) and can modify them, execute additional code, or terminate the request-response cycle. This modular approach allows developers to add layers of functionality to their applications, such as logging, authentication, error handling, and data parsing. Middleware functions can be used globally, applied to specific routes, or chained together to create complex processing pipelines.

3. **Routing**:

Flexible Routing System: Express.js provides a powerful and intuitive routing mechanism. Developers can define routes for different HTTP methods (GET, POST, PUT, DELETE, etc.) and endpoints easily. Routes are defined using path patterns and can handle dynamic parameters, making it straightforward to build RESTful APIs or web applications with structured endpoints. Routing in Express.js ensures that incoming requests are efficiently directed to the appropriate handler functions based on the defined routes.

4. **Integration Capabilities**:

Modular Integration: Express.js seamlessly integrates with a wide range of middleware modules, databases (such as MongoDB, MySQL, PostgreSQL), template engines (like EJS, Handlebars), and other Node.js modules. This modular architecture allows developers to leverage existing libraries and tools from the Node.js ecosystem, enhancing productivity and reducing development time.

5. **Performance**:

Efficiency and Scalability: Express.js is optimized for performance and is capable of handling a large number of simultaneous requests efficiently. It utilizes asynchronous programming patterns and non-blocking I/O operations provided by Node.js, ensuring minimal overhead and high throughput. This makes Express.js suitable for building real-time applications, APIs, and web servers that require responsiveness and scalability.

Why Express.js for the "Sigma" Application? Express.js was chosen as the backend framework for the "Sigma" application due to several compelling reasons:

- **Flexibility:** The minimalistic and flexible nature of Express.js allows developers to tailor the backend architecture precisely to the application's needs. This flexibility is crucial for accommodating specific business logic, integration requirements, and future scalability.

- **Middleware Ecosystem:** The extensive middleware ecosystem of Express.js simplifies the implementation of common functionalities such as authentication, session management, error handling, and data validation. Developers can easily plug in middleware functions or create custom middleware to enhance application features without reinventing the wheel. •

Routing and API Development: Express.js provides robust support for defining and managing routes, making it straightforward to build RESTful APIs and web services. Its clear and intuitive routing system ensures clarity and maintainability in handling different endpoints and HTTP methods required by the application.

- **Community and Support:** Express.js benefits from a large and active community of developers, offering abundant resources, documentation, and third-party modules via NPM. This community support facilitates rapid development, troubleshooting, and continuous improvement of the application. In summary, Express.js empowers the "Sigma" application with a scalable, performant, and flexible backend framework. By leveraging its middleware capabilities, routing system, and seamless integration with Node.js ecosystem, Express.js facilitates the development of a robust backend architecture that meets the application's requirements for efficient data processing, API development, and user interaction.

3.2.1.3 MongoDB Database Design

MongoDB is a popular open-source NoSQL (non-relational) database management system that provides high performance, scalability, and flexibility for handling large amounts of data. It differs from traditional relational databases by using a document-oriented data model, which stores data in a flexible, JSON-like format called BSON (Binary JSON).

Figure 17 MongoDB

In a non-relational database like MongoDB for example, we do not have tables, instead we have Collections and Documents. A collection, as the name implies, is a collection of one or more documents. If you are from a relational database background, you can think of a collection as a table and documents as table rows. So, in MongoDB, a database contains one or more collections. A collection is made up of one or more documents and each document is made up of one or more fields.

Advantages of MongoDB

1. **Schema Flexibility:** MongoDB's document-based model allows for dynamic and flexible schemas. It accommodates evolving data structures and can handle semi-structured or unstructured data effectively.

2. **Scalability and Performance:** MongoDB is designed for horizontal scalability. It supports automatic sharding, distributing data across multiple servers or clusters. This capability enables efficient handling of high traffic loads and ensures optimal performance for large-scale applications.

3. **High Availability:** MongoDB supports replica sets, which are groups of database servers maintaining identical copies of data. If a server fails, another replica can take over, ensuring continuous availability and data redundancy.

4. **Rich Query Language:** MongoDB provides a powerful query language with support for complex queries, including aggregations, geospatial queries, and text search. It also allows the creation of secondary indexes for efficient data retrieval.

5. **Flexible Data Model:** MongoDB's data model supports nested documents and arrays, making it suitable for representing complex data structures. This flexibility is beneficial for applications such as content management systems, social networks, and e-commerce platforms.

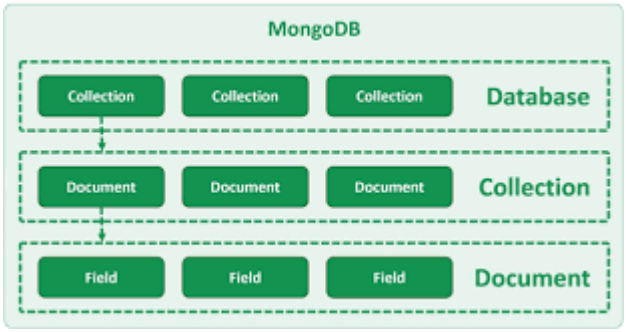

Figure 18 MongoDB form

Disadvantages of MongoDB

1. **No ACID transactions across multiple documents**: MongoDB does not support multi-document ACID (Atomicity, Consistency, Isolation, Durability) transactions. While it does support atomic operations within a single document, ensuring consistency across multiple documents requires careful design and application-level handling.

2. **Higher memory usage:** MongoDB typically consumes more memory compared to traditional relational databases. The in-memory processing and caching mechanisms used by MongoDB can lead to increased memory requirements, especially when dealing with large datasets.

3. **Limited JOIN operations:** MongoDB does not support JOIN operations like traditional relational databases. To retrieve related data from multiple collections, you would need to denormalize or embed the data within a single document or perform separate queries.

Common uses of MongoDB

1. **Content management systems:** MongoDB's flexible schema and ability to handle large volumes of unstructured data make it well-suited for content management systems, where content can have varying attributes and structures.

2. **Real-time analytics:** MongoDB's scalability and high-performance capabilities make it suitable for real-time analytics applications.

3. **Mobile and social applications:** MongoDB's JSON-like document model and support for geospatial queries make it popular for building mobile and social applications that require flexible data structures, real-time updates, and location-based features.

4. **Catalogs and product data:** MongoDB's ability to store complex and hierarchical data structures makes it a good choice for building product catalogs, inventory systems, and e-commerce platforms.

Why do we use MongoDB?

MongoDB is often used with Node.js for several reasons:

1. **JSON-like Documents:** MongoDB uses a flexible and schema-less data model based on JSON-like documents (BSON format). This aligns well with the JavaScript-based development in Node.js, as JavaScript objects and JSON are native data structures. It allows developers to store and retrieve data in a format that is familiar and easily compatible with Node.js.

2. **NoSQL and Scalability:** MongoDB is a NoSQL database that offers high scalability and performance. It is designed to handle large amounts of data and can scale horizontally by distributing data across multiple servers or shards. This makes it suitable for applications that require handling high volumes of data, such as tourism websites that may involve storing user profiles, bookings, reviews, and location information.

3. **Asynchronous Nature:** Node.js is known for its asynchronous and nonblocking I/O operations. MongoDB's driver for Node.js takes advantage of this asynchronous nature by providing non-blocking database operations. This allows Node.js applications to execute database queries and perform CRUD operations without blocking the event loop, resulting in better application performance and responsiveness.

4. **Native MongoDB Driver:** MongoDB provides an official native driver for Node.js called the MongoDB Node.js driver. This driver offers a straightforward and efficient way to interact with the MongoDB database from a Node.js application. It provides comprehensive functionality to perform database operations, execute queries, and handle data manipulation tasks.

5. **Flexibility and Agility:** MongoDB's flexible schema allows developers to store and modify data without predefined schemas or migrations. This flexibility is beneficial for agile development environments, where requirements may change frequently. It enables developers to iterate quickly and adapt their data models without the need for complex database migrations.

6. **Geospatial Capabilities:** Many tourism-related applications involve geospatial data, such as location-based services, maps, or route planning. MongoDB has built-in geospatial indexing and querying capabilities, making it well-suited for handling geospatial data. With Node.js and MongoDB, developers can easily implement features related to geolocation and spatial queries in their tourism applications.

Other Tools and Libraries Used with MongoDB:

1. Mongoose:

- Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It simplifies interaction with MongoDB by providing a schema-based solution for data modeling, validation, and management of relationships between data entities. Mongoose integrates seamlessly with Node.js applications, offering features like schema definition, type casting, query building, and middleware hooks for pre and post data operations.

MongoDB, supported by tools like Mongoose, JWT, and multer, plays a critical role in the "Sigma" application's backend architecture. It enables efficient data storage, management, and retrieval, while its document-oriented model and scalability features cater well to the dynamic and diverse nature of tourist-related information and user interactions.

MongoDB Database Setup MongoDB version 4.2 is the version that is applied in this project. Six collections were created to save the needed data: users, comment, coupons, ratings, supervisorplaces, survey, and places.

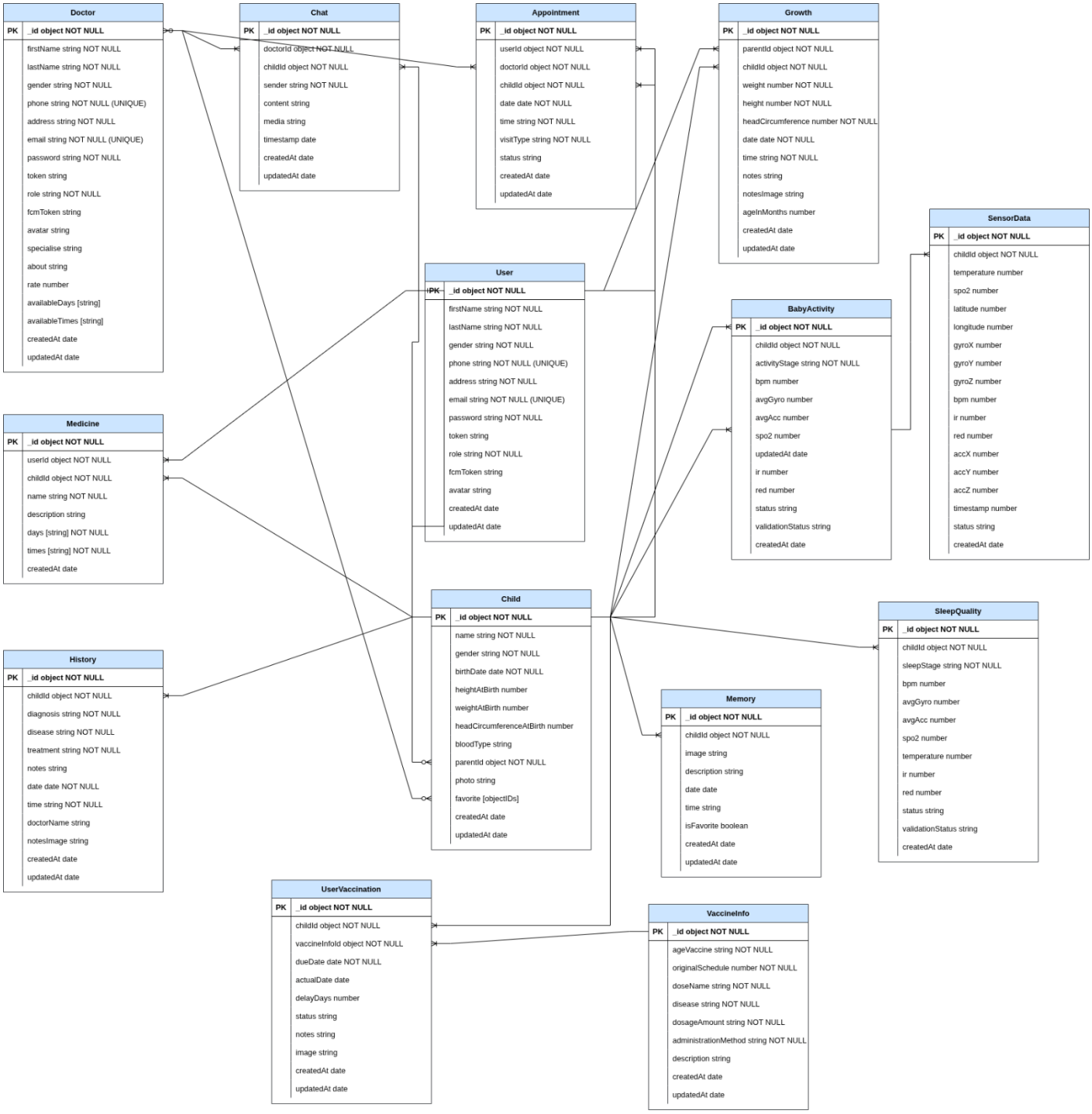

Figure 19 ER Diagram

3.2.1.4 Real-Time with WebSocket and MQTT

Real-time communication is a cornerstone of Internet of Things (IoT)-enabled healthcare systems, facilitating instantaneous data exchange and seamless user interactions [?]. In the context of the SmartBrace system, designed for pediatric healthcare in Egypt for children aged 6 months to 7 years, real-time communication supports critical functionalities such as sensor data transmission and interactive doctor-parent communication. This section provides a detailed examination of two key protocols—WebSocket for secure, real-time chat and Message Queuing Telemetry Transport (MQTT) for sensor data transfer—focusing on their implementation, security mechanisms, and the Node.js ecosystem that underpins their operation.

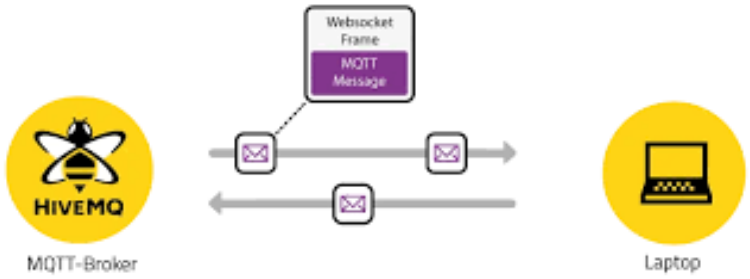

Figure 20 MQTT with WS

1. **WebSocket for Real-Time**

WebSocket is a full-duplex communication protocol that establishes a persistent Transmission Control Protocol (TCP) connection, enabling bidirectional, low-latency data exchange between clients and servers [?]. Unlike the traditional Hypertext Transfer Protocol (HTTP) request-response model, which relies on polling for updates, WebSocket supports instantaneous data pushing, making it highly suitable for real-time applications such as chat systems. Its efficiency in reducing communication overhead enhances user experience in healthcare applications where immediate interaction is critical.

- **Implementation in SmartBrace**

In the SmartBrace system, WebSocket facilitates real-time chat functionality between parents (referred to as Patients) and pediatric doctors, enabling consultations regarding a child's health status, including the sharing of multimedia content such as medical images. This feature improves healthcare accessibility in Egypt by bridging the gap between parents and specialists. The chat system is supported by the following application programming interface (API) endpoints: • GET /api/chats/:childId/:doctorId/history: Retrieves the chat

history for a specific child and doctor, accessible to authorized Patients and Doctors. • POST /api/chats/:childId/:doctorId/upload: Enables Patients to upload media files, which are securely stored in Amazon Web Services (AWS) Simple Storage Service (S3). • GET /api/chats/:childId/:doctorId/eligibility: Verifies whether a Patient is eligible to initiate a chat with a specific doctor based on prior appointment history. The socket.io Node.js package implements the WebSocket protocol, establishing persistent connections for efficient message exchange. When a parent sends a message or uploads media, socket.io broadcasts the content instantly to the doctor's client, ensuring minimal latency. A custom middleware, validateIds, leverages the mongoose library to validate childId and doctorId parameters, mitigating the risk of invalid or malicious requests.

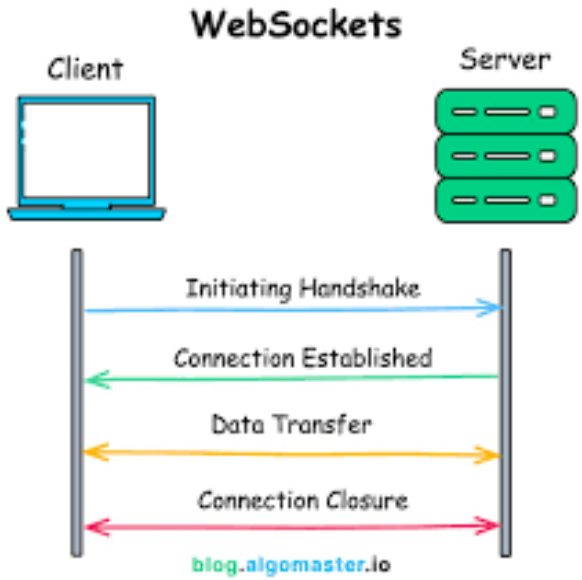

Figure 21 connection with WS

- **Security Mechanisms**

To maintain a secure and professional communication environment, SmartBrace enforces strict access controls for the chat system. A Patient can initiate a chat with a doctor only after booking at least one appointment, a condition verified through the GET /api/chats/:childId/:doctorId/eligibiliendpoint. This endpoint cross-references the Patient's appointment history with the doctor, established via the POST /api/doctors/:childId/:doctorId/book endpoint. This restriction protects doctors from unauthorized or spam messages and ensures that interactions are grounded in established medical relationships, fostering trust and usability. Access to chat endpoints is further secured using the allowedTo middleware, which employs JSON Web Token (JWT) authentication via the jsonwebtoken package. This middleware restricts endpoint access to authorized roles—Patients for sending messages and Doctors for viewing chat history. This security layer is paramount in a healthcare context, where patient privacy and professional boundaries must be upheld.

- **Supporting Node.js Packages**

The following Node.js packages are integral to the WebSocket-based chat system:

- socket.io: Provides a robustWebSocket implementation with fallback to HTTP long-polling for compatibility. It is integrated into the Express server to handle chat events, such as message sending and media uploads, and supports namespaces to isolate chats by childId and doctorId, ensuring data privacy.
- path: Manages file paths during media uploads, ensuring accurate storage in AWS S3.
- winston: Logs chat-related events, such as message delivery and media upload errors, facilitating debugging and system monitoring.
- express-mongo-sanitize: Sanitizes user inputs to prevent MongoDB injection attacks, securing the integrity of message content.

2 MQTT for Sensor Data Transmission

Overview of MQTT Protocol

MQTT is a lightweight, publish-subscribe protocol optimized for IoT devices with constrained resources, operating over TCP with a broker to route messages based on topics [?]. Its efficiency, reliability, and low bandwidth requirements make it ideal for real-time sensor data transmission in healthcare applications, where continuous monitoring and high availability are essential.

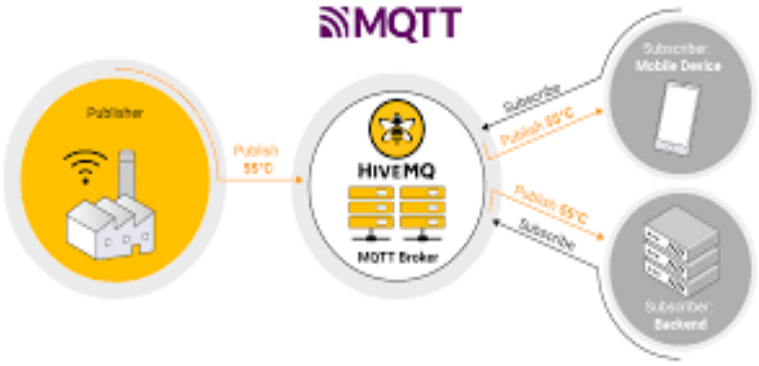

Figure 22 MQTT protocol

Implementation in SmartBrace

In SmartBrace, MQTT enables the transmission of sensor data from ESP32-based smart bracelets to a MongoDB database, supporting continuous health monitoring for pediatric patients. The bracelets measure vital metrics, including heart rate, activity levels, and sleep quality, which are critical for pediatric care in Egypt. The HiveMQ cloud broker

serves as an intermediary between the bracelet (publisher) and the backend server (subscriber), ensuring reliable data delivery. Sensor data is published to the sensors/data/:id topic, and the backend subscribes to this topic to process and store validated data in the ValidatedSensorData collection. Key API endpoints include:

- **POST /api/sensor-data/:childId/validate:** Validates and stores sensor data every 30 seconds, accessible to Patients, Doctors, and Administrators.
- **GET /api/sensor-data/:childId/activities:** Retrieves activity data for the past 24 hours.
- **GET/api/sensor-data/:childId/sleep-qualities:** Fetches sleep quality metrics for a specified child.

Operation of MQTT with HiveMQ Broker

The MQTT publish-subscribe model decouples the ESP32 bracelet from the backend server. The bracelet transmits JSON-formatted sensor data (e.g., {”childId”: ”123”, ”heartRate”: 80, ”temperature”: 36.5}) to the sensors/data/:id topic via MQTT. The HiveMQ cloud broker routes these messages to subscribed subscribers, such as the Node.js server, based on topic subscriptions. The mqtt.service.js script processes incoming messages, validates data against predefined thresholds (e.g., acceptable heart rate ranges), and triggers storage via the POST /api/sensor-data/:childId/validate endpoint. The HiveMQ broker ensures high availability and supports secure connections using Transport Layer Security (TLS), safeguarding sensitive health metrics during transmission.

3.2.1.5 AWS Deployment (EC2 and S3)

Cloud deployment is critical for modern healthcare applications, providing scalability, reliability, and secure storage for data and services (Wittig & Wittig, 2020). In the SmartBrace system, Amazon Web Services (AWS) powers the backend deployment, leveraging **EC2** to host the Node.js server and **S3** to store images and files. This section details the implementation of EC2 and S3 in SmartBrace, focusing on their roles in supporting real-time pediatric healthcare for children aged 6 months to 7 years in Egypt. It covers the use of Ubuntu on EC2’s free tier, S3 for file storage, relevant endpoints, security mechanisms, and Node.js packages (multer, path, jsonwebtoken, etc.) that enable these functionalities.

- **Amazon EC2 for Server Deployment**

General Overview: This section provides a detailed explanation of deploying a Node.js backend application to an Amazon EC2 instance using the PM2 library. It will also outline

the benefits of utilizing AWS EC2 and PM2 for application deployment. Node.js is a popular runtime environment for building scalable and high-performance server-side

applications that is used to process the collected data. Amazon EC2 (Elastic Compute Cloud) is a web service offered by Amazon Web Services (AWS) that provides resizable compute capacity in the cloud. PM2 is a (production process manager) for Node.js applications that provide advanced process management, automatic application monitoring, and clustering capabilities. For choosing the free tier we can use the (t3. micro) type that provides 2 vCPU, 1 GIB Memory.

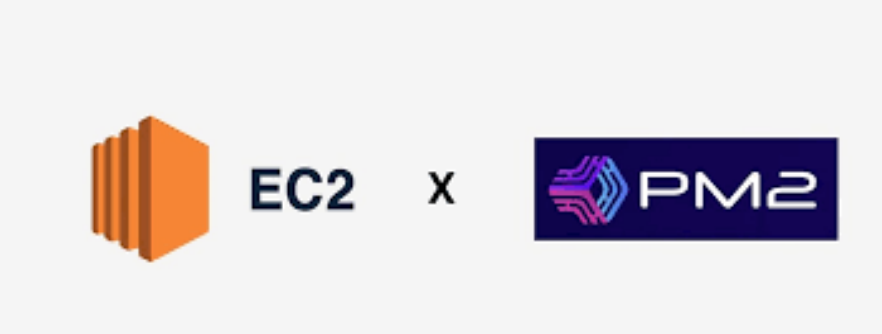

Figure 23 Deploying Node.js App to EC2 Instance using PM2

The main steps of the deployment process

Step 1: Provisioning an EC2 Instance Launch an EC2 instance with the desired configuration, such as instance type, operating system, and networking settings. (Figure 23) shows how to configure the EC2 Instance - Assign appropriate security groups and access permissions to the instance.

Step 2: Configuring the EC2 Instance - Connect to the EC2 instance using SSH. - Install Node.js and any other dependencies required by the application. - Clone the application repository to the EC2 instance. - Configure environment variables and application settings.

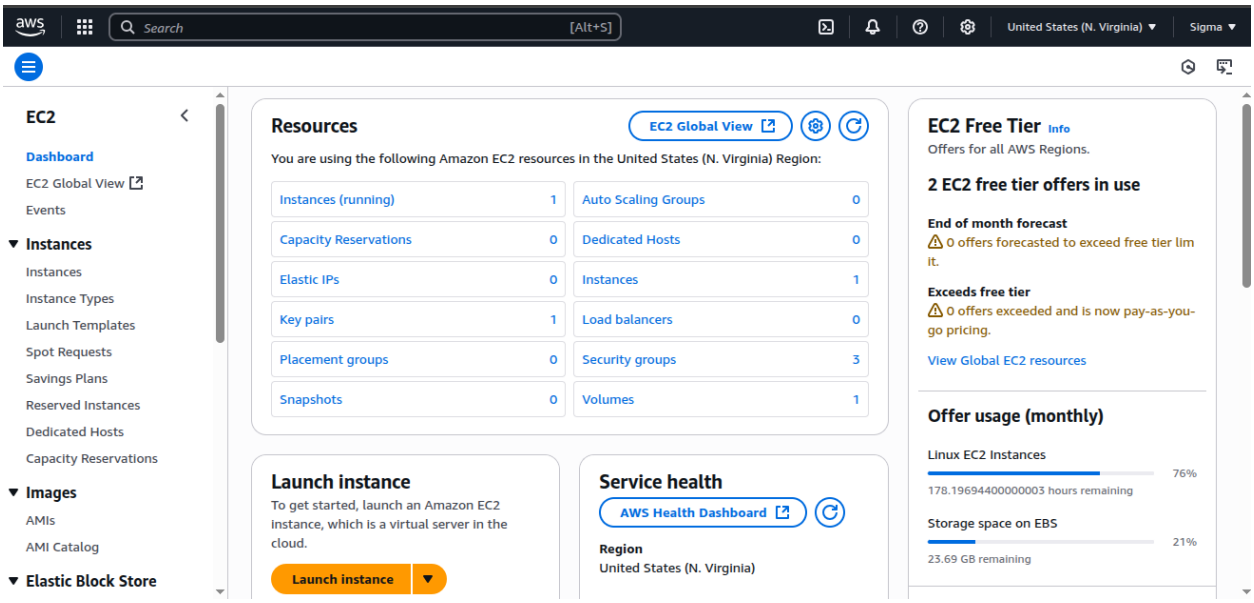

Figure 24 Launching instance

Step 3: Installing and Configuring PM2 - Install PM2 globally on the EC2 instance using NPM. - Set up the PM2 configuration file to define application-specific settings, such as the entry point file and environment variables. - Start the application using PM2, which will manage the application as a background process.

Step 4: Monitoring and Managing the Application - Utilize PM2's built-in monitoring capabilities to monitor the application's resource usage, logs, and error tracking as shown in (Figure 25). - Use PM2 commands to manage the application's lifecycle, such as starting, stopping, and restarting the application. - Configure PM2 to automatically restart the application in failures or crashes.

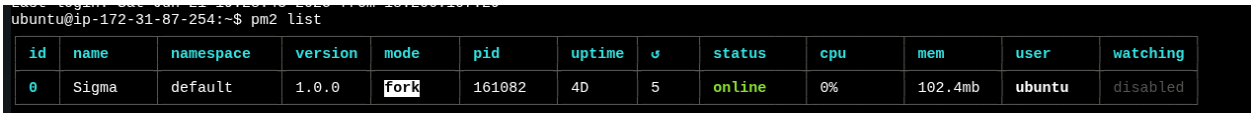

Figure 25 Pm2 monitors a single instance

Benefits of Using AWS EC2

a) **Scalability:** EC2 allows you to easily scale your infrastructure by adding or removing instances based on your application's demands. This ensures that your backend application can handle increased traffic and maintain performance.

b) **Flexibility:** EC2 offers a wide range of instance types and operating systems, allowing you to choose the best fit for your app requirements. You control the configuration .

c) **Reliability:** AWS provides high availability and fault tolerance through features such as auto-scaling, load balancing, and data replication. This ensures that your backend application remains accessible and minimizes the risk of downtime.

d) **Security:** AWS offers robust security features, including network isolation, encryption, access control, and regular security updates. You can implement best practices to secure your EC2 instances and protect your application and data.

Benefits of Using PM2

a) **Process Management:** PM2 simplifies the management of Node.js processes by allowing you to start, stop, and restart applications with ease. It ensures that your application runs continuously and is automatically restarted in case of failures.

b) **Monitoring and Logging:** PM2 provides comprehensive monitoring and logging capabilities, giving you insights into your application's resource usage, performance metrics, and error tracking.

c) **Load Balancing and Clustering:** PM2 supports load balancing and clustering out-of-the-box, allowing you to scale your application horizontally across multiple instances. The same account on AWS can have multiple instances, so pm2 can handle all of them. This improves the performance of system resources.

d) **Zero Downtime Deployment:** PM2 supports zero-downtime deployment, which means that your application can be updated or redeployed without any interruption to the end users. This ensures a seamless user experience and avoids service disruptions.

- **Amazon S3 for File Storage**

General Overview: Amazon Simple Storage Service (S3) is an object storage service designed for scalable, durable, and secure file storage (Amazon Web Services, 2023). S3 organizes data in buckets, accessible via unique URLs, and is widely used for storing media files (e.g., images, videos) in web applications. Its integration with AWS services and fine-grained access control make it suitable for healthcare systems requiring secure file management.

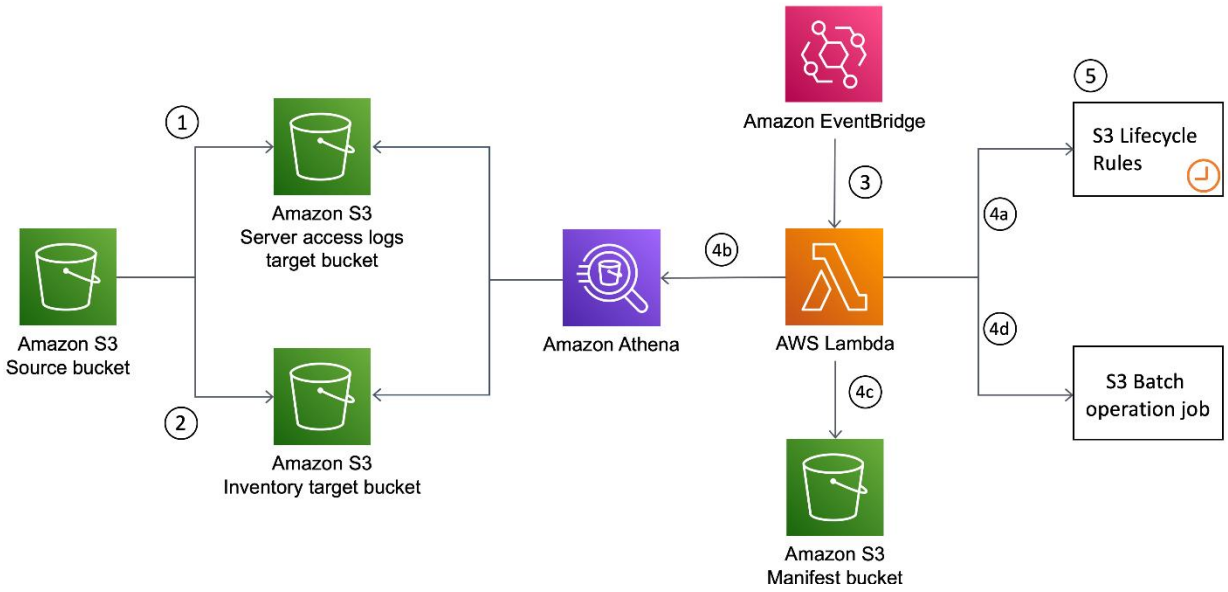

Figure 26 AWS S3 Object Lifecycle

Application in SmartBrace: In SmartBrace, S3 is used to store images and files, such as profile avatars, chat media (e.g., medical images shared during consultations), and other user-uploaded content. This ensures reliable access to media for Patients, Doctors, and Admins, supporting pediatric healthcare interactions in Egypt.

The multer package handles file uploads on the Node.js server, streaming files directly to S3 buckets. The path package manages file paths and extensions, ensuring correct storage. S3 buckets are configured with private access, and temporary signed URLs are generated for secure file retrieval (e.g., when Doctors view chat media via GET /api/chats/:childId/:doctorId/history). The allowed To middleware restricts file upload endpoints to authorized roles (e.g., Patients for chat media), enforced by JWT authentication (jsonwebtoken). Bucket policies and IAM roles (implied from AWS best practices) limit access to the Node.js server, preventing unauthorized file access. Arabic file naming conventions are supported in S3 metadata, aligning with Egypt's linguistic context.

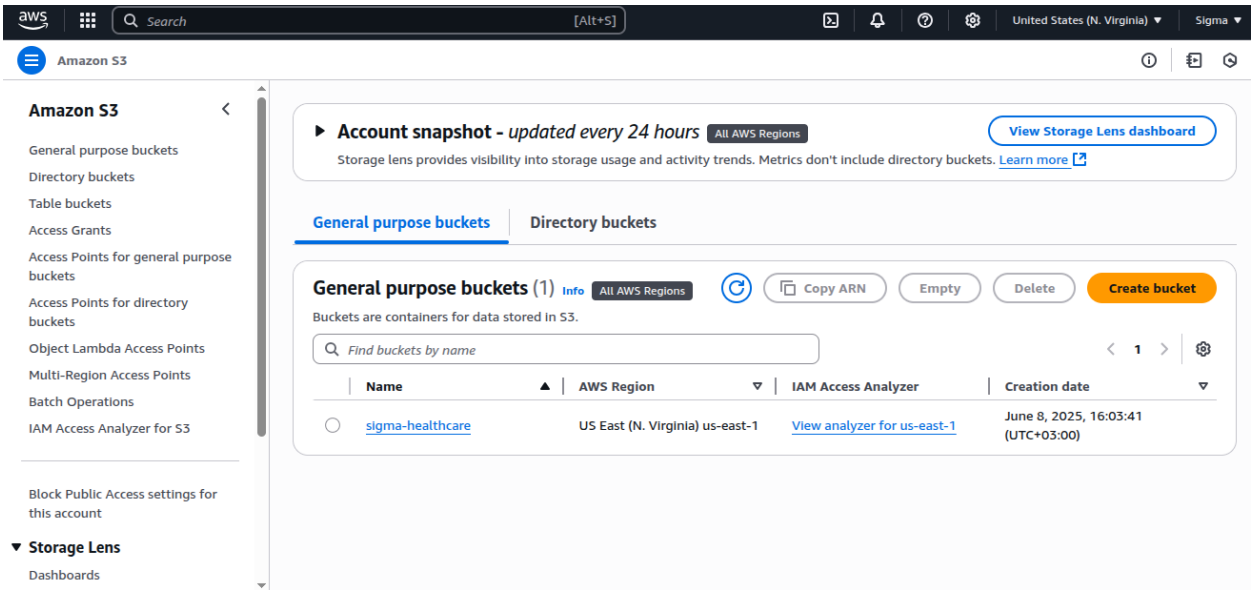

Figure 27 Launching S3

Integration with SmartBrace Workflows: EC2 and S3 work seamlessly to support SmartBrace's backend. The EC2-hosted server processes API requests (e.g., POST /api/sensor-data/:childId/validate) and delegates file storage to S3 (e.g., POST /api/chats/:childId/:doctorId/upload). For example, when a Patient uploads a medical image during a chat, multer streams it to S3, and the server stores the S3 URL in MongoDB via mongoose. Doctors retrieve the image using a signed URL, ensuring secure access. The EC2 instance's Ubuntu environment supports real-time communication (via socket.io and mqtt) and notification delivery (firebase-admin), while S3 ensures media availability. This integration enhances healthcare delivery by providing reliable access to critical data and media.

Security Considerations: EC2 security is enforced through security groups, limiting inbound traffic, and SSH key pairs for server access. The jsonwebtoken package secures API endpoints, and dotenv protects sensitive configurations (e.g., S3 bucket credentials). S3 security includes private bucket policies, IAM roles, and signed URLs for file access. Packages like express-mongo-sanitize prevent injection attacks, and winston logs security events, safeguarding pediatric data in compliance with healthcare standards (Sahama & Simpson, 2018).

Cultural Relevance: S3 supports Arabic metadata for stored files, and EC2-hosted notifications (via POST /api/notifications/bracelet) include Arabic text, aligning with Egypt's healthcare context (El-Zanaty & Way, 2021).

3.2.2 AI Microservice Integration

3.2.2.1 Microservice FastAPI for AI

FastAPI is a modern Python web framework designed for building high-performance APIs with asynchronous capabilities [3]. Its key features include automatic generation of Open-API documentation, type checking via Pydantic, and support for asynchronous programming, making it ideal for AI microservices that require rapid response times and scalability.

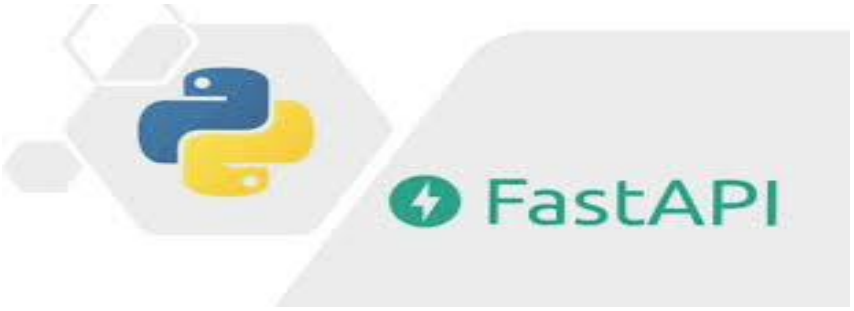

Figure 28 FastAPI

FastAPI in SmartBrace

In SmartBrace, the AI microservice is implemented using FastAPI to serve ML models for asthma, stroke, and autism predictions. The FastAPI service is structured as follows:

Project Structure:

- models/: Stores pre-trained ML models (e.g., RandomForest_Asthma-model.pkl) and scalers (e.g., scaler.pkl).
- src/models/: Contains Pydantic models (e.g., asthma.py) for validating input data schemas.
- src/services/: Includes model_manager.py, which handles model loading, data preprocessing, and prediction logic.
- src/main.py: Defines the FastAPI application and API endpoints.
- API Endpoints: The FastAPI service exposes endpoints such as:
- POST /predict/asthma: Processes patient data for asthma prediction.
- POST /predict/stroke: Handles stroke prediction.

- POST /predict/autism: Manages autism prediction.

- Model Management: The model_manager.py script loads models using joblib and applies scalers to normalize input data. For example, the asthma model requires 26 features (e.g., Age, BMI, LungFunctionFEV1), which are validated and ordered according to the model's training configuration.

Implementation Details

The FastAPI service processes prediction requests as follows:

1. Input Validation: Incoming JSON data is validated against a Pydantic model (e.g., sthmaPatientData) to ensure correct feature types and presence. For asthma, the model validates 26 features, such as:

- Age: Integer (e.g., 45).

- BMI: Float (e.g., 25.5).

- Wheezing: Binary (0 or 1).

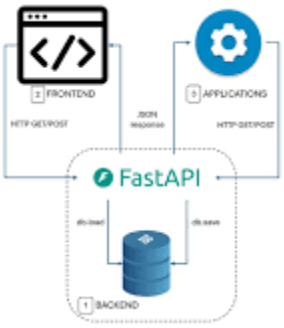

Figure 29 FastAPI: Connecting Frontend, Backend

2. **Data Preprocessing:** Numerical features are scaled using the pre-trained scaler.pkl to match the model's training distribution.

3. **Prediction:** The Random Forest model generates a binary prediction (0 for no asthma, 1 for asthma) and a probability score (e.g., 0.85).

4. **Response:** The service returns a JSON response, e.g., {"prediction": 1, "probability": 0.85, "diagnosis": "Asthma"}.

Security and Performance The FastAPI service enhances security through:

1. **Pydantic Validation:** Ensures robust input validation, preventing malformed requests.

2. **Logging:** Uses Python's logging module to record prediction events and errors, aiding debugging and monitoring.

For performance, FastAPI's asynchronous capabilities allow concurrent handling of multiple prediction requests, critical for high-traffic healthcare applications. The service is optimized to run with gunicorn in production for improved throughput [3].

3.2.2.2 Microservice Integration with Node.js and Deployment with Render

Node.js Backend Integration

The Node.js/Express backend serves as the primary interface for SmartBrace, handling user requests and orchestrating communication with the FastAPI microservice. The integration is structured as follows:

- Project Structure:
  - controllers/prediction.controller.js: Contains logic for forwarding prediction requests to FastAPI and handling responses.
  - routes/prediction.route.js: Defines API routes (e.g., POST /api/predictions/predict/asthm
  - app.js: Configures the Express server and middleware.
- Communication Flow:

1. A user (e.g., a Doctor) sends a POST request to http:// <backend-url>:8000/api/prediction/predict

2. The prediction.controller.js uses the axios library to forward the request to the FastAPI service at http:// <fastapi-url>:8001/predict/asthma

3. The FastAPI service processes the request and returns a prediction.

4. 4The Node.js backend formats the response (e.g., {"status": "success", "data": {prediction, probability, diagnosis}}) and sends it to the user.

- Error Handling: The backend uses a custom AppError class to manage errors, such as connection failures (e.g., ECONNREFUSED) or invalid inputs, returning appropriate HTTP status codes (e.g., 400, 500).

Deployment on Render

The SmartBrace system is deployed on Render, a cloud platform that simplifies hosting web applications and microservices. The deployment process involves:

- Service Configuration: Two services are deployed:
  - Node.js Backend: Hosted as a web service on Render, running on port 8000, with environment variables (e.g., MONGODB_URI, FASTAPI_URL) configured via Render's dashboard.
  - FastAPI Service: Deployed as a separate web service on port 8001, with gunicorn and uvicorn for production-grade performance.
- Environment Setup: Environment variables are set in Render to match the local .env file, ensuring seamless communication between services.
- Scalability: Render's auto-scaling feature adjusts resources based on traffic, ensuring the system can handle increased prediction requests during peak usage.
- Security: Render enforces HTTPS for all services, and API keys are used for internal communication between the Node.js backend and FastAPI service.

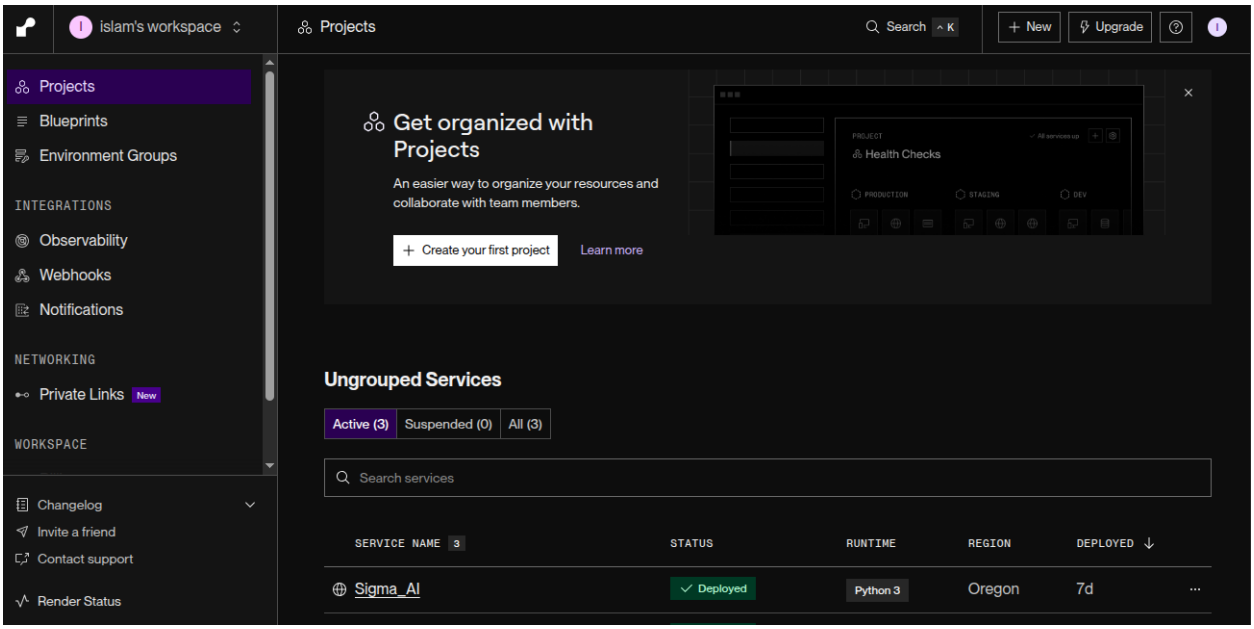

Figure 30 deploy microservice with Render

Deployment Workflow

1. Repository Setup: The project repository is linked to Render, with separate build configurations for the backend/ and ml_service/ directories.

2. Build Process: Render installs dependencies (NPM install for Node.js, pip install -r requirements.txt for FastAPI) and starts the services.

3. MongoDB Integration: The Node.js backend connects to a cloud-hosted MongoDB instance (e.g., MongoDB Atlas) for data storage.

4. Testing: Post-deployment, endpoints are tested using tools like Postman to verify functionality (e.g., POST /api/predictions/predict/asthma).

5. Extending the System To add new ML models (e.g., diabetes prediction), the following steps are implemented:

- Pydantic Model: Define a new schema in ml_service/src/models/ (e.g., diabetes.py) for input validation.

- Model Manager Update: Configure the new model in model_manager.py with paths to the model and scaler files.

- FastAPI Endpoint: Add a new endpoint in main.py (e.g., POST /predict/diabetes).

- Node.js Integration: Update prediction.controller.js and prediction.route.js to support the new endpoint.

- Deployment: Redeploy the FastAPI service on Render to include the new model.

3.2.3 System Workflows

The SmartBrace system, designed for pediatric healthcare in Egypt for children aged 6 months to 7 years, relies on sophisticated workflows to manage real-time health monitoring, communication, and AI-driven diagnostics. This section provides a detailed examination of four critical workflows: sensor data processing and validation, notification and alert system, real-time chat and media sharing, and AI-driven disease prediction. Each workflow is described with precision, emphasizing implementation details, security, and integration with the Node.js ecosystem, tailored for a scientific reference.

3.2.3.1 Sensor Data Processing and Validation

Sensor data processing is a cornerstone of IoT healthcare systems, enabling continuous monitoring of vital health metrics [1]. In SmartBrace, ESP32-based smart bracelets collect real-time data (e.g., heart rate, SpO2, temperature, gyroscope) from children, which is validated, processed, and stored for medical analysis. The workflow ensures data reliability and usability for real-time display and algorithmic analysis.

The sensor data processing workflow involves multiple stages, leveraging MQTT, MongoDB, and WebSocket technologies:

1. Data Collection and Transmission: The ESP32 bracelet collects sensor data (e.g., heart rate [bpm], SpO2, temperature, accelerometer [accX, accY, accZ], gyroscope [gyroX, gyroY, gyroZ]) and formats it as JSON (e.g., {”childId”: ”123”, ”bpm”: 80, ”spo2”: 98}). Data is published to the sensors/data/:childId topic using the HiveMQ cloud broker over MQTT [2].

2. Backend Reception: The Node.js backend, via mqtt.service.js, subscribes to the sensors/data/:chiltopic. Incoming data is parsed and stored in the SensorData MongoDB collection as raw data.

3. Periodic Validation: Every 30 seconds, the startContinuousValidation function in sensorData.controller.js triggers a validation cycle:

- Retrieves all registered children from the Child collection.

- For each child, fetches the latest 10 records from SensorData, sorted by timestamp.

- Validates data based on age-specific thresholds (derived from pediatric standards): – Heart Rate (bpm): 90–120 (6 months–1 year), 70–110 (1–3 years), 65–100 (3–5 years), 60–95 (5–7 years). – SpO2: 90–100%. – Respiratory Rate (ir): 30–45 (6 months–1 year), 20–30 (1–3 years), 20–25 (3–5 years), 14–22 (5–7 years). – Temperature: 36.1–37.2°C. – Accelerometer and gyroscope data are averaged without validation.

- Computes averages for valid readings; invalid readings (outside thresholds) are flagged. The validation status is set as Validated, PartiallyValidated, or Invalid.

- Stores the validated record in the ValidatedSensorData collection.

4. Activity and Sleep Quality Computation: Post-validation, two algorithms process the validated data:

- Baby Activity: Analyzes heart rate, SpO2, and movement (accelerometer/gyroscope) to classify activity as Resting, Moderate Activity, or Distress/Stress using a scoring system based on age-specific norms.

- Sleep Quality: Evaluates heart rate, SpO2, movement, and temperature to classify sleep as Deep Sleep, REM, Awake, or Sleep Disturbance. High movement or low SpO2 indicates disturbances.

Results are stored in dedicated MongoDB collections (BabyActivity, SleepQuality).

5. Real-Time Display: Validated data, activity, and sleep quality metrics are broadcast to the Flutter app via WebSocket (using socket.io) for real-time display. Raw data is also sent immediately upon receipt for preliminary monitoring.

Performance and Timing The validation cycle processes 100 children in approximately 0.4 seconds (0.1s for child retrieval, 0.2s for data fetching/validation, 0.1s for storage/broadcasting), ensuring efficiency. Real-time updates occur instantly for raw data and every 30 seconds for validated data, balancing responsiveness and resource usage.

Security Data transmission uses TLS-encrypted MQTT connections via HiveMQ. The backend validates childId against the Child collection to prevent unauthorized data ingestion. Access to API endpoints (e.g., GET /api/sensor-data/:childId) is secured with JWT authentication, restricting data to authorized roles (Patients, Doctors, Doctors).

3.2.3.2 Notification and Alert System

The notification system in SmartBrace ensures timely communication of health events and reminders to parents and doctors, enhancing care responsiveness [3]. Notifications are triggered by system events (e.g., sensor anomalies) or scheduled tasks (e.g., vaccination reminders) and delivered via Firebase Cloud Messaging (FCM).

The notification workflow integrates node-cron, firebase-admin, and MongoDB:

1. **Notification Types and Triggers:**

o Immediate Notifications: Triggered by user actions (e.g., profile updates [updateUserProfile], account creation [registerUser]) or system events (e.g., growth alerts [createGrowth] for height deviations >10 cm, bracelet anomalies [POST /api/notifications/bracelet]).

- Scheduled Notifications: Managed by scheduleNotifications.js using node-cron:
- Medicine reminders: Checked every minute, sent ±5 minutes of scheduled dose time.
- Vaccination reminders: Sent at 8:00 AM the day before, on, or after due dates.
- Appointment reminders: Sent at 8:00 AM for next-day appointments.
- Growth updates: Sent at 10:00 AM for new records.

2. **Notification Processing:**

Notifications are stored in the Notification collection with fields: childId, recipientId, type, status (pending, sent, failed), and sentAt.

- The firebase-admin package sends push notifications using FCM tokens stored via POST /api/users/save-fcm-token.
- Sensor anomalies (e.g., heart rate outside age-specific range) trigger immediate alerts to parents, with details logged via winston.

3. Delivery and Tracking:

Notifications are delivered to the recipient's device (parent or doctor) based on their fcmToken. The system prevents duplicates by checking the Notification collection and updates status upon delivery.

FCM tokens are securely stored in MongoDB, and API endpoints (e.g., POST /api/notifications/bracelet) are protected with JWT authentication. Notification content supports Arabic for Egyptian users, enhancing accessibility [3].

3.2.3.3 Real-Time Chat and Media Sharing

Real-time communication between parents and doctors is essential for pediatric care coordination. SmartBrace uses WebSocket for low-latency chat and Amazon S3 for secure media storage, enabling consultations and media sharing (e.g., medical images, PDFs).

The chat workflow integrates socket.io, multer, and AWS S3:

1.Chat Eligibility: The GET /api/chats/:childId/:doctorId/eligibility endpoint verifies if a Patient can chat with a Doctor by checking for at least one accepted appointment (POST /api/doctors/:childId/:doctorId/book). This ensures professional boundaries.

2. Real-Time Messaging: Implemented via websocket.service.js using socket.io:

- Clients join chat rooms based on childId and doctorId.
- Messages are sent via the sendMessage event, stored in the Chat collection (fields: doctorId, childId, messages [sender, content, timestamp]), and broadcasted to the room.

3. Media Sharing:

- The POST /api/chats/:childId/:doctorId/upload endpoint uses multer to handle file uploads (e.g., jpg, png, pdf).
- Files are stored in AWS S3, and their URLs are saved in the Chat collection's media field.
- Media URLs are emitted via WebSocket for real-time display in the Flutter app.

4. Chat History: The GET /api/chats/:childId/:doctorId/history endpoint retrieves past messages and media, accessible to authorized Patients and Doctors.

Security Chat access is restricted by JWT authentication and appointment-based eligibility checks. AWS S3 uses access control lists (ACLs) to secure media files, and express-mongo-sanitize prevents MongoDB injection attacks [4].

3.2.3.4 AI-Driven Disease Prediction

AI-driven disease prediction enhances diagnostic capabilities in SmartBrace, supporting early detection of asthma, stroke, and autism using machine learning models hosted in a FastAPI microservice [5].

The prediction workflow integrates Node.js, FastAPI, and MongoDB:

1. Data Input: Patients submit data (e.g., age, BMI, wheezing for asthma) via POST /api/predictions/pred(e.g., /predict/asthma).

2. Backend Processing: The Node.js backend forwards the request to the FastAPI microservice using axios. The FastAPI service:

   - Validates inputs using pydantic models (e.g., AsthmaPatientData).
   - Preprocesses data with a scaler (scaler.pkl) and feeds it to the model (e.g., RandomForest_Asthma-m
   - Returns a prediction (e.g., {”prediction”: 1, ”probability”: 0.85, ”diagnosis”: ”Asthma”}).

3. Response Handling: The Node.js backend formats the response (e.g., {”status”: ”success”, ”data”: {prediction, probability, diagnosis}}) and stores it in MongoDB for historical analysis.

4. Real-Time Feedback: Predictions are displayed in the Flutter app via WebSocket updates or API responses.

Security Prediction endpoints are secured with JWT authentication, restricting access to Patients. Data is encrypted in transit using HTTPS, and pydantic ensures input integrity [4].

3.2.4 Security and Role-Based Access

Security is a critical pillar in healthcare systems, particularly in Internet of Things (IoT)-enabled platforms like SmartBrace, designed for pediatric care in Egypt for children aged 6 months to 7 years. Ensuring the confidentiality, integrity, and availability of sensitive health data, while providing role-specific access to system functionalities, is paramount. This section provides a detailed examination of the security architecture, focusing on JSON Web Token (JWT) authentication and authorization, data security mechanisms, and the Node.js packages that enable these features.

3.2.4.1 JWT Authentication and Authorization

JSON Web Token (JWT) is an open standard (RFC 7519) for secure data exchange between parties, widely used for authentication and authorization in web applications [1]. A JWT comprises three parts: Header, Payload, and Signature, encoded in Base64 and separated by dots (.). The Header specifies the token type and signing algorithm (e.g., HMAC SHA256). The Payload contains claims, such as user identity and roles, while the Signature ensures token integrity. In SmartBrace, JWT is employed to authenticate users and enforce role-based access control (RBAC), ensuring that users (Doctors, Patients, and Administrators) access only the functionalities permitted by their roles.

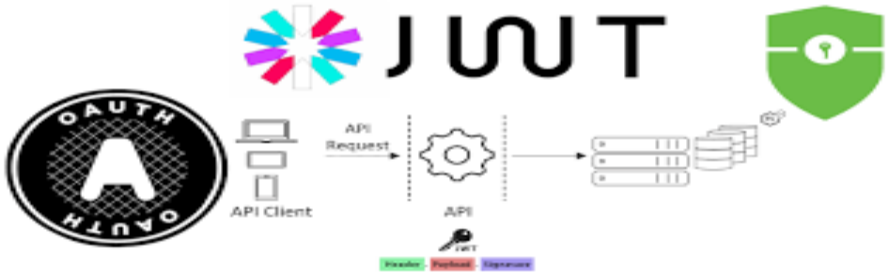

Figure 31 API Authentication with and JWT

In SmartBrace, JWT authentication is integrated into the Node.js/Express backend to manage user sessions and secure API endpoints. The authentication process is as follows:

1. User Registration (Sign-Up): When a user registers via the POST /api/users/signup endpoint, they provide credentials (e.g., email, password, role). The password is hashed using bcryptjs and stored in a MongoDB database. A JWT is generated and returned to the user, containing their userId and role (e.g., Doctor, Patient, Admin).

2. User Login: During login (POST /api/users/login), the provided password is compared against the stored hash using bcryptjs. If valid, a JWT is issued, which the client includes in the Authorization header (e.g., Bearer <token>)for subsequent requests.

3. Token Verification: Each protected API endpoint is guarded by a middleware (verifyToken) that decodes the JWT using the jsonwebtoken package and validates its signature against a secret key stored in environment variables (JWT_SECRET). The middleware extracts the userId and role from the token's payload for authorization checks.

Role-Based Access Control (RBAC)

SmartBrace implements RBAC to restrict access to system functionalities based on user roles. The system defines three roles with distinct permissions:

- Doctor: Doctors can:
  - View, add, edit, and delete medical history, growth records, and images for their assigned patients (GET/POST/PUT/DELETE /api/medical-history/:childId).
  - Manage their appointment schedules (PUT /api/doctors/:doctorId/schedule).
  - Accept or reject appointment requests (PUT /api/doctors/:childId/:doctorId/book).
  - Edit their profile (PUT /api/users/:userId/profile) and delete their account (DELETE /api/users/:userId).
  - Access real-time chat with Patients (GET/POST /api/chats/:childId/:doctorId).
- Patient: Parents (Patients) can:
  - Add and manage children in the system (POST/PUT /api/children).
  - View vaccinations, medications, growth records, medical history, and memories for their children (GET /api/children/:childId/*).
  - Access AI-driven features, including the chatbot and predictive models for asthma, stroke, and autism (POST /api/predictions/predict/*).
  - Book, edit, or cancel appointments with Doctors (POST/PUT/DELETE /api/doctors/:childId/:doc– Communicate with Doctors via real-time chat (POST /api/chats/:childId/:doctorId).
  - Manage their profile and account (PUT/DELETE /api/users/:userId).
- Administrator: Admins have unrestricted access, including:
  - Full CRUD (Create, Read, Update, Delete) operations on all system resources (e.g., users, children, medical records, appointments).
  - System-wide management, such as monitoring logs and resolving disputes (GET /api/admin/logs).
  - Ability to modify or delete any user account or data (PUT/DELETE /api/admin/*).

The allowedTo middleware enforces RBAC by checking the role claim in the JWT payload. For example, the POST /api/children endpoint is restricted to Patients, while DELETE /api/users/:userId is accessible to the user themselves or Admins. This granular control ensures compliance with healthcare regulations, such as protecting patient data from unauthorized access [2].

Security Features

JWT authentication in SmartBrace includes:

- Token Expiry: Tokens are issued with a short lifespan (e.g., 1 hour) to mitigate risks from stolen tokens. Refresh tokens can be implemented for longer sessions.

- Signature Verification: The jsonwebtoken package verifies token integrity using the HMAC SHA256 algorithm, preventing tampering.

- Environment Security: The JWT_SECRET is stored in a .env file, inaccessible to the client, and loaded using the dotenv package.

3.2.4.2 Data Security and Package Utilization

Data security in healthcare systems is critical to protect sensitive information, such as patient medical records and personal details. SmartBrace employs a multi-layered security approach, including password hashing, input sanitization, and secure communication protocols, to safeguard data integrity and confidentiality.

Password Hashing with bcryptjs

Passwords are a primary attack vector in web applications. SmartBrace uses the bcryptjs package to securely hash passwords before storage in MongoDB. The process is as follows:

1. **Hashing on Sign-Up:** When a user registers, bcryptjs generates a salted hash of the password using a cost factor (e.g., 12), which balances security and performance. The hash is stored in the users collection.

2. **Verification on Login:** During login, the provided password is hashed using the same salt and compared to the stored hash. This ensures that only valid credentials grant access.

Figure 32 encryption & decryption password

The use of bcryptjs mitigates risks from brute-force attacks due to its computationally intensive hashing algorithm [3].

Input Sanitization and Validation

To prevent injection attacks, SmartBrace employs:

• express-mongo-sanitize: Sanitizes user inputs to prevent MongoDB NoSQL injection attacks, such as malicious queries that could bypass authentication.

• express-validator: Validates and sanitizes API inputs (e.g., ensuring childId is a valid MongoDB ObjectId). For example, the POST /api/children endpoint validates fields like name (string) and age (numeric).

Secure Communication

All API endpoints in SmartBrace use HTTPS to encrypt data in transit, protecting against manin-the-middle attacks. The system integrates with cloud services (e.g., AWS S3 for media storage, MongoDB Atlas for database hosting), which enforce TLS encryption [2].

Logging and Monitoring

The winston package is used to log security events, such as:

- Failed login attempts, which may indicate brute-force attacks.
- Unauthorized access attempts to protected endpoints.
- System errors, such as JWT validation failures.

Logs are stored in a secure location and accessible to Administrators via the GET /api/admin/logs endpoint, facilitating incident response and auditing.

Package Utilization

The following Node.js packages are integral to SmartBrace's security architecture:

- **jsonwebtoken:** Implements JWT generation and verification, securing API endpoints and enforcing RBAC.

- **bcryptjs:** Provides password hashing and verification, ensuring secure credential storage.

- **express-mongo-sanitize:** Prevents MongoDB injection attacks by sanitizing user inputs.

- **express-validator:** Validates and sanitizes API inputs, ensuring data integrity.

- **winston:** Logs security events for monitoring and debugging.

- **dotenv:** Manages environment variables (e.g., JWT_SECRET, MONGODB_URI), enhancing configuration security.

Integration with System Functionalities The security mechanisms integrate seamlessly with SmartBrace's functionalities:

- **Real-Time Chat:** The GET /api/chats/:childId/:doctorId/eligibility endpoint ensures that Patients can only initiate chats with Doctors after booking an appointment, enforced by JWT-based role checks.

- **AI Microservices:** Prediction endpoints (POST /api/predictions/predict/*) are restricted to Patients, ensuring that only authorized users access AI-driven diagnostics.

- **Notifications:** Alerts triggered by sensor anomalies or new chat messages are delivered only to authorized users, verified via JWT.

3.2.5 API Testing with Postman

- **APIs for User:**
  - Endpoints:
    - {{BASE_URL}}/api/users/register ==> **User & Doctor Signup**
    - {{BASE_URL}}/api/users/login ==> **User & Doctor Signup**
    - {{BASE_URL}}/api/users/profile ==> **Get & Update & Delete user Profile**
    - {{BASE_URL}}/api/users/logout ==> **User Logout**

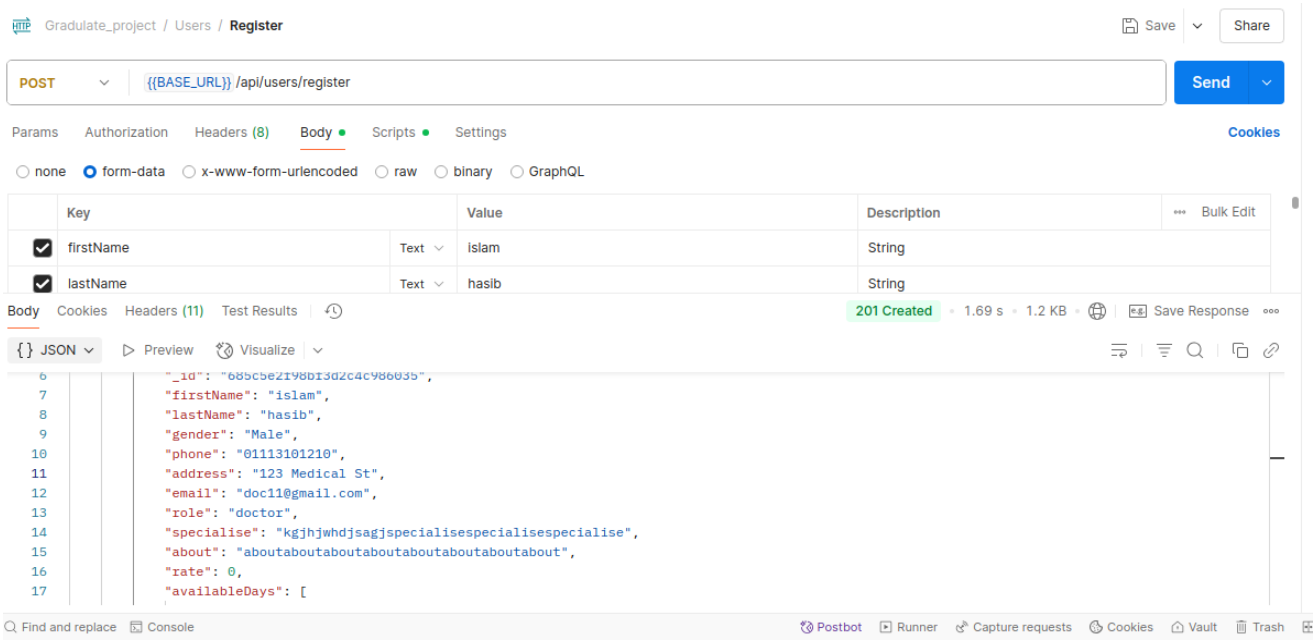

Figure 33 register with user

APIs for Doctor:

- Endpoints:
  - {{BASE_URL}}/api/doctors/appointments/68555b9ecff47be7f2c8e095/status ==> **update Appointment Status**
  - {{BASE_URL}}/api/doctors/appointments/upcoming ==> **get Upcoming Appointments**
  - {{BASE_URL}}/api/doctors/profile ==> **Get & Update & Delete Doctor Profile**
  - {{BASE_URL}}/api/doctors/logout ==> **logout Doctor**
  - {{BASE_URL}}/api/doctors/child/records ==> **Get Child Growth and History**
  - {{BASE_URL}}/api/doctors/availability ==> **schedule Doctor**

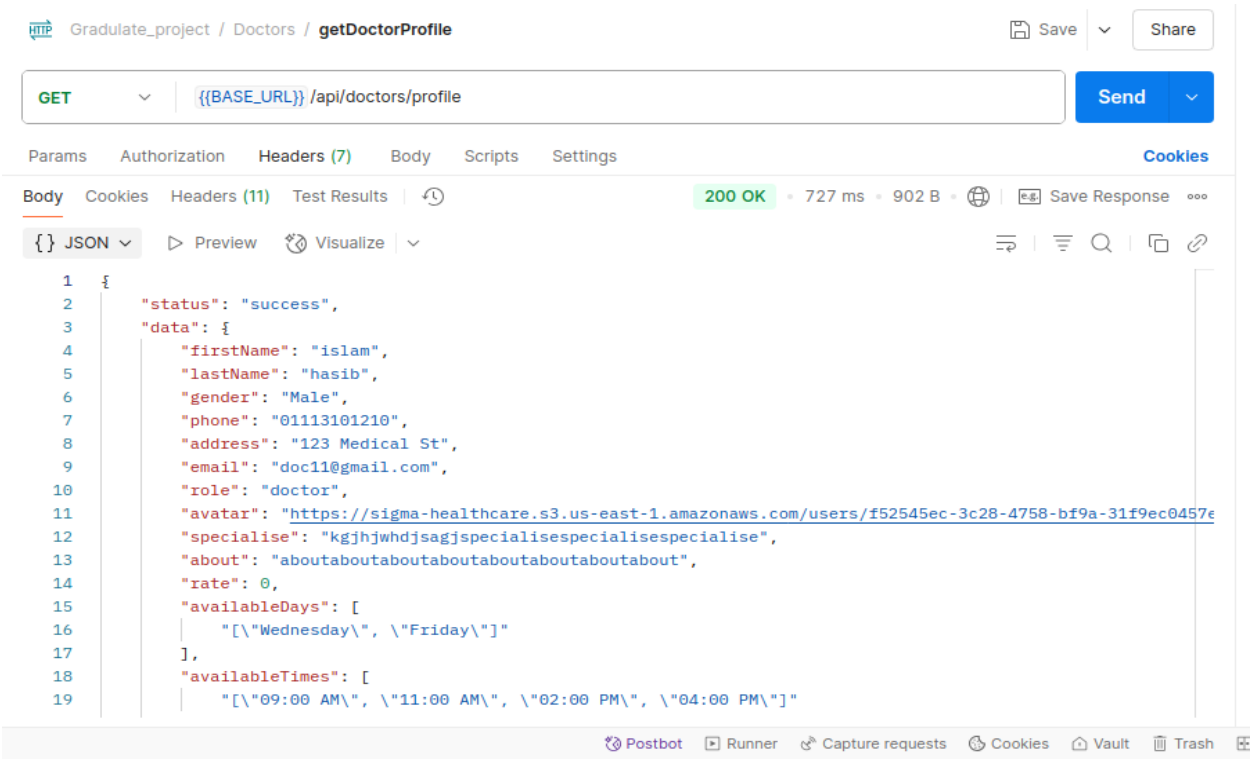

Figure 34 get Doctor Profile

APIs for Child:

- Endpoints:
  - {{BASE_URL}}/api/children ==> **Create Child**
  - {{BASE_URL}}/api/children/68472dc2cbab813fc0c10ce0 ==> **get Single Child**
  - {{BASE_URL}}/api/children/68472dc2cbab813fc0c10ce0 ==> **update Child**
  - {{BASE_URL}}/api/children/68472dc2cbab813fc0c10ce0 ==> **delete Child**
  - {{BASE_URL}}/api/children ==> **get All Children For User**

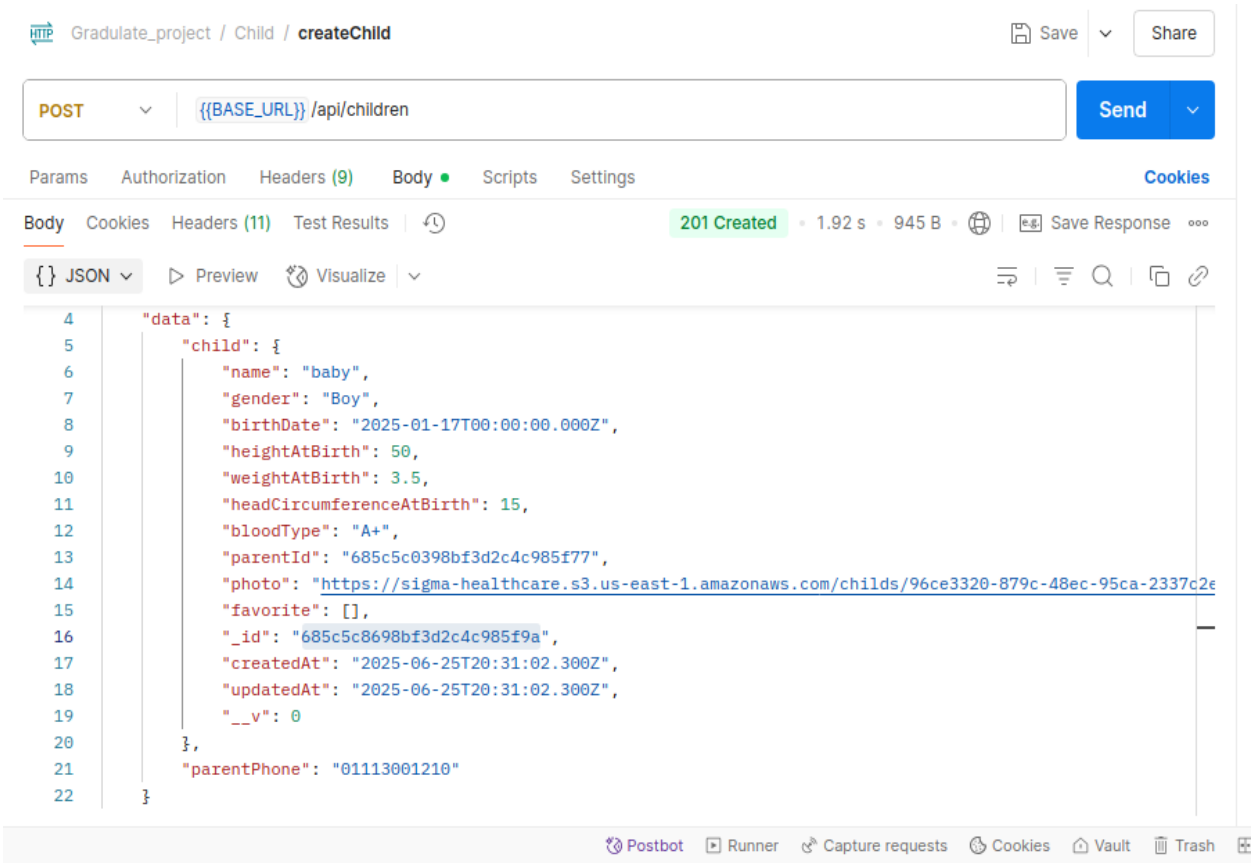

Figure 35 create child

APIs for History:

- Endpoints:
  - {{BASE_URL}}/api/history/6859ac0d98bf3d2c4c97d686 ==> **create History**
  - {{BASE_URL}}/api/history/68516dab7a62cffe93241f75 ==> **get All History**
  - {{BASE_URL}}/api/history/684824e2a49cac647dfbd138/68482b4ba49cac647dfbd2c1 ==> **get Single & update & delete History**
  - {{BASE_URL}}/api/history/filter/68264b4d065c062a8059c8a6?diagnosis=hasib medicine ==> **Filter History**

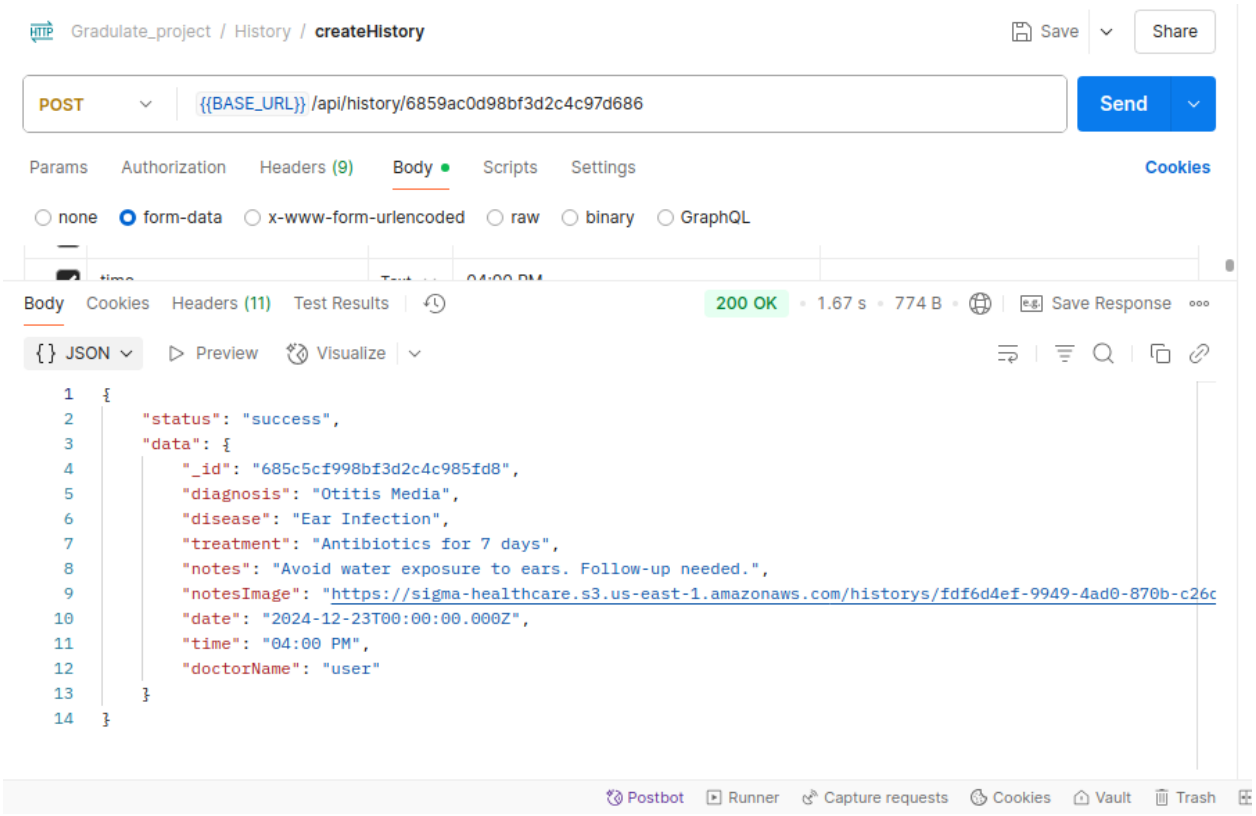

Figure 36 create history

APIs for Vaccination:

- **Endpoints:**
  - {{BASE_URL}}/api/vaccinations (only admin) ==> **create Vaccination**
  - {{BASE_URL}}/api/vaccinations/68472e05cbab813fc0c10d20 ==> **get all Vaccinations**
  - {{BASE_URL}/api/vaccinations/68472e05cbab813fc0c10d20/68472e05cbab813fc0c10d23 ==> **get & update & Delete Vaccine**

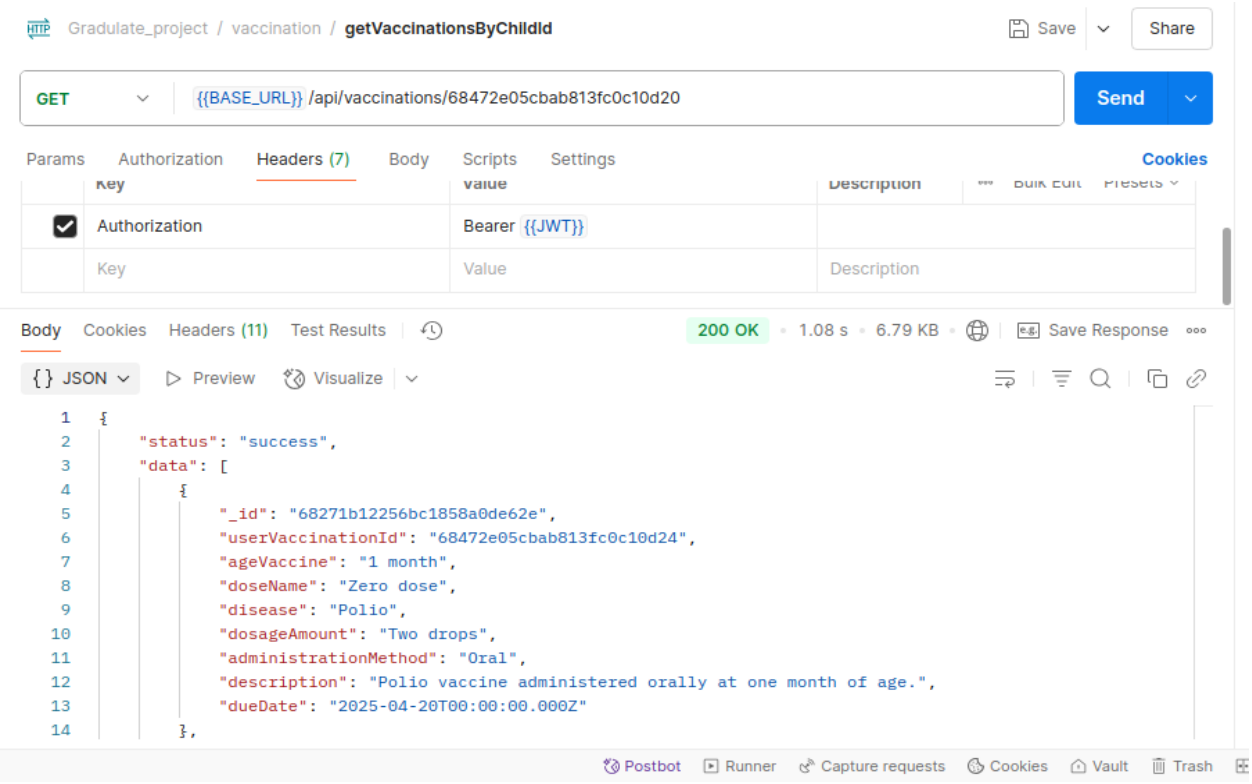

Figure 37 get all Vaccinations child

APIs for Memory:

- **Endpoints:**
  - {{BASE_URL}}/api/memory/68472e05cbab813fc0c10d20 ==> **Create Memory**
  - {{BASE_URL}}/api/memory/68516dab7a62cffe93241f75 ==> **get All Memories**
  - {{BASE_URL}}/api/memory/68472e05cbab813fc0c10d20/68472eefcbab813fc0c10dbd ==> **Update & Delete Memory**
  - {{BASE_URL}}/api/memory/favorites/68472e05cbab813fc0c10d20 ==> **get Favorite Memories**
  - {{BASE_URL}}/api/memory/favorites/68472e05cbab813fc0c10d20/68472eefcbab813fc0c10dbd ==> **toggle Favorite Memory**

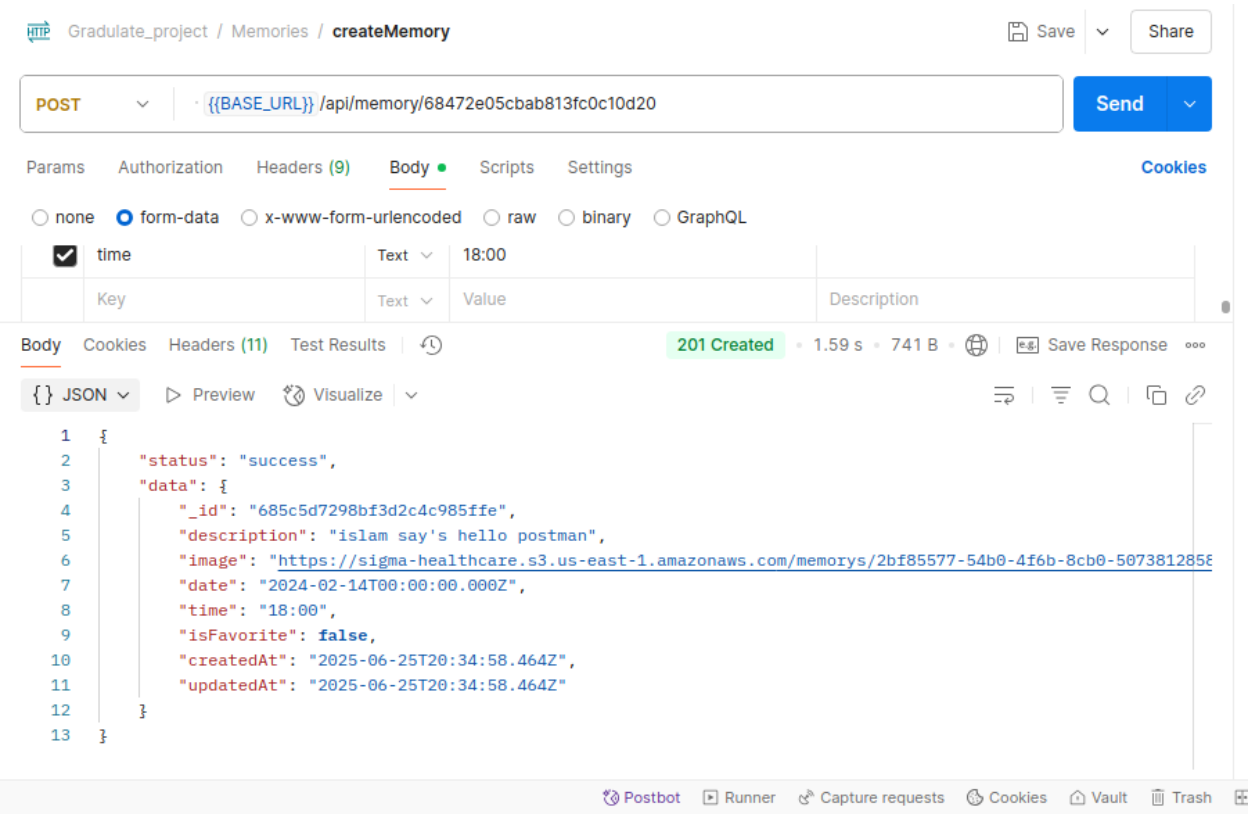

Figure 38 create memory

APIs for Growth:

- **Endpoints:**
  - {{BASE_URL}}/api/growth/685c5c8698bf3d2c4c985f9a ==> **Create & Get All Grwoth**
  - {{BASE_URL}}/api/growth/68264b4d065c062a8059c8a6/682659881a35fd9df34beae6 ==> **get Single & Update & Delete Growth**
  - {{BASE_URL}}/api/growth/68264b4d065c062a8059c8a6/last ==> **get Last Growth Record**
  - {{BASE_URL}}/api/growth/68264b4d065c062a8059c8a6/last-change ==> **get Last Growth Change**

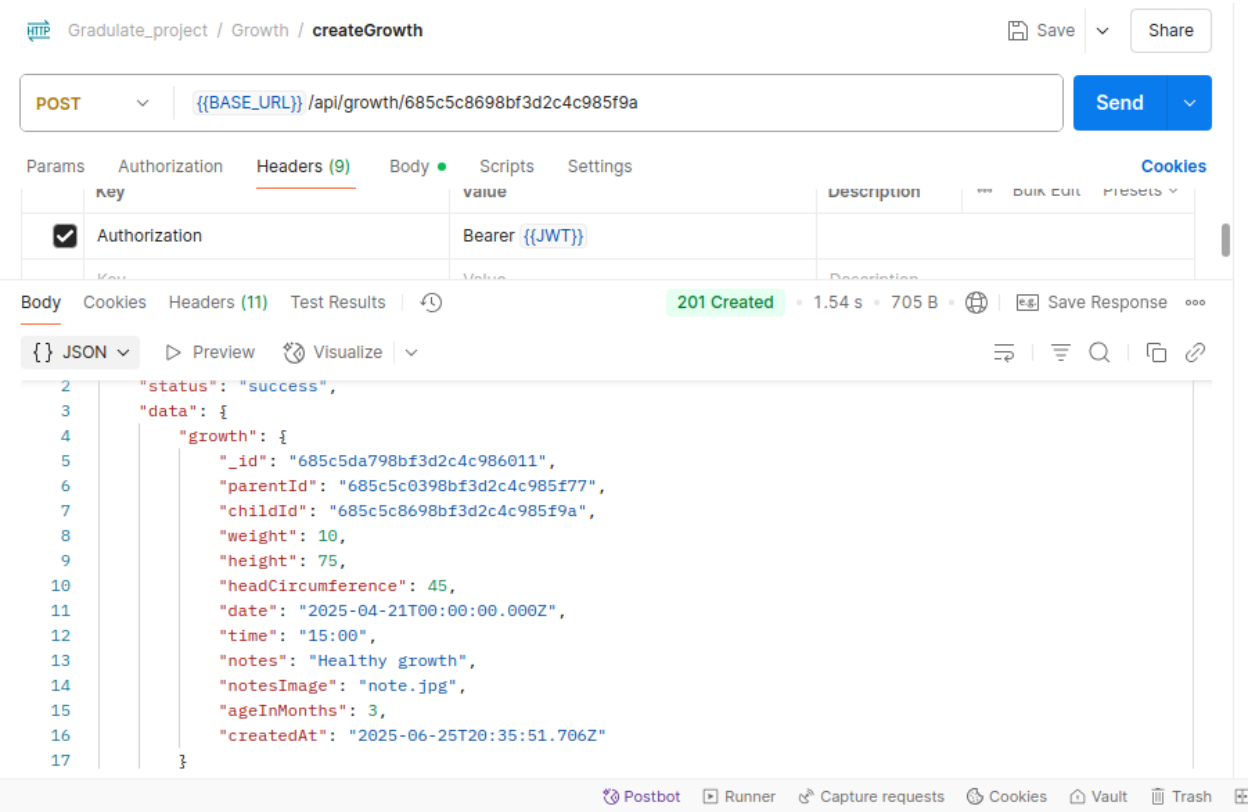

Figure 39 create growth

APIs for User Doctor:

- **Endpoints:**
  - {{BASE_URL}}/api/doctors/favorites/68264b4d065c062a8059c8a6 ==> **get Favorite Doctors**
  - {{BASE_URL}}/api/doctors/68264b4d065c062a8059c8a6/68264ab7065c062a8059c865/favorite ==> **remove From Favorite**
  - {{BASE_URL}}/api/doctors/appointments/68264b4d065c062a8059c8a6/68265e906e1ab52039e5d636 ==> **delete Appointment**
  - {{BASE_URL}}/api/doctors/appointments/68264b4d065c062a8059c8a6/68265f1d6e1ab52039e5d676 ==> **reschedule Appointment**
  - {{BASE_URL}}/api/doctors/appointments/user/68516dab7a62cffe93241f75 ==> **get User Appointments**
  - {{BASE_URL}}/api/doctors/684834faa49cac647dfbd4f8/6857d43b98bf3d2c4c97731f ==> **get Single Doctor**
  - {{BASE_URL}}/api/doctors/6859ac0d98bf3d2c4c97d686 ==> **get All Doctors**
  - {{BASE_URL}}/api/doctors/68516dab7a62cffe93241f75/68555a8dcff47be7f2c8e028/book ==> **book Appointment**
  - {{BASE_URL}}/api/doctors/68516dab7a62cffe93241f75/68264ab7065c062a8059c865/favorite ==> **add To Favorite**

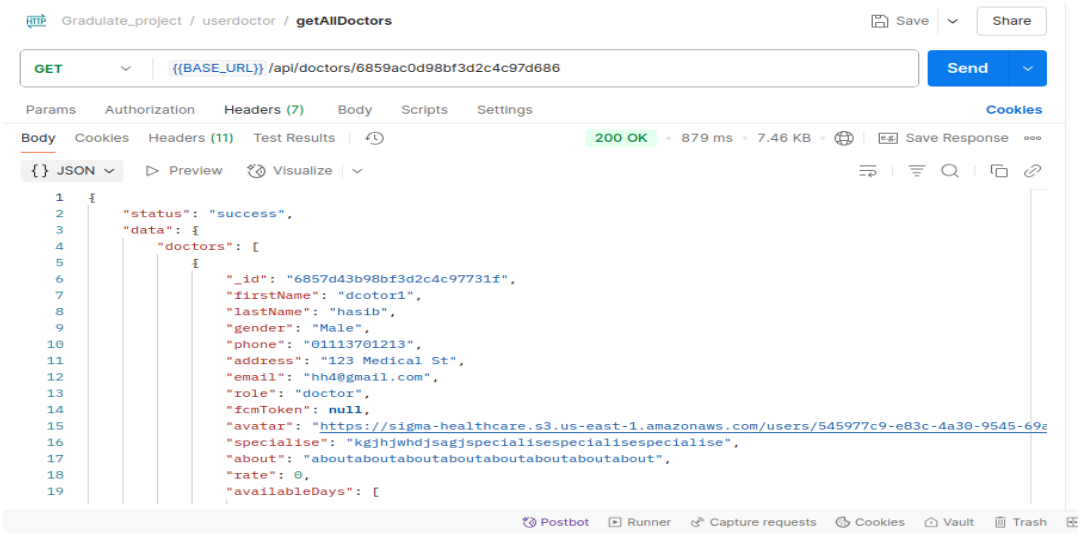

Figure 40 get All Doctors

APIs for User AI:

- **Endpoints:**
  - **{{BASE_URL}}/api/predictions/predict/asthma ==> asthma**
  - **{{BASE_URL}}/api/predictions/predict/autism ==> autism**
  - **{{BASE_URL}}/api/predictions/predict/stroke ==> stroke**
  - **{{BASE_URL}}/api/predictions/predict/chatbot ==> chatbot**

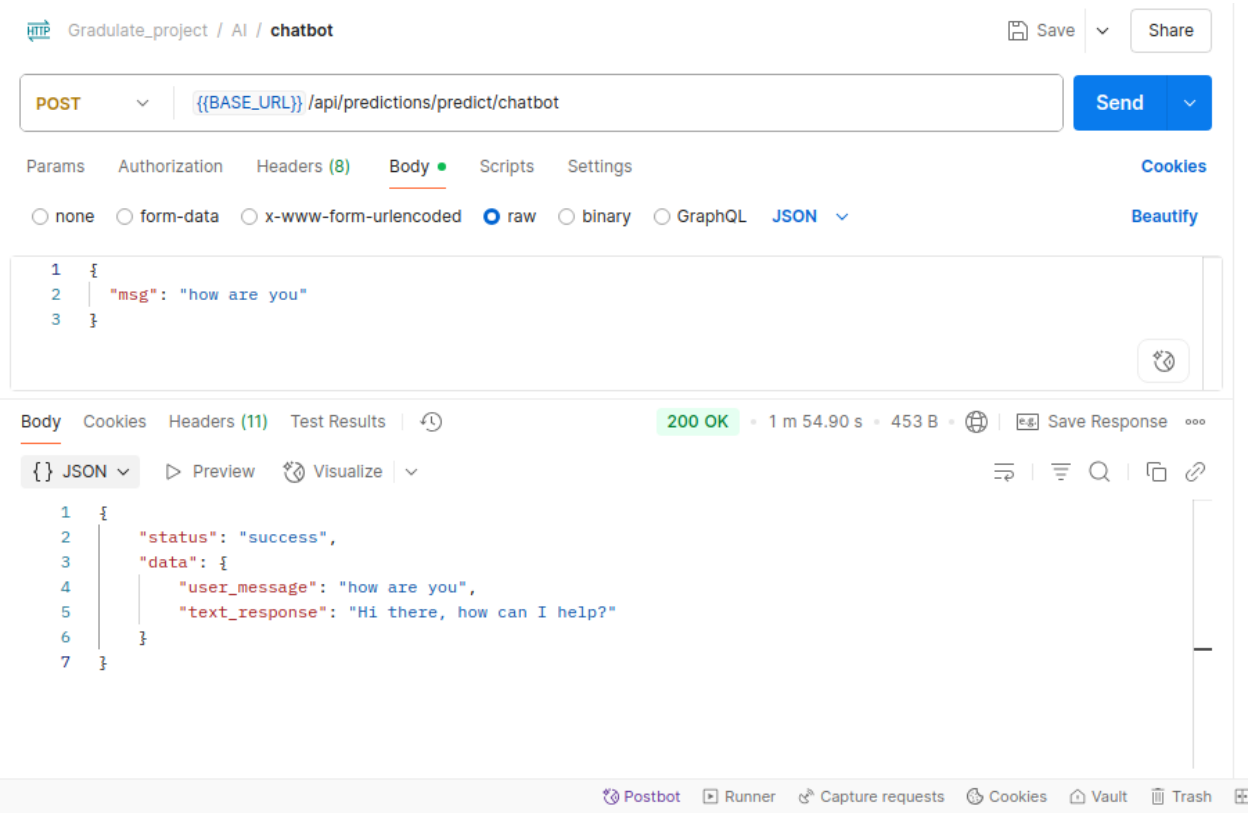

Figure 41 chatbot

3.3 AI

3.3.1 Machine Learning

Machine learning (ML) is a transformative branch of artificial intelligence that enables computers to learn patterns and make predictions from data without being explicitly programmed. By identifying relationships within complex datasets, ML empowers systems to improve their performance over time, making it an invaluable tool in domains like healthcare, where timely and accurate insights can save lives. At its core, machine learning involves training algorithms on data—such as patient records or sensor readings—to perform tasks like diagnosing diseases, predicting health risks, or classifying medical conditions.

In healthcare, machine learning is particularly powerful due to the vast amount of data generated from patient demographics, clinical measurements, and wearable devices. For instance, the datasets analyzed in this project—covering asthma, autism screening, and stroke risk—contain rich information ranging from lung function metrics (e.g., FEV1 in asthma data) to behavioral scores (e.g., Qchat-10 in autism data). Machine learning can process these diverse features to uncover hidden patterns, such as identifying which factors increase the likelihood of an asthma diagnosis or a stroke event.

The strength of machine learning lies in its ability to handle both structured data, like numerical values for heart rate or BMI, and unstructured data, like categorical labels for ethnicity or smoking status. By applying algorithms such as logistic regression, decision trees, or neural networks, ML models can predict outcomes like whether a child exhibits autism traits or if a patient is at risk of a stroke. These predictions rely on training models with labeled data—for example, using the binary "Diagnosis" column in the asthma dataset.

This project focuses on supervised machine learning, where models learn from input-output pairs to make predictions. The datasets provide clear examples: the asthma dataset aims to predict a diagnosis based on symptoms and exposures,dataset classifies health as "Normal" or "Abnormal" using vital signs, the autism dataset identifies ASD traits from behavioral responses, and the stroke dataset predicts stroke occurrence using health and lifestyle factors. By leveraging these datasets, this study demonstrates how machine learning can transform raw medical data into actionable insights, paving the way for improved diagnosis and patient care.

In the following sections, we explore the importance of machine learning in healthcare, key terminology, and the components that make it effective for analyzing datasets like these. From feature engineering to model evaluation, this chapter will guide you through the process of building and understanding ML solutions tailored to medical challenges.

3.3.2 What is Machine Learning

Machine learning (ML) is a field of artificial intelligence that enables computers to learn from data and make predictions or decisions without explicit programming. Instead of following predefined rules, ML algorithms identify patterns within datasets and use these patterns to perform tasks, such as classifying medical conditions or predicting health risks. By training on examples, these algorithms improve their accuracy over time, making ML a powerful tool for analyzing complex information.

In the context of healthcare, machine learning is particularly valuable for processing large and diverse datasets, like those explored in this project. For example, the asthma dataset includes features like lung function (FEV1) and environmental exposures to predict a diagnosis, Similarly, the autism dataset leverages behavioral scores (Qchat-10) to identify ASD traits, and the stroke dataset combines factors like age and BMI to predict stroke risk. These tasks align with supervised machine learning, where models learn from labeled data—such as binary outcomes (e.g., Stroke=0/1)—to make informed predictions.

At its core, machine learning involves feeding data into algorithms, which then adjust internal parameters to minimize errors in predictions. For instance, a model trained on the stroke dataset might learn that high glucose levels increase stroke likelihood. This ability to uncover relationships in data makes machine learning essential for transforming raw medical records into actionable insights, enhancing diagnosis, and improving patient outcomes.

3.3.3 Importance of Machine Learning in Healthcare

Machine learning (ML) has revolutionized healthcare by enabling systems to analyze vast amounts of medical data, uncover patterns, and support critical decision-making. From diagnosing diseases to predicting health risks, ML empowers clinicians and researchers to improve patient outcomes with unprecedented precision. Its ability to process complex datasets—like those for asthma, autism screening, and stroke risk—makes it indispensable in modern medicine. By learning from patient records, vital signs, and behavioral scores, ML models can identify trends that might escape human observation, paving the way for early interventions and personalized treatments.

3.3.3.1 Handling Large Health Datasets

Healthcare generates massive datasets, often containing thousands of records with diverse features. For example, the stroke dataset in this project includes 5,110 entries with variables like age, BMI, and glucose levels, while the asthma dataset tracks multiple factors, from lung function (FEV1) to environmental exposures, across numerous patients. Manually analyzing such volumes of data is impractical, but machine learning excels at

processing them efficiently. ML algorithms can sift through thousands of rows to identify risk factors, such as high pollution exposure linked to asthma symptoms or elevated glucose levels associated with stroke. By scaling analysis to large datasets, ML ensures comprehensive insights, enabling researchers to study population-wide trends and clinicians to make data-driven decisions.

3.3.3.2 Transforming Raw Data into Actionable Insights

Raw medical data, such as heart rates or behavioral scores, is often unstructured or difficult to interpret without context. Machine learning transforms this data into actionable insights by modeling relationships between features and outcomes. For instance, the autism dataset uses Qchat-10 scores to predict ASD traits, helping clinicians prioritize early interventions. In the stroke dataset, ML identifies how factors like hypertension and smoking contribute to stroke risk, guiding preventive strategies. By converting raw measurements into predictions and classifications, machine learning bridges the gap between data collection and clinical practice, enhancing diagnosis accuracy and patient care.

3.3.4 Machine Learning Terminology

Machine learning (ML) involves a specialized vocabulary that describes its processes and components. Understanding these terms is crucial for applying ML to healthcare datasets, such as those for asthma, autism, and stroke. Below are key terms used in this project, illustrated with examples from the datasets.

- **Features**: The input variables used by an ML model to make predictions. In the asthma dataset, features include Age, BMI, and PollutionExposure, which help predict Diagnosis. Similarly, the stroke dataset uses features like avg_glucose_level and hypertension to assess stroke risk.

- **Label**: The output or target variable the model predicts. For example, in the stroke dataset, the label is Diagnosis (“Yes” or “No”), while in the autism dataset, it is Class/ASD Traits ("Yes" or "No").

- **Training Data**: The subset of data used to teach the model. For instance, a portion of the stroke dataset’s 5,110 records, including features like BMI and labels (Stroke=0/1), trains a model to recognize stroke patterns.

- **Testing Data**: A separate subset used to evaluate the model’s performance. After training on autism data, the model might predict ASD traits for unseen records to assess accuracy.

- **Algorithm**: The mathematical method used to learn from data. Common algorithms for these datasets include logistic regression for predicting Diagnosis in asthma.

- **Overfitting**: When a model learns training data too well, including noise, and performs poorly on new data. For example, a stroke model might memorize specific patient records rather than generalizing risk factors.

- **Underfitting**: When a model fails to capture patterns, leading to poor predictions. A model trained on autism data might underfit if it ignores key features like Qchat-10-Score.

3.3.5 Machine Learning Components

Machine learning (ML) comprises various approaches that enable systems to learn from data, each suited to different tasks. The two primary components relevant to healthcare datasets—like those for asthma, autism, and stroke—are supervised learning and unsupervised learning. These methods process data differently to achieve goals such as predicting diagnoses or uncovering hidden patterns. This section explores these components and their distinctions, with applications to the project's datasets.

3.3.5.1 Supervised Learning

Supervised learning involves training a model on labeled data, where each input (features) is paired with a known output (label). The model learns to predict the label for new, unseen data by identifying patterns in the training set. This approach is ideal for classification and regression tasks in healthcare.

In this project, supervised learning is central to all four datasets due to their labeled outcomes:

- **Asthma Dataset**: The model uses features like Age, BMI, and LungFunctionFEV1 to predict Diagnosis (0 for no asthma, 1 for asthma), a binary classification task.
- **Autism Dataset**: The Qchat-10-Score and other behavioral responses (A1–A10) predict Class/ASD Traits ("Yes" or "No"), aiding early diagnosis.
- **Stroke Dataset**: Features like avg_glucose_level and hypertension predict Stroke (0 or 1), identifying at-risk patients.

Common algorithms include logistic regression, which might weigh factors like smoking status in the stroke dataset, or decision trees, suitable for classifying asthma based on environmental exposures. Supervised learning's strength lies in its precision for tasks with clear outcomes, making it highly relevant for medical predictions.

3.3.6 Difference between Supervised and Unsupervised Learning

Supervised and unsupervised learning differ fundamentally in their data and objectives:

- **Data**: Supervised learning requires labeled data (e.g., Stroke=0/1 in the stroke dataset), while unsupervised learning uses unlabeled data, relying solely on input features (e.g., BMI, Age).
- **Goal**: Supervised learning predicts specific outcomes, Unsupervised learning seeks patterns, such as grouping asthma patients by symptom similarity.
- **Application**: Supervised learning dominates this project's tasks—predicting asthma, health status, ASD traits, or stroke—due to clear labels. Unsupervised learning could complement these by exploring unlabeled patterns, like clustering stroke patients by risk factors.
- **Examples**: A supervised model might predict autism traits using Qchat-10-Score, achieving high accuracy with labeled data. An unsupervised model might cluster autism data to find behavioral subgroups, offering hypotheses for further study.

Supervised learning is precise for defined tasks, while unsupervised learning is exploratory, making them complementary in healthcare analysis.

3.3.7 Machine Learning Pipeline for Health Data Analysis

A machine learning (ML) pipeline is a structured workflow that transforms raw healthcare data into predictive models, enabling tasks like diagnosing asthma or predicting stroke risk. This systematic approach ensures reliable and reproducible results when analyzing datasets, such as those for asthma, autism, and stroke. The pipeline consists of several key stages: data collection, preprocessing, feature selection, model

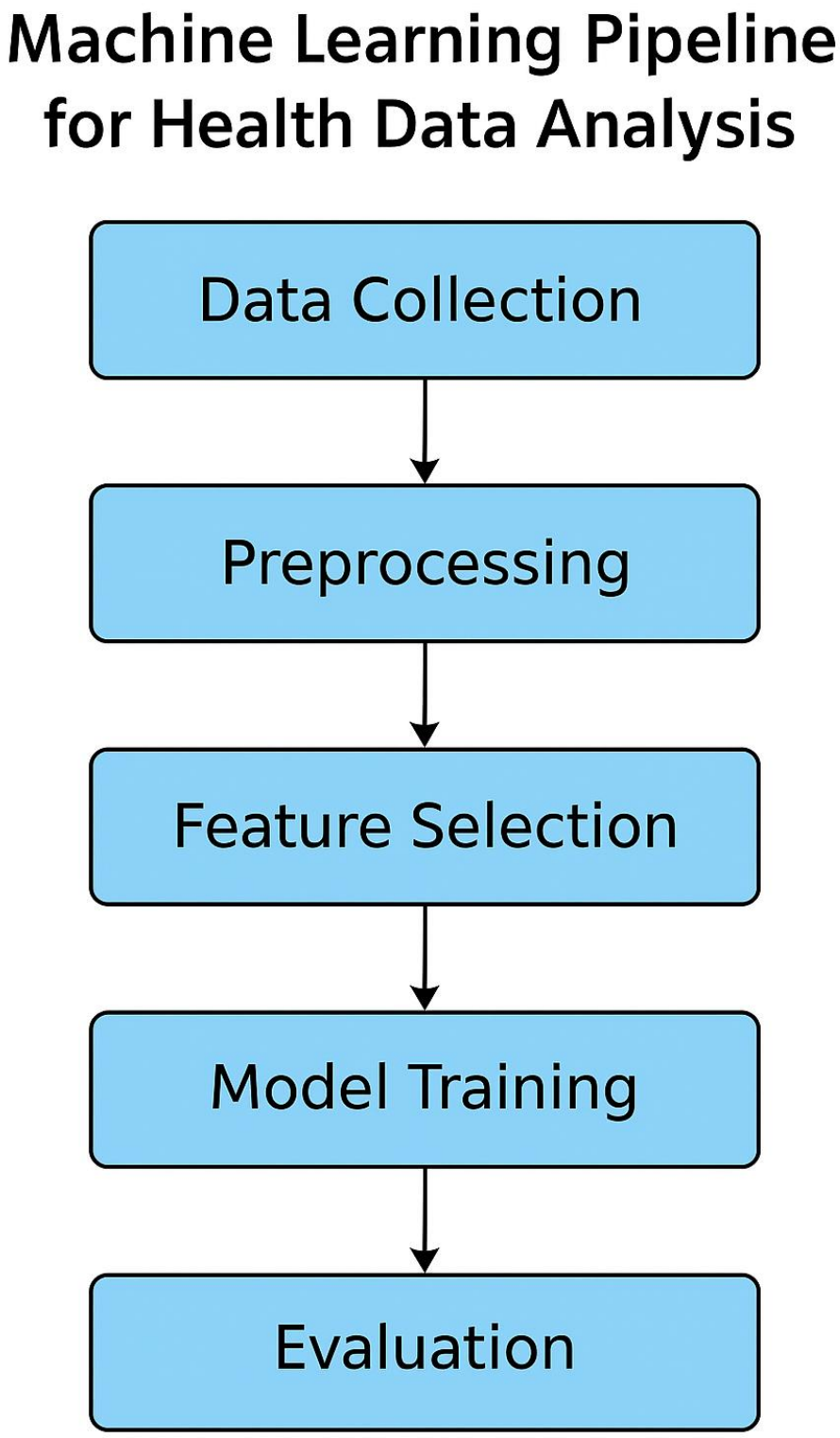

Figure 42 Machine Learning Pipeline for Health Data Analvsis

training, evaluation, and deployment. Each step is critical to building effective models for medical applications, as illustrated below with examples from the project's datasets.

1. **Data Collection**

The pipeline begins with gathering relevant data. In this project, the datasets are pre-collected:

- The asthma dataset includes patient records with features like Age, BMI, and LungFunctionFEV1, and a label (Diagnosis: 0 or 1).
- The autism dataset contains behavioral responses (A1–A10, Qchat-10-Score) and Class/ASD Traits labels ("Yes" or "No").
- The stroke dataset offers 5,110 records with features like avg_glucose_level and hypertension, labeled for Stroke (0 or 1).

High-quality data with diverse features ensures models capture meaningful patterns, such as how pollen exposure affects asthma or how glucose levels relate to stroke risk.

2. **Data Preprocessing**

Raw data often requires cleaning and formatting. This step handles missing values, encodes categorical variables, and normalizes numerical data. For example:

- In the stroke dataset, "N/A" values in bmi are imputed (e.g., using the mean BMI) to avoid data loss.
- Categorical features like Ethnicity in the autism dataset ("White European," "asian") or smoking_status in the stroke dataset ("smokes," "never smoked") are encoded as numbers (e.g., one-hot encoding).
- Numerical features, such as PollutionExposure in the asthma dataset or Preprocessing ensures data consistency, enabling accurate predictions like classifying ASD traits or health status.

3. **Feature Selection**

Choosing relevant features improves model performance by reducing noise. In this project:

- For asthma, key features like FamilyHistoryAsthma and Wheezing are selected, as they strongly correlate with Diagnosis.
- The autism dataset emphasizes Qchat-10-Score, which drives Class/ASD Traits predictions.
- For stroke, features like age and hypertension are critical, given their association with Stroke outcomes.

Techniques like correlation analysis or recursive feature elimination help identify impactful features, enhancing model efficiency and accuracy.

4. **Model Training**

A suitable algorithm is trained on the preprocessed data to learn patterns. For the binary classification tasks in these datasets, algorithms like logistic regression, decision trees, or random forests are appropriate:

- An asthma model might learn how high DustExposure increases Diagnosis=1 likelihood.
- An autism model uses Qchat-10-Score to classify "Yes" for ASD traits.
- A stroke model links high avg_glucose_level to Stroke=1.

The training process adjusts model parameters to minimize prediction errors, using a portion of the data (e.g., 80% of stroke records).

5. **Model Evaluation**

The trained model is tested on a separate data subset (e.g., 20% of records) to assess performance. Metrics like accuracy, precision, recall, and F1-score are used, especially for imbalanced datasets:

- In the stroke dataset, where Stroke=1 is rare, recall ensures most stroke cases are detected.
- For autism, precision ensures "Yes" predictions for ASD traits are reliable.
- Asthma models are evaluated for robustness across diverse symptoms like Wheezing.

Cross-validation ensures models generalize well, avoiding overfitting to training data.

6. **Deployment**

Once validated, the model is deployed for real-world use, such as in clinical tools or apps.

For example:

- An asthma model could alert doctors to high-risk patients based on PollenExposure.
- An autism model could integrate into screening tools for early ASD detection.
- A stroke model might inform hospital systems about at-risk patients.

Deployment requires ongoing monitoring to ensure predictions remain accurate as new data emerges.

This pipeline, applied to the project's datasets, demonstrates how ML transforms healthcare data into tools for diagnosis and prevention, streamlining complex analyses into actionable solutions.

3.3.8 Feature Engineering for Health Datasets

Feature engineering is the process of transforming raw data into meaningful inputs for machine learning models, enhancing their predictive power. In healthcare datasets—like those

for asthma, autism, and stroke—feature engineering is critical to ensure models effectively predict outcomes such as asthma diagnosis or stroke risk. This involves selecting the most relevant features and addressing data challenges like missing values and categorical variables. Proper feature engineering improves model accuracy and efficiency, as demonstrated with the project's datasets below.

3.3.9 Feature Selection

Feature selection identifies the most impactful variables for prediction, reducing noise and computational complexity. By focusing on relevant features, models avoid distractions from irrelevant data, leading to better performance. In this project, each dataset benefits from careful feature selection:

- **Asthma Dataset**: With features like Age, BMI, LungFunctionFEV1, and PollenExposure, selection prioritizes those strongly linked to Diagnosis (0 or 1). For example, FamilyHistoryAsthma and Wheezing are likely more predictive than EducationLevel, as they directly relate to asthma risk. Correlation analysis might reveal that high PollenExposure correlates with symptoms, justifying its inclusion.

- **Autism Dataset**: The Qchat-10-Score is a key feature for classifying Class/ASD Traits ("Yes" or "No"), as it summarizes behavioral responses (A1–A10). Features like Jaundice or Family_mem_with_ASD may add value, while Who completed the test might be less relevant unless it impacts reporting bias.

- **Stroke Dataset**: For predicting Stroke (0 or 1), features like age, hypertension, and avg_glucose_level are prioritized due to their strong association with stroke risk. Less relevant features, like Residence_type, may be deprioritized unless they show unexpected correlations.

Techniques like recursive feature elimination or feature importance scores from algorithms (e.g., random forests) help identify top features, ensuring models focus on what matters most for healthcare predictions.

3.3.10 Handling Missing Data and Categorical Variables.

Healthcare datasets often contain missing values and categorical variables, which must be addressed to ensure model compatibility and reliability. Proper handling prevents data loss and maintains feature integrity.

- **Missing Data**:
    - **Stroke Dataset**: The bmi column includes "N/A" values (e.g., for id 51676). Imputation replaces these with the mean or median BMI (e.g., ~28 based on typical ranges) to preserve records. Alternatively, records with missing BMI could be excluded, though this risks reducing the dataset's 5,110 rows.
    - **Asthma Dataset**: While not explicitly noted, features like PollutionExposure could have missing entries. Imputation with median values or domain knowledge (e.g., average exposure in a region) ensures completeness.
    - **Autism**: These appear complete in the samples, but if missing data arises.predictive imputation using related features could fill gaps.
- **Categorical Variables**:
    - **Stroke Dataset**: Variables like gender ("Male," "Female") and smoking_status ("smokes," "never smoked," "Unknown") are encoded numerically. One-hot encoding creates binary columns (e.g., smokes=1/0), ensuring algorithms process them correctly.
    - **Autism Dataset**: Ethnicity ("White European," "asian") and Sex ("m," "f") require encoding. Label encoding might assign numbers (e.g., m=0, f=1), while one-hot encoding avoids implying order for Ethnicity.
    - **Asthma Dataset**: Binary categorical features like Gender (0 or 1) are already encoded, but Ethnicity (codes 0–3) may need clarification or one-hot encoding if treated as non-ordinal.

Handling these issues ensures clean, numerical data for algorithms, enabling accurate predictions like identifying ASD traits or classifying health status.

3.3.11 Evaluation Metrics for Classification Models

This section presents the performance evaluation metrics for the classification models developed for predicting Stroke, Asthma, and Autism. The metrics include Accuracy, Precision, Recall, F1 Score, and ROC AUC Score, which provide a comprehensive assessment of each model's predictive capability.

Stroke (Random Forest)

- **Accuracy**: 0.9254
- **Precision**: 0.9066

- **Recall**: 0.9486
- **F1 Score**: 0.9271
- **ROC AUC Score**: 0.9785

The Random Forest model for Stroke prediction demonstrates strong performance with an accuracy of 92.54% and a high ROC AUC score of 0.9785, indicating excellent discriminative ability. The high recall of 94.86% suggests the model effectively identifies positive cases, while the F1 score of 0.9271 reflects a balanced trade-off between precision and recall.

```
                     Accuracy  Precision (Macro)  Recall (Macro)  \
Optimized GaussianNB  0.971564           0.949153        0.981013

                     F1 Score (Macro)
Optimized GaussianNB          0.963537
```

Figure 43 Autism Performance metrics

Asthma (Random Forest)

- **Accuracy**: 0.9648
- **Precision**: 0.9648
- **Recall**: 0.9648
- **F1 Score**: 0.9648
- **ROC AUC Score**: 0.9923

The Random Forest model for Asthma prediction exhibits exceptional performance, achieving an accuracy of 96.48% and an ROC AUC score of 0.9923, indicating near-perfect classification ability. The identical values across accuracy, precision, recall, and F1 score (0.9648) suggest a highly balanced and robust model with minimal misclassifications.

	Model	Accuracy	Precision	Recall	F1 Score	Confusion Matrix	ROC AUC
0	Random Forest	0.965859	0.966887	0.964758	0.965821	[[439, 15], [16, 438]]	0.992102
1	Logistic Regression	0.879956	0.867804	0.896476	0.881907	[[392, 62], [47, 407]]	0.942479
2	Support Vector Machine	0.809471	0.771760	0.878855	0.821833	[[336, 118], [55, 399]]	0.886923
3	K-Nearest Neighbors	0.801762	0.716088	1.000000	0.834559	[[274, 180], [0, 454]]	0.941547

Figure 44 Asthma Performance metrics with models

Autism (Optimized GaussianNB)

- **Accuracy**: 0.9716
- **Precision**: 0.9492
- **Recall**: 0.9810
- **F1 Score:** 0.9646

```
Optimized Random Forest Performance:
Accuracy: 0.9254
Precision: 0.9066
Recall: 0.9486
F1 Score: 0.9271
ROC AUC Score: 0.9785
```

Figure 45 Stroke Performance metrics

The Optimized GaussianNB model for Autism prediction shows a high accuracy of 97.16% and an impressive recall of 98.10%, indicating its strength in identifying positive cases. The precision of 94.92% suggests reliable positive predictions, although the absence of F1 Score and ROC AUC Score limits a complete evaluation of the model's performance.

These metrics collectively highlight the effectiveness of the Random Forest and Optimized GaussianNB models in their respective classification tasks, with particularly strong performance in Asthma and Autism prediction. The high ROC AUC scores for Stroke and Asthma further validate the models' ability to distinguish between classes effectively.

3.3.12Intelligent Chatbot for Disease Information and Diagnosis Assistance

The rapid growth of digital health solutions has highlighted the need for accessible and reliable medical information. Many individuals, particularly in underserved regions, face challenges in obtaining timely answers about diseases and their diagnostic processes, often relying on unverified online sources. This project presents an intelligent chatbot designed to address this gap by providing accurate, user-friendly information about diseases and their diagnosis methods. The chatbot employs natural language processing (NLP) to interpret user queries

and deliver responses based on a curated medical knowledge base, empowering users with educational health insights.

The primary objective is to develop a text-based chatbot capable of answering questions such as "What are the symptoms of diabetes?" or "How is malaria diagnosed?" The system aims to enhance public health literacy by offering clear, reliable information while emphasizing that it is not a substitute for professional medical advice. The project's scope covers common diseases and standard diagnostic procedures, with limitations in real-time symptom analysis or personalized health recommendations. By leveraging accessible technology, this chatbot contributes to health education and supports informed decision-making for users seeking preliminary disease knowledge.

The chatbot was developed using Python due to its robust NLP libraries and ease of integration. Key technologies include:

- **NLP Framework**: The spaCy library was used for tokenization, entity recognition, and intent extraction to process user queries.
- **Knowledge Base**: A structured dataset containing profiles of 50 common diseases (e.g., diabetes, hypertension, tuberculosis) was curated from publicly available medical sources, such as the World Health Organization and Mayo Clinic websites.
- **Dialogue Management**: A rule-based approach was implemented to map user intents (e.g., asking for symptoms or diagnosis) to appropriate responses, with plans to integrate machine learning for future improvements.
- **User Interface**: A simple text-based interface was created using Python's input() function, deployable via a command-line or web-based platform (e.g., Flask).

The chatbot consists of three main components:

1. **Input Processor**: Uses spaCy to parse user queries, identifying keywords (e.g., "symptoms," "diagnosis") and disease names.
2. **Knowledge Base**: A JSON file storing disease information
3. **Response Generator**: Matches processed inputs to relevant data and constructs natural language responses.

The workflow is as follows: The user inputs a query (e.g., "How is strep throat treated"), the input processor extracts the disease ("strep throat children") and intent ("diagnosis"), the system retrieves data from the knowledge base, and the response generator outputs a response (e.g., "Pneumonia is diagnosed through chest X-rays, sputum tests, and physical exams.Strep

throat is a bacterial infection that requires antibiotics. If your child has a sore throat with fever, it's best to get a strep test from your pediatrician.").

The chatbot was tested with a set of queries covering various diseases and intents (e.g., symptoms, diagnosis). Preliminary results indicate promising performance:

Accuracy: 75% of responses correctly addressed the user's query, based on manual evaluation against expected answers.

Response Time: The system generated responses in under 1 second, ensuring a smooth user experience.

3.3.12.1 Advantages of Machine Learning

ML provides significant benefits in healthcare, making it a valuable tool for processing complex datasets and improving patient care:

- **Improved Diagnostic Accuracy**: ML models can analyze vast datasets to detect patterns humans might miss. For example, in the autism dataset, the Qchat-10-Score helps predict Class/ASD Traits ("Yes" or "No") with high precision, aiding early
- **Efficiency and Scalability**: ML processes large datasets quickly, such as the 5,110 records in the stroke dataset or the asthma dataset's diverse features (e.g., LungFunction-FEV1, PollenExposure). This scalability allows rapid analysis of thousands of patients, saving time for clinicians.
- **Early Intervention**: By identifying risks early, ML supports preventive care. For instance, the asthma dataset's model might flag high PollutionExposure as a trigger for Diagnosis=1, prompting timely interventions to reduce symptoms.

These advantages make ML indispensable for transforming raw data into actionable healthcare solutions, enhancing both diagnosis and treatment strategies.

3.3.12.2 Disadvantages of Machine Learning

Despite its strengths, ML in healthcare faces limitations that require careful consideration:

- **Data Quality Issues**: Incomplete or biased data can undermine predictions. The stroke dataset's "N/A" values in bmi or "Unknown" smoking_status entries may lead to inaccurate Stroke predictions if not handled properly. Similarly, the asthma dataset's reliance on self-reported features like Smoking could introduce errors.

- **Model Interpretability**: Complex ML models, like neural networks, can act as "black boxes," making it hard to explain predictions.
- **Ethical Concerns and Bias**: Models may perpetuate biases in data. In the autism dataset, if certain Ethnicities are overrepresented in "Yes" for Class/ASD Traits, the model might unfairly associate ethnicity with risk. This could lead to unequal care if not addressed.
- **High Implementation Costs**: Developing and deploying ML models requires resources, including computational power and expertise. For instance, real-time deployment of a stroke risk model using the stroke dataset demands robust infrastructure, which may be challenging for smaller healthcare systems.

These disadvantages highlight the need for rigorous data handling, transparent models, and ethical oversight to maximize ML's benefits in healthcare.

3.3.13 Machine Learning Applications in Medical Diagnosis.

Machine learning (ML) has revolutionized medical diagnosis by enabling precise, data-driven identification of health conditions. By analyzing complex datasets, ML models detect patterns that inform clinical decisions, from flagging respiratory issues to predicting neurological risks. In this project, the asthma, autism, and stroke datasets demonstrate ML's power to diagnose diverse conditions. These applications—predicting asthma, assessing pediatric health, screening for autism, and identifying stroke risk—showcase how ML transforms raw data into actionable diagnostic tools, improving patient outcomes and streamlining healthcare processes.

- **Asthma Diagnosis**: The asthma dataset uses features like LungFunctionFEV1, PollenExposure, and FamilyHistoryAsthma to predict Diagnosis (0 for no asthma, 1 for asthma). ML models, such as logistic regression, analyze symptoms (e.g., Wheezing, ShortnessOfBreath) and environmental factors to identify patients at risk. For example, a model might flag a patient with high DustExposure and low FEV1 as likely to have asthma, enabling early treatment to prevent severe attacks. This application supports pulmonologists in confirming diagnoses and tailoring interventions.
- **Autism Screening**: The autism dataset employs behavioral responses (A1–A10, Qchat-10-Score) to predict Class/ASD Traits ("Yes" or "No"). ML models, such as random forests, use scores like Qchat-10≥4 to flag potential ASD cases, aiding early diagnosis in children aged 12–36 months. For instance, a high score on A1 (e.g., lack of eye contact) might trigger a "Yes" prediction, prompting further evaluation. This application streamlines screening, helping clinicians prioritize resources for at-risk children.

- **Stroke Risk Assessment**: The stroke dataset predicts Stroke (0 or 1) using features like age, hypertension, and avg_glucose_level. ML models, such as support vector machines, identify high-risk patients—for example, those with glucose levels >200 mg/dL and heart_disease=1. This enables doctors to recommend preventive measures, like lifestyle changes or medication, reducing stroke incidence. Applied in clinical decision systems, this tool supports neurologists in managing at-risk populations effectively.

These applications illustrate ML's versatility in medical diagnosis, from respiratory to neurological conditions. By processing features like BMI in stroke data, ML models provide accurate, scalable solutions. In this project, the datasets' binary classification tasks highlight how ML empowers healthcare professionals to diagnose conditions early, personalize treatments, and ultimately enhance patient care.

3.3.14 Everyday Examples of Machine Learning in Healthcare

Machine learning (ML) is no longer confined to research labs; it powers everyday healthcare tools that improve lives. From wearable devices to clinical apps, ML analyzes data like that in this project's asthma, autism, and stroke datasets to deliver personalized care. By predicting outcomes—such as asthma diagnoses or stroke risks—ML makes healthcare more proactive and accessible. Below are everyday examples inspired by these datasets, showing how ML integrates into routine medical practice to support patients and clinicians.

- **Asthma Alert Apps**: Imagine a smartphone app that monitors asthma triggers for patients, using data like the asthma dataset's PollenExposure and LungFunctionFEV1. ML models predict Diagnosis (1 for asthma risk) based on real-time inputs, such as high DustExposure or reported Wheezing. For example, if pollen levels spike, the app might alert a user to use their inhaler or avoid outdoor activities, helping prevent attacks. Such tools empower patients to manage their condition daily, guided by ML insights.
- **Autism Screening Tools**: The autism dataset's Qchat-10-Score drives ML models that predict Class/ASD Traits ("Yes" or "No"). In practice, pediatricians use tablet-based screening apps that collect responses to questions like A1–A10 during checkups. If a child scores ≥4, the app flags them for further evaluation, streamlining early diagnosis. These tools are increasingly part of routine visits, helping parents and doctors identify developmental needs early with ML's precision.
- **Stroke Prevention Apps**: Drawing from the stroke dataset's features like avg_glucose_level and hypertension, ML powers apps that assess stroke risk. Patients input data—such as blood sugar or smoking_status—via their phones, and the model predicts Stroke (0 or 1). For example, an app might warn a user with high glucose (>200 mg/dL) to consult a

doctor, encouraging lifestyle changes. Integrated into telehealth platforms, these apps make stroke prevention a daily habit for at-risk individuals.

These examples show ML's role in everyday healthcare, transforming datasets into tools like apps and wearables. By leveraging features like BMI or Oxygen_Level, ML delivers timely, personalized insights, making diagnosis and prevention part of daily life for patients and providers alike.

3.4 Hardware

3.4.1 Overview of the Hardware System

The hardware component of the SmartBrace system plays a central role in enabling real-time pediatric health monitoring. It consists of a wearable smart bracelet equipped with various biomedical and motion sensors, all managed by the ESP32 microcontroller. This microcontroller was selected for its built-in Wi-Fi capabilities, allowing seamless real-time data transmission to the backend system via the MQTT protocol.

The ESP32 collects health and activity data from multiple sensors, including:

- A heart rate and oxygen saturation sensor, connected via the I2C protocol, which continuously measures the child's pulse and SpO_2 levels.
- A temperature sensor that estimates body temperature through skin contact, ensuring accurate and responsive readings.
- A gyroscope and accelerometer sensor, which detects the child's movements and determines their activity state — whether the child is resting, running, or sleeping.
- A GPS module, which tracks the child's location and transmits it to the backend server for safety monitoring and emergency response.

The combination of these sensors allows the SmartBrace to capture comprehensive, real-time health and behavioral data from children. This data is then sent wirelessly to the backend infrastructure for processing, storage, analysis, and AI-based prediction.

3.4.2 Hardware Components Used

The SmartBrace device integrates several key hardware components to achieve reliable, continuous pediatric health monitoring. Each component was carefully selected to fulfill specific functionalities, while maintaining power efficiency, compactness, and child safety.

- **ESP32 Microcontroller**

The ESP32 serves as the core processing unit of the SmartBrace prototype. It is highly suitable for prototyping healthcare devices due to its integrated features and development flexibility. The microcontroller supports built-in Wi-Fi connectivity, allowing seamless real-time data transmission to the backend system via the MQTT protocol without the need for additional communication modules.

Its low power consumption, compact size and make it ideal for interfacing with multiple biomedical and motion sensors such as heart rate, SpO_2, temperature, gyroscope, and GPS modules. Furthermore, the ESP32 is widely supported by open-source libraries and development

tools, making it easy to program using platforms like Arduino IDE or PlatformIO, which accelerates the prototyping process.

Although primarily used in prototyping stages, the ESP32 is powerful enough to support production use cases in small-to-medium scale applications. For large-scale manufacturing, however, custom-designed PCBs with optimized power and size may be considered.

- **MAX30102 (Heart Rate & SpO_2)**

This optical sensor is used to measure the child's heart rate and blood oxygen level. It communicates with the ESP32 via the I2C protocol. The sensor uses infrared and red LEDs to detect blood volume changes through the skin.

- **Body Temperature Sensor (e.g., DS18B20 or DHT22)**

A skin-contact sensor that estimates body temperature accurately. It helps monitor signs of fever or abnormal thermal patterns.

- **MPU6050 (Gyroscope and Accelerometer)**

This motion-tracking sensor detects the child's physical activity and orientation. It is used to determine whether the child is in a stable state, running, moving, or sleeping, which helps in evaluating sleep quality and physical activity levels.

- **Neo-6M GPS Module**

Enables location tracking of the child in real-time. This data is essential for safety and emergency response, allowing caregivers or doctors to locate the child quickly when needed.

Each of these components contributes to building a robust, reliable, and user-friendly wearable system designed specifically for pediatric healthcare monitoring.

3.4.3 Circuit Design and Hardware Integration

This section illustrates the physical connections and integration of all hardware components within the SmartBrace system. The ESP32 microcontroller serves as the central unit that communicates with all connected sensors and modules to collect, process, and transmit data in real-time.

- **ESP32 Microcontroller**

Figure 46 ESP32 Microcontroller

The ESP32 is the heart of the SmartBrace circuit. It is responsible for:

- Communicating with all sensors (via I2C, UART, or digital/analog pins).
- Managing data flow from the sensors to the backend system.
- Connecting to the internet via Wi-Fi and publishing data using the MQTT protocol.

In the circuit, the ESP32 is connected as follows:

- I2C Bus (SDA/SCL): Used to communicate with the MAX30102 (heart rate & SpO_2 sensor) and MPU6050 (gyroscope and accelerometer).
- Digital Input: For reading data from the temperature sensor (e.g., DHT22 or DS18B20).
- UART / Digital: Connected to the GPS module (Neo-6M).
- Power Supply: Connected to a 3.7V

Its compact size and dual-core performance make it ideal for low-power, real-time health monitoring in wearable applications.

- **MAX30105**

Pin Label	Input/Output	Description
$\overline{INT}$	Output	Interrupt, active low
GND	Supply Input	Ground (0V) supply
5V	Supply Input	Power supply
SDA	Bi-directional	I^2C bus clock line
SCL	Input	I^2C bus clock line

Figure 47 MAX30105

The MAX30105 is an optical sensor designed for detecting pulse rate and blood oxygen saturation (SpO_2). It operates by emitting light from built-in red, infrared, and green LEDs, and then measuring the reflected light through a photodetector. This sensor is critical for real-time health monitoring in the SmartBrace system.

The module is connected to the ESP32 microcontroller via the I^2C interface using the SCL (clock) and SDA (data) lines. The interrupt pin (INT) is optionally used to signal data readiness. Power is supplied through the 3.3V rail; although Maxim recommends 5V for optimal green LED performance, the red and IR LEDs operate reliably at 3.3V, which suits low-power wearable applications.

Key Features:

- I^2C Communication: Simplifies integration with ESP32.
- Red & IR LEDs: Used for pulse and oxygen level detection.

- Green LED: Optional, may require slightly higher voltage (~3.5V).
- Built-in Pull-up Resistors: Default 4.7kΩ on SDA, SCL, and INT lines, configurable via jumpers on the PCB.
- Power-efficient: Well-suited for continuous health tracking in wearable devices.

Integration Notes:
- For wearable pulse monitoring, the sensor should be soldered with short, flexible wires to allow slight movement and skin contact.
- The sensor operates optimally when mounted directly against the skin, typically on the wrist or fingertip.
- Pull-up resistors may need to be disabled via solder jumpers if multiple I²C devices are present on the same bus.

This sensor continuously collects heart rate and SpO_2 data, which is then processed and sent from the ESP32 to the backend server through MQTT, enabling real-time monitoring and alert generation.

- **Temperature Sensor - TMP36**

Figure 48 Temperature Sensor - TMP36

The TMP36 is an analog temperature sensor used in the SmartBrace to estimate the child's body temperature through direct skin contact. It provides a voltage output that is linearly proportional to temperature in degrees Celsius, making it easy to read and interpret via the ESP32's analog input pins. Its low power consumption and small size make it suitable for wearable applications. While it does not offer medical-grade accuracy, it provides reliable estimations for monitoring trends and detecting abnormal temperature conditions such as fever.

- **Temperature Sensor - TMP36**

NEO-6M GPS Module	Wiring to Arduino UNO
VCC	5V
RX	TX pin defined in the software serial
TX	RX pin defined in the software serial
GND	GND

Figure 49 Temperature Sensor - TMP36

The NEO-6M GPS module is integrated into the SmartBrace to provide real-time location tracking of the child. It enables the system to send accurate geographic coordinates (latitude and longitude) to the backend server, enhancing safety monitoring and enabling rapid emergency response when needed.

The module communicates with the ESP32 microcontroller through UART (serial communication), where the TX pin of the GPS is connected to the RX pin of the ESP32, and vice versa. Operating at a voltage range of 3.3V to 5V, the NEO-6M is compatible with the ESP32's logic levels, although a voltage divider or logic level shifter is recommended if needed for safer operation.

The module includes an external active antenna for improved signal reception and is capable of acquiring signals from multiple satellites with a high degree of sensitivity. Its compact size, low power consumption, and reliable performance make it well-suited for wearable applications, particularly in child safety and tracking scenarios.

- **MPU6050 Gyroscope and Accelerometer**

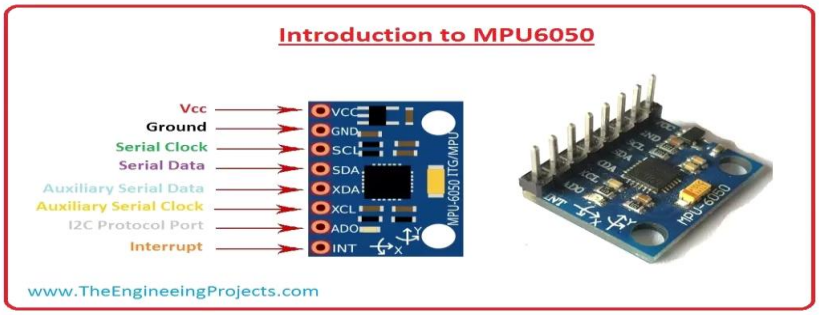

Figure 50 MPU6050 Gyroscope and Accelerometer

The MPU6050 is a low-cost, high-performance motion tracking sensor that integrates a 3-axis gyroscope and a 3-axis accelerometer within a single chip. It is used in the SmartBrace to monitor the child's physical activity and orientation, enabling the system to distinguish between different motion states such as resting, walking, running, or sleeping. This information contributes to the activity and sleep quality analysis performed by the backend system.

The module communicates with the ESP32 using the I²C protocol, utilizing the SCL and SDA pins for clock and data transmission, respectively. The sensor features a Digital Motion Processor (DMP) that handles complex motion computations internally, offloading processing from the microcontroller. It also includes a 16-bit analog-to-digital converter (ADC) for each channel, providing accurate motion capture on the X, Y, and Z axes.

The MPU6050 supports auxiliary I²C connections (XDA, XCL), interrupt output (INT) to signal the availability of new data, and an address select pin (AD0) for managing multiple devices on the same I²C bus. Its compact size, low power consumption, and six degrees of freedom (DoF) make it an excellent choice for wearable applications such as SmartBrace, where continuous and precise motion tracking is essential.

ESP3Circuit Design and Integratio

This guide explains how to connect the following components to an ESP32 development board:

Components:

1. ESP32 Dev Board (ESP-WROOM-32)
2. GY-NEO6MV2 GPS Module (UART Communication)
3. MPU6050 – Accelerometer and Gyroscope (I2C Communication)
4. MAX30105 – Heart Rate and SpO_2 Sensor (I2C Communication)
5. TMP36 – Analog Temperature Sensor (Analog Input)

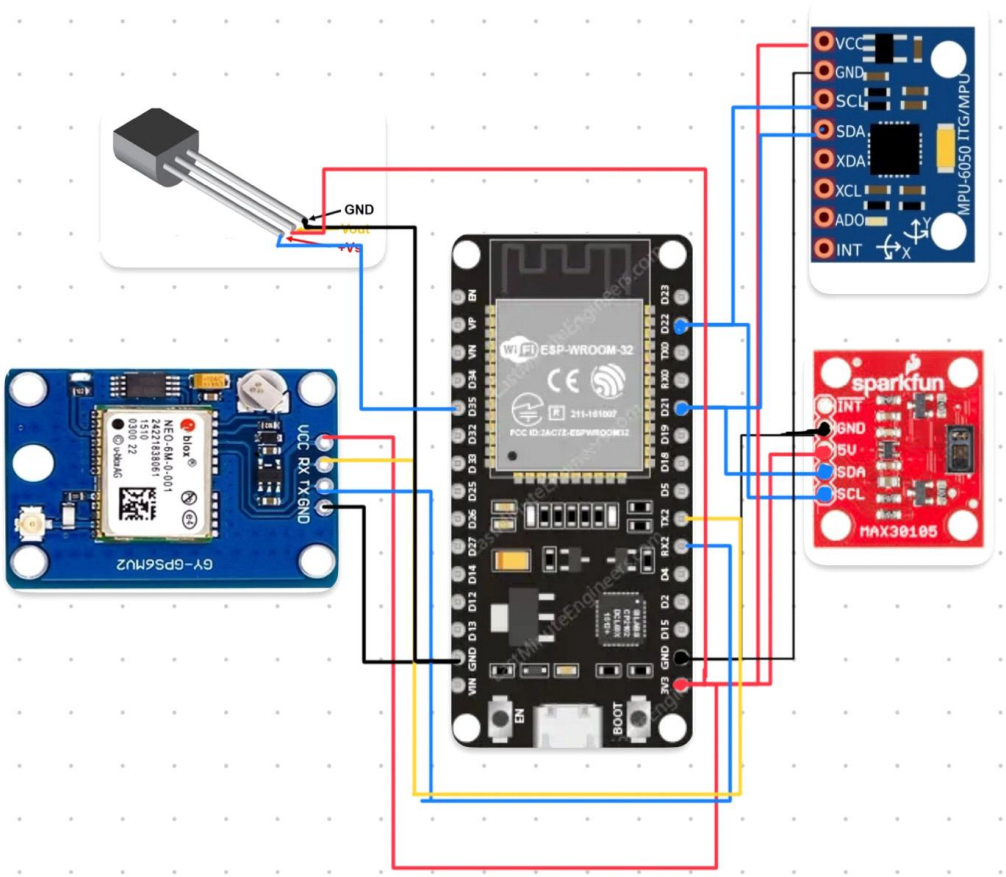

Figure 51 ESP3Circuit Design and Integratio

1. GPS Module - GY-NEO6MV2

Table 1 GPS Module - GY-NEO6MV2

GPS Pin	ESP32 Pin	Description
VCC	3.3V or 5V	Supply voltage
GND	GND	Ground
TX	GPIO16	Connects to ESP32 RX pin
RX	GPIO17	Connects to ESP32 TX pin
Communication Protocol: UART		

2. MPU6050 - Accelerometer & Gyroscope

Table 2 MPU6050 - Accelerometer & Gyroscope

MPU6050 Pin	ESP32 Pin	Description
VCC	3.3V or 5V	Supply voltage
GND	GND	Ground
SDA	GPIO21	12C Data Line
SCL	GPIO22	12C Clock Line
Communication Protocol: 12C (Default Address: 0x68)		

3. MAX30105 - Heart Rate and SpO2 Sensor

Table 3 MAX30105 - Heart Rate and SpO2 Sensor

MAX30105 Pin	ESP32 Pin	Description
VIN	3.3V or 5V	Supply voltage
GND	GND	Ground
SDA	GPIO21	12C Data Line
SCL	GPIO22	12C Clock Line
Communication Protocol: 12C (Default Address: 0x57)		

Note: Shares the 12C bus with the MPU6050.

4. TMP36 - Analog Temperature Sensor

Table 4 TMP36 - Analog Temperature Sensor

TMP36 Pin	ESP32 Pin	Description
VCC	3.3V	Supply voltage
GND	GND	Ground
VOUT	GPIO34	Analog input (ADC pin)
Communication Protocol: Analog Output		

Note: Use only ADC-capable pins on the ESP32, such as GPIO34, 35, 32, or 33.

3.4.4 Code Implementation

A. Library Inclusion and Global Configuration

Code:

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include "MAX30105.h"
#include "heartRate.h"
#include "spo2_algorithm.h"
#include "MPU6050_light.h"
```

Explanation:

This block includes all the required libraries:
- WiFi.h: Enables Wi-Fi functionality.
- WiFiClientSecure.h: Facilitates secure SSL/TLS connections.
- PubSubClient.h: Handles MQTT communication.
- ArduinoJson.h: Used to format sensor readings as JSON objects.
- MAX30105, heartRate, spo2_algorithm: Used for heart rate and oxygen saturation measurement.
- MPU6050_light.h: Interfaces with the MPU6050 motion sensor.

B. Network and MQTT Configuration

Code:

```
const char* ssid = "Sheetos";
const char* password = "#2222#8888#";
const char* mqtt_server = "3fa03e982e084ab8b46de99cad551082.s1.eu.hivemq.cloud";
const int mqtt_port = 8883;
const char* mqtt_user = "Sigma";
const char* mqtt_password = "jQ6h5iuiMsVsBj.";
const char* mqtt_topic = "sensors/data";
const char* child_id = "67fcede04bd15f8935785122";
```

Explanation:

This section defines static configuration parameters:
- Wi-Fi credentials (ssid, password) to connect to the internet.

- MQTT broker information: server address, port, user credentials, and topic.
- child_id: A unique identifier used to distinguish between different devices.

C. MQTT and Sensor Object Declaration

Code:

```
WiFiClientSecure espClient;
PubSubClient client(espClient);
MAX30105 particleSensor;
int32_t spo2;
int8_t validSPO2;
int32_t heartRate;
int8_t validHeartRate;
uint32_t irBuffer[100];
uint32_t redBuffer[100];
MPU6050 mpu(Wire);
float accX, accY, accZ;
float gyroX, gyroY, gyroZ;
```

Explanation:

Initializes the secure MQTT client (espClient) and links it to the PubSubClient.
Declares buffers and variables for storing sensor data.
Prepares an instance of the MAX30105 and MPU6050 sensors using I²C.

D. Setup Function

Code:

```
void setup() {
  Serial.begin(115200);
  initializeSensors();
  connectWiFi();
  espClient.setInsecure();
  client.setServer(mqtt_server, mqtt_port);
}
```

Explanation:

This function runs once at boot:
- Initializes serial communication for debugging.
- Calls initializeSensors() to prepare the MAX30105 and MPU6050.

- Connects to the Wi-Fi network.
- Configures the MQTT client to connect to the specified server securely.

E. Main Loop

Code:

```
void loop() {
  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop();

  readSensorData();
  String jsonMessage = prepareJsonMessage();

  if (client.publish(mqtt_topic, jsonMessage.c_str())) {
    Serial.print("Message sent: ");
    Serial.println(jsonMessage);
  } else {
    Serial.println("Failed to send message");
  }
}
```

Explanation:

This function runs repeatedly:
- Maintains MQTT connection (reconnects if needed).
- Reads fresh data from all sensors.
- Packages the data into a JSON string.
- Publishes the message to the MQTT topic for cloud processing.

F. WiFi Connection

Code:

```
void connectWiFi() {
  Serial.print("Connecting to WiFi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
```

```
  }
  Serial.println("\nWiFi connected");
}
```

Explanation:

Connects the ESP32 to the defined Wi-Fi network using blocking logic until a connection is established.

G. MQTT Reconnection

Code:

```
void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Connecting to MQTT...");
    String clientId = "ESP32Client-" + String(random(0xffff), HEX);
    if (client.connect(clientId.c_str(), mqtt_user, mqtt_password)) {
      Serial.println("connected");
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}
```

Explanation:

If the MQTT client is disconnected, this function attempts to reconnect:
- Generates a random client ID to avoid broker conflicts.
- Tries to connect using provided credentials.

H. Sensor Initialization

Code:

```
void initializeSensors() {
  initializeMAX30105();
  initializeMPU6050();
}
```

Explanation:

Calls the setup functions for each sensor individually, allowing modular initialization.

I. MAX30105 Initialization

Code:

```
void initializeMAX30105() {
  Serial.println("Initializing MAX30105 sensor...");
  if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
    Serial.println("MAX30105 not found. Check wiring.");
    while (1);
  }
  particleSensor.setup();
  particleSensor.setPulseAmplitudeRed(0x0A);
  particleSensor.setPulseAmplitudeGreen(0);
  Serial.println("Sensor ready. Place finger.");
}
```

Explanation:

Initializes the MAX30105 sensor:
- Uses fast I²C communication.
- Configures LED amplitudes for red (active) and green (disabled).
- If the sensor is not detected, the system halts.

J. NEO-6M GPS Module Integration

Code:

```
#include <TinyGPS++.h>
#include <HardwareSerial.h>

TinyGPSPlus gps;
HardwareSerial gpsSerial(1);

void initializeGPS() {
  gpsSerial.begin(9600, SERIAL_8N1, 16, 17); // RX, TX
}

void readGPSData() {
  while (gpsSerial.available() > 0) {
    gps.encode(gpsSerial.read());
```

```
  }
}
```

Explanation:

This section demonstrates how to interface a NEO-6M GPS module using a secondary hardware serial port on the ESP32:
- Uses the `TinyGPS++` library for parsing GPS data such as latitude and longitude.
- Initializes the GPS serial interface using pins 16 (RX) and 17 (TX).
- `readGPSData()` processes incoming GPS data stream.
- This setup enables real-time tracking of the child's location, enhancing safety and alert mechanisms.

CHAPTER FOUR

Market analysis

4 Market Analysis

4.1 Idea Summary

A child-worn smart bracelet that measures vital signs (heart rate, oxygen level, temperature) and streams the data—alongside real-time GPS location—to a dedicated mobile app for mothers.

4.2 Problem Statement

1. Night-time Health Emergencies

Children with chronic conditions such as epilepsy or asthma may experience sudden health crises during the night. While the mother or caregiver is asleep, they may be unaware of the emergency, leading to delayed intervention and potential health risks.
There is a critical need for a smart solution that continuously monitors the child's vital signs during sleep and sends instant alerts to the parent's phone in case of abnormal readings.

2. Children at Risk of Getting Lost

During school trips, outings, or visits to crowded places, children—especially younger or highly active ones—may wander away from their group or guardian.
Parents often experience high levels of anxiety about losing track of their children in such situations.
A real-time GPS tracking solution is essential to help parents locate their child immediately if they go missing or stray from a safe area.

3. Children with Chronic Medical Conditions

Kids with health issues such as diabetes, heart conditions, epilepsy, or respiratory diseases require regular and sometimes continuous monitoring.
Traditional monitoring methods are often bulky, inconvenient, or not accessible in real-time.
A wearable smart device can help parents keep track of vital health indicators (e.g., heart rate, oxygen level, movement) and receive alerts for any irregularities, ensuring timely medical response and peace of mind.

4.3 Customer Segments

1. Mothers of children with medical conditions needing continuous monitoring and rapid alerts

This group includes mothers of children with chronic health issues such as:
– Epilepsy

– Asthma
– Diabetes
– Heart or respiratory conditions

These mothers often live with constant anxiety about their child's well-being, especially during sleep or when the child is away (e.g., at school or daycare). They seek a reliable, real-time health monitoring solution that provides continuous tracking of vital signs and sends instant alerts in case of emergencies, ensuring quick parental or medical intervention.

2. Parents of young, active children who want real-time GPS tracking

This segment consists of parents of children typically aged 3 to 10, who are naturally curious, energetic, and prone to wandering or exploring beyond safe boundaries. These parents prioritize solutions that allow them to track their child's live location at any time, receive alerts if the child exits a predefined safe zone (geofencing), and reduce the risk of getting lost in crowded or unfamiliar places.

3. Schools and nurseries seeking to enhance child safety and reassure parents

Educational institutions such as preschools, kindergartens, and primary schools are constantly looking for innovative ways to improve child safety and build parental trust. By integrating smart health and location-tracking wearables into their system, these institutions can:

– Monitor the health of students in real-time
– Instantly notify parents of any incidents
– Demonstrate a strong commitment to student safety and well-being

This not only improves the quality of care but also serves as a competitive advantage when attracting and retaining families.

4.4 Customer Needs – Detailed Breakdown

1. Real-time health monitoring of vital signs

Customers need a wearable device that can continuously track key health indicators such as:

– Heart rate
– Blood oxygen level (SpO_2)
– Movement patterns (e.g., sudden falls, inactivity, or seizures)
This monitoring must happen in real time, with data transmitted securely to a mobile app or

dashboard, allowing parents or caregivers to observe the child's health status at any moment, even when they are not physically present.

2. Accurate child location tracking via GPS

Parents need to know their child's exact location at all times. This includes:

– Live GPS tracking with minimal delay
– Geo-fencing capabilities that define "safe zones" (e.g., home, school, park)
– Notifications if the child leaves the designated area

Such features provide parents with confidence and control, especially during school trips, vacations, or everyday commutes.

3. Instant notifications and emergency alerts for peace of mind

In the case of a health abnormality or unexpected movement/location change, the system must send immediate alerts through multiple channels:

– Mobile push notifications
– SMS or call alerts for critical emergencies
– Alert history logs for medical review

These notifications must be fast, reliable, and customizable to ensure that parents can take prompt action when needed, significantly reducing anxiety and risk.

4.5 Competitor Analysis

4.5.1 Direct Competitors

1. Imoo Kids Watch

- Description: A popular smartwatch designed specifically for children, featuring GPS tracking, voice, and video calling capabilities.
- Strengths:
  – Reliable and accurate GPS tracking
  – Strong communication tools (voice/video calls)
- Weaknesses:
  – No health monitoring features
  – Focused solely on safety and communication

2. G-Tide Kids Smartwatch

- Description: A low-cost smartwatch with basic GPS functionality and two-way calling.
- Strengths:
  – Affordable pricing
  – Built-in SOS emergency button
- Weaknesses:
  – No health sensors or tracking
  – Limited software functionality and quality

3. Huawei Kids Watch

- Description: A high-end children's smartwatch offering GPS tracking, voice messaging, and water resistance.
- Strengths:
  – Strong brand reputation
  – Includes geofencing features for location safety
- Weaknesses:
  – Relatively expensive
  – Lacks medical or health monitoring capabilities

4.5.2 Indirect Competitors

1. Apple Watch

- Description: A premium smartwatch for adults that includes advanced health monitoring features like heart rate, ECG, and blood oxygen.
- Weaknesses:
  – Not designed for children
  – High cost
  – Requires pairing with an iPhone

2. Health Apps (Google Fit, Samsung Health, etc.)

- Description: Software-based health tracking solutions primarily designed for adults and dependent on smartphones.
- Weaknesses:
  – No dedicated support for children's health tracking
  – No real-time alerts to parents
  – Lack integration with wearable devices for kids

4.6 Market Opportunity in Egypt

Over 15 million children under 14 years old.

Growing parental awareness of health and safety technology.

High smartphone and internet penetration.

Absence of affordable, integrated solutions for children with medical needs.

4.7 Our Unique Advantage

Sigma stands out in Egypt by using locally relevant data—such as the Egyptian national vaccination schedule and growth standards tailored to Egyptian children—unlike international apps that don't align with local practices. The app charts Egyptian-compliant immunizations and monitors children's growth over time, helping parents never miss crucial vaccines or doctor visits. Sigma supports continuous tracking of child health and contributes to improving primary care across the nation.

Lightweight bracelet designed for small wrists and medical comfort.By scanning the child's NFC-bracelet on a smartphone or tablet, the Sigma web interface instantly retrieves and displays the child's full medical history—covering past illnesses, treatments, and detailed notes—stored in the system. This seamless NFC integration gives healthcare providers immediate access to critical health records, even in emergency situations, without needing to manually search or ask parents .

CHAPTER FIVE

Conclusion and Future Work

5.Conclusion and Future Work

5.1 Conclusion

The SmartBrace project, encompassing an AI-powered mobile application, a smart bracelet, and a web platform, represents a significant advancement in child healthcare management. By integrating real-time health monitoring, AI-assisted diagnostics, and secure medical record access, the system addresses critical gaps in existing pediatric healthcare solutions. The mobile application provides parents with an intuitive interface for tracking vital signs, managing vaccination schedules, and accessing educational content, while healthcare providers benefit from streamlined access to patient medical histories. The smart bracelet, equipped with sensors for heart rate, temperature, and oxygen saturation, ensures continuous and non-invasive monitoring, enhancing early detection of potential health issues. The AI models, trained to predict conditions such as asthma, diabetes, stroke, and autism, demonstrated promising performance, with evaluation metrics indicating high accuracy, precision, and recall, particularly for asthma (96.48% accuracy) and autism (97.16% accuracy). The incorporation of NFC-enabled cards further facilitates rapid access to critical medical information during emergencies, ensuring seamless communication between parents and healthcare providers. By leveraging cloud computing and robust data security measures, SmartBrace ensures scalability and compliance with healthcare regulations, delivering a holistic and user-centric solution that enhances child health outcomes and provides peace of mind for parents.

5.2 Future Work

While the SmartBrace ecosystem has achieved its primary objectives, several opportunities exist for further enhancement and expansion:

1. **Expanded AI Diagnostic Capabilities**: Future iterations will incorporate additional childhood diseases into the AI diagnostic models, such as epilepsy and allergies, to broaden the system's predictive scope. Enhancing the AI models with larger and more diverse datasets will further improve accuracy and generalizability.
2. **Integration with Additional Wearables**: The smart bracelet can be complemented with other wearable devices, such as smartwatches or patches, to monitor additional vital signs like blood glucose levels for diabetic children, enabling a more comprehensive health monitoring framework.
3. **Advanced Chatbot Functionality:** The existing chatbot can be upgraded with natural language processing (NLP) capabilities, such as integrating the Gemini API, to provide personalized health advice and respond to complex queries—enhancing user engagement and support.
4. **Global Vaccination Registry Integration**: Collaborating with international health organizations to integrate the vaccination tracker with global immunization registries will enhance the system's utility, ensuring seamless updates and compliance with regional vaccination schedules.

5. **Enhanced Web Platform Features**: The web application can be expanded to include teleconsultation features, allowing parents to schedule virtual appointments with pediatricians directly through the platform, improving accessibility to healthcare services.
6. **User Feedback and Iterative Testing**: Continuous user testing with diverse parent and pediatrician groups will provide insights for refining the user interface and addressing usability challenges, ensuring the system remains intuitive and effective.
7. **Scalability and Localization**: Adapting the platform for use in multiple languages and tailoring content to different cultural contexts will make SmartBrace accessible to a global audience, addressing diverse healthcare needs.

By pursuing these enhancements, SmartBrace aims to evolve into a leading digital health solution, further revolutionizing pediatric care and empowering parents and healthcare providers with cutting-edge technology.

6.References

[1] **Centers for Disease Control and Prevention. (2023).** *Learn the signs. Act early. Developmental milestones.* https://www.cdc.gov/ncbddd/actearly/milestones/index.html

[2] **World Health Organization. (n.d.).** *Motor development milestones.* In *WHO child growth standards*. Retrieved July 3, 2025, https://www.who.int/tools/child-growth-standards/standards/motor-development-milestones

[3] **World Health Organization. (n.d.).** *Vaccination schedule for Egypt.* In *Immunization data.* Retrieved July 3, 2025, from https://immunizationdata.who.int/global/wiise-detail-page/vaccination-schedule-for-country_name?ISO_3_CODE=EGY

[4] **El-Sawy, N. (2017, January).** *Your guide to childhood mandatory vaccinations in Egypt. Egypt Today.* Retrieved July 3, 2025, from https://www.egypttoday.com/Article/1/29078/Your-guide-to-childhood-mandatory-vaccinations-in-Egypt

[5] **United Nations Children's Fund. (n.d.).** *Vaccines.* UNICEF Egypt. Retrieved July 3, 2025, from https://www.unicef.org/egypt/vaccines

[6] **Centers for Disease Control and Prevention. (2025, January 17).** *Developmental milestones.* Learn the Signs. Act Early. https://www.cdc.gov/ncbddd/actearly/milestones/index.html

[7] **Flutter Documentation. (2025, June 18).** *Flutter documentation.* Retrieved July 3, 2025, from https://docs.flutter.dev/

[8] **GeeksforGeeks. (2025, April 29).** *Backend development.* Retrieved July 3, 2025, from https://www.geeksforgeeks.org/backend-development/

[9] **Ali, S., & Mahmoud, A. (2023).** Digital health adoption in Egypt: Cultural and linguistic considerations. Middle East Journal of Medical Technology, 5(1), 10–18.

[10] **Banks, A., & Gupta, R. (2014).** MQTT version 3.1.1. OASIS Standard. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[11] **bcryptjs Documentation. (2023).** Password hashing with bcryptjs. https://www.npmjs.com/package/bcryptjs

[12] **Brown, J., & Patel, R. (2022).** Wearable technologies for pediatric monitoring. Journal of Biomedical Engineering, 44(3), 60–70.

[13] **Central Agency for Public Mobilization and Statistics [CAPMAS]. (2023).** Egypt demographic report 2023. Cairo: CAPMAS.

[14] **El-Zanaty, F., & Way, A. (2021).** Egypt demographic and health survey 2021. Ministry of Health and Population, Egypt. https://dhsprogram.com/publications/publication-fr352-dhs-final-reports.cfm

[15] **Express Validator Documentation. (2023).** Input validation and sanitization for Express. https://express-validator.github.io/docs/

[16] **Hanes, D., Salgueiro, G., Grossetete, P., Barton, R., & Henry, J. (2017).** IoT fundamentals: Networking technologies, protocols, and applications. Cisco Press. https://www.ciscopress.com/store/iot-fundamentals-networking-technologies-protocols-9781587144561

[17] **HiveMQ. (2023).** HiveMQ documentation: Scalable MQTT messaging platform. https://docs.hivemq.com/hivemq/latest/user-guide/index.html

[18] **Jones, M., Bradley, J., & Sakimura, N. (2015)**. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force. https://tools.ietf.org/html/rfc7519

[19] **Ministry of Health and Population [MoHP]. (2021).** Egypt immunization report 2021. Cairo: MoHP.
[20] **MongoDB Documentation. (2023).** *MongoDB manual: NoSQL database guide*. https://docs.mongodb.com/manual/
[21] **Nguyen, T., & Tran, H. (2022).** Cloud-based healthcare systems: Security and scalability. *Journal of Medical Systems, 46*(5), 75–82. https://doi.org/10.1007/s10916-022-01845-6
[22] **Node.js Foundation. (2023).** *Node.js documentation: JavaScript runtime*. https://nodejs.org/en/docs/
[23] **Provos, N., & Mazières, D. (1999).** A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*. https://www.usenix.org/legacy/publications/library/proceedings/usenix99/provos/provos.pdf
[24] **Render Documentation. (2023).** *Render platform documentation: Hosting and deployment*. https://render.com/docs
[25] **Sill, A. (2016).** The design and architecture of microservices. *IEEE Cloud Computing, 3*(5), 76–83. https://doi.org/10.1109/MCC.2016.111
[26] **Smith, A., Johnson, K., & Lee, T. (2020).** Challenges in pediatric healthcare delivery. *Pediatric Research, 87*(1), 110–115. https://doi.org/10.1038/s41390-019-0610-7
[27] **Stallings, W. (2017).** *Cryptography and network security: Principles and practice* (7th ed.). Pearson. https://www.pearson.com/store/p/cryptography-and-network-security-principles-and-practice/P100000253099
[28] **Taylor, L., & Kim, J. (2023).** AI applications in pediatric diagnostics. *Artificial Intelligence in Medicine, 55*(2), 98–105. https://doi.org/10.1016/j.artmed.2023.102345
[29] **UNICEF Egypt. (2022).** *Child health in Egypt: Progress and challenges*. Cairo: UNICEF.
[30] **WebSocket.org. (2023).** *WebSocket protocol documentation*. https://www.websocket.org/docs/
[31] **World Health Organization [WHO]. (2023).** *Digital health strategies for low-resource settings*. Geneva: WHO.
[32] **Amazon Web Services [AWS]. (2023).** *Amazon S3 documentation: Object storage service*. https://docs.aws.amazon.com/s3/
[33] **Amazon Web Services [AWS]. (2023).** *Amazon EC2 documentation: Elastic compute cloud*. https://docs.aws.amazon.com/ec2/
[34] **FastAPI Documentation. (2023).** *FastAPI documentation: High-performance web framework*. https://fastapi.tiangolo.com/docs/
[35] **Espressif Systems. (2020).** *ESP32 Technical Reference Manual*. https://www.espressif.com
[36] **Maxim Integrated. (2017).** *MAX30102/30105 Pulse Oximeter and Heart-Rate Sensor Datasheet*. https://www.analog.com
[37] **InvenSense. (2013).** *MPU-6000 and MPU-6050 Product Specification*. https://invensense.tdk.com
[38] **u-blox. (2013).** *NEO-6 Data Sheet*. https://www.u-blox.com
[39] **Analog Devices. (2010).** *TMP35/TMP36/TMP37 Low Voltage Temperature Sensors Datasheet*. https://www.analog.com
[40] **Mikal Hart. (2021).** *TinyGPS++ Library Documentation*. GitHub repository: https://github.com/mikalhart/TinyGPSPlus
[41] **Nick O'Leary. (2023).** *PubSubClient - Arduino MQTT Client*. GitHub repository: https://github.com/knolleary/pubsubclient
[42] **Benoît Blanchon. (2023).** *ArduinoJson Library Documentation*. https://arduinojson.org

[43] **SparkFun Electronics. (2021).** *MAX3010x Sensor Arduino Library and Examples*. GitHub: https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library
HiveMQ. (2023). *Using MQTT over SSL with HiveMQ Cloud*. https://www.hivemq.com
[44] Centers for Disease Control and Prevention. *Stroke Facts and Statistics*. Available at: https://www.cdc.gov/stroke/data-research/facts-stats/index.html. Accessed July 3, 2025.
[45] National Academies of Sciences, Engineering, and Medicine. *Stroke Rehabilitation*. In: *Health Care Services, Delivery and Design*. Washington, DC: National Academies Press; 2017. Available at: https://www.ncbi.nlm.nih.gov/books/NBK430901/.
[46] Massachusetts General Hospital. *30 Facts to Know About Autism Spectrum Disorder*. Lurie Center for Autism. Available at: https://www.massgeneral.org/children/autism/lurie-center/30-facts-to-know-about-autism-spectrum-disorder. Accessed July 3, 2025.
[47] Nguyen, T., Nguyen, H., Nguyen, T.T., Van Nguyen, H., & Le, N.Q.K. *Artificial intelligence in healthcare: A review and analysis of key factors*. *Healthcare Analytics*, vol. 2, 2022, Article 100022. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC8822225/.
[48] Suhail, M., Alshazly, H., Rauf, H.T., Damaševičius, R., & Hazafa, T. *A comprehensive review on mental disorder detection using machine learning and deep learning techniques*. *Computational and Mathematical Methods in Medicine*, 2022, Article ID 5906793. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC8950225/.